

Zuma VDAS2

Порт под FT812 / 640×480

Учебник по разработке

ZX-Evo · TS-Conf · Z80 · FT81x

Сквозной пример: классическая Zuma на нативном железе

Версия: baseline v041 (2026-05-26) · 31 глава

Содержание

Содержание

- Структура учебника
- 1. Аппаратная связка ZX-Evo + VDAC2
 - 1.1. Что такое VDAC2
 - 1.2. Порты TS-Conf для общения с FT812
 - 1.3. VCONFIG для VDAC2-режима
- 2. SPI-протокол FT812
 - 2.1. Три типа транзакций (по 2-битному префиксу)
 - 2.2. Каждая транзакция в обёртке CS
 - 2.3. Готовые asm-функции (из учебника #2)
 - 2.4. Когда размер блока удобен
- 3. Memory Map FT812
 - 3.1. Ключевые регистры RAM_REG
 - 3.2. DLSWAP modes
- 4. Видеотайминги для 640×480
 - 4.1. VM_640_480_57Hz (PCLK 24 MHz, F_MUL=3) — целевой для Zuma
 - 4.2. VM_640_480_74Hz (PCLK 32 MHz, F_MUL=4) — повышенная частота
 - 4.3. VM_640_480_76Hz (PCLK 32 MHz, F_MUL=4)
 - 4.4. Соответствие регистрам FT812
 - 4.5. Применение через TSLib-макрос
 - 4.6. Полная init-последовательность (TSLib FT_BOOT_UP)
- 5. Display List
 - 5.1. Структура
 - 5.2. Базовые opcodes (минимальный набор для Zuma)
 - 5.3. Минимальный «Hello World» DL
 - 5.4. Цепочка спрайтов из атласа
 - 5.5. Co-processor (RAM_CMD) — для удобства
- 6. Bitmap-форматы для FT812
- 7. Производительность и DMA
 - 7.1. Оценка bandwidth Z80 → FT812 через SPI
 - 7.2. Полный кадр при 60 fps
 - 7.3. NO_GFX=1 экономит DMA
- 8. План разработки Zuma VDAC2 (контекст)
- 9. Главный цикл рендера (Hello World pattern из TSLib)
 - 9.1. Что делает FT_CMD_Start/FT_CMD_Write

- 9.2. FT_DLSWAP_FRAME / FT_INT_SWAP
- 10. TSLib API — карта макросов
 - 10.1. Низкий уровень: Include\FT\812 Macro.inc
 - 10.2. Прямой Display List в RAM_DL: Include\FT\DL Macro.inc
 - 10.3. Сборка DL через co-processor: Include\FT\Coprocessor\BufferMacro.inc
 - 10.4. Coprocessor функции (Include\FT\Coprocessor\Buffer.asm + Cmd.asm)
 - 10.5. Прочее
 - 10.6. Готовый Init_Video для Zuma VDAC2
 - 10.7. Готовый MainLoop для Zuma VDAC2 (каркас)
- 11. Открытые вопросы / TODO для углубления учебника
- Глава 12. Bitmap rendering — matrix transform, scale, paletted formats (опыт 2026-05-09)
 - 12.1 Главный урок: BITMAP_TRANSFORM работает на bitmap UV, не на screen position
 - 12.2 cmd_scale convention
 - 12.3 BITMAP_SIZE при upscale
 - 12.4 Memory budget для bg (1 MB RAM_G FT812)
 - 12.5 Asymmetric downscale (X≠Y)
 - 12.6 spgbld page-padding gotcha
 - 12.7 Compression: PNG/JPEG
 - 12.8 Финальный выбор для Zuma VDAC2 (level 1 spiral)
 - 12.8.1 Полный pipeline компрессии bg (нюансы практики)
 - 12.8.2 Bug retro: bg-padding затирает atlas (#0A6000..#0A8000)
- Глава 13. Frog composition: HD-стиль pipeline (опыт 2026-05-09/10)
 - 13.1 Источники подспрайтов в frog.png
 - 13.2 Render pipeline (Frog_Draw порядок)
 - 13.3 Rotation matrix pattern (см. также §12.1)
 - 13.4 Atan2 от курсора → angle
 - 13.5 Hybrid follow для плавного rotation
 - 13.6 Tongue bbox (для будущего расчёта tongueExpand)
- Глава 14. RNG: LFSR Galois + bias + RTC-scramble (опыт 2026-05-10)
 - 14.1 LFSR Galois 16-bit
 - 14.2 Bias-ловушка: AND N + clamp на не-степени двойки
 - 14.3 Mul-then-shift: равномерное распределение
 - 14.4 RTC-scramble seed (для разнообразия per launch)
- Глава 15. Frog с полной HD-композицией (2026-05-10)
 - 15.1 Render order (HD-1:1)
 - 15.2 RAM_G layout (1 МБ, baseline 2026-05-10)
 - 15.3 Tongue — pos + tongueExpand-dir (HD orbit)
 - 15.4 Ball-now / Next-ball через chain atlas (handle o)
 - 15.5 Recoil cycle (HD-style fire animation)

- 15.6 FT81x cmd_scale: matrix хранит INVERSE
- 15.7 Critical bugs found and fixed (2026-05-10 session)
- 15.8 Python visual_emulator.py — prototype-first workflow
- Глава 16. FT81x DL persistent state — Cell, BITMAP_HANDLE и ловушки наследования (2026-05-10)
 - Что мы исключили (типичные кандидаты, оказавшиеся неверными)
 - Root cause — Cell как persistent DL state
 - Fix
 - Универсальное правило: какой DL state в FT81x persists
 - Методология поиска
- Глава 17. Vsync-first sync: race между Z80 build и FT812 render (2026-05-10)
 - Гипотезы и проверка
 - Решение — parallel build + vsync-first write
 - Почему это работает
 - Урок (универсальный)
 - Открытое (TODO для следующей итерации)
- Глава 18. DXT1-эмуляция на FT812: компрессия фона до 0.5 байт/пикс через L2-mask + RGB565 blend (2026-05-12)
 - Задача
 - Идея
 - Формат raw файла
 - L2 alpha mapping (нелинейный)
 - Display List — 3 прохода
 - Подводные камни (на отладку ушёл вечер)
 - Сравнение объёмов 640×480
 - Когда использовать
 - Конвертер ft812_dxt_convert.py
 - Multi-level в Zuma — что меняется
 - Источники
- Глава 19. Апгрейд DXT1-эмуляции с L2 до L4: +50% SPI за фотокачество (2026-05-12)
 - Зачем понадобился L4
 - Сравнение L2 vs L4 в одном блоке
 - Что меняется в raw layout
 - Изменения в asm (минимально)
 - Подводный камень: CPU энкодер на Windows нежизнеспособен
 - Сплит в spgbld pages
 - Итоговый бюджет
 - Когда выбирать L2 vs L4
 - Источники
- Глава 20. Render-loop оптимизации и DL-emit ловушки (2026-05-17)

- 20.1 Bucket-grouped tangent rotation: 32 cmd_rotate → 16, а потом обратно
- 20.2 Per-sprite alpha fade через COLOR_A — плавное поглощение
- 20.3 Cell/ColorA корраптят BC/DE — координаты грузить ПОСЛЕ, а не ДО
- 20.4 Скип лишнего DL: bg-baked = overlay не нужен
- 20.5 Continuous-motion absorb через HSub-advance (mirror of fast-spawn)
- Источники
- Глава 21. Per-ball matrix с per-slot hysteresis и grouped emit (2026-05-18)
 - 21.1 Постановка задачи
 - 21.2 Альтернатива #1: бакеты — почему не подошло
 - 21.3 Альтернатива #2: чистый per-ball — почему сломалось на реале
 - 21.4 Гибрид: per-slot hysteresis + run-length grouped emit
 - 21.5 Ловушки реализации
 - 21.6 RNG bias как побочный bug (2026-05-18)
 - 21.7 Применимость в других случаях
 - Источники
- Глава 22. Расщепление Core на main0 + main1 (slot 1 + slot 3) и невидимая ловушка CMD_ADDRESS_PTR (2026-05-18)
 - 22.1 Проблема: лимит "считать байты Core" 9216
 - 22.2 Архитектура split
 - 22.3 Cross-slot CALL работает без thunks
 - 22.4 Ловушка 1: Main0_End в неправильном месте
 - 22.5 Ловушка 2: TSLib CMD_ADDRESS_PTR = #C000 молча затирает main1
 - 22.6 Чек-лист split-проекта
 - 22.7 Запас на будущее
 - Источники
- Глава 23. Экономия RAM_G шаров: путь к PALETTED4444 (v025)
 - 23.1 Постановка задачи
 - 23.2 Опции форматов FT812
 - 23.3 Неудачные попытки (mid-may 2026)
 - 23.4 PALETTED4444: три условия
 - 23.5 Setup PALETTED4444 (baseline v025)
 - 23.6 Результаты v025 (HW-verified)
 - 23.7 Урок: «не работает» ≠ «формат не поддерживается»
 - Источники
- Глава 24. Почему отказались от псевдо-DXT фона (2026-05-19)
 - TL;DR
 - Что делал псевдо-DXT (краткое напоминание)
 - Симптом на реале — «срыв строчной»
 - Реальная причина — пиксельный клок-бюджет на строку
 - Решение — один полноэкранный проход

- Что потеряли по сравнению с псевдо-DXT
- Почему не разделили на полосы
- Bigger picture: правило бюджета строки
- Когда стоит вернуться к псевдо-DXT
- Уточнение модели (ground-truth через эмулятор, см. главу 25)
- Источники
- Глава 25. Bridgetek EveApps FT812 Emulator — настоящая эмуляция чипа (2026-05-19)
 - Зачем понадобился новый эмулятор
 - Что выбрали
 - Установка (Codex, 2026-05-19)
 - Что делает ZumaPlayback.c
 - Bundle export (Python side)
 - Запуск эмулятора
 - Что дало в v028
 - Подход playback vs interactive
 - Readback реального RAM_DL — ground-truth аудит Display List (доводка)
 - Что осталось сделать
 - Ссылки
- Глава 26. BITMAP_HANDLE binding ловушка FT812 (2026-05-20)
 - TL;DR
 - Семантика BITMAP_HANDLE в FT812
 - Bad pattern (rainbow noise!)
 - Good pattern
 - Защита от случайной перестановки
 - Когда баг проявляется
 - Почему Unreal x64 HE видит этот баг
 - Ссылки
- Глава 27. Сессия 2026-05-20: scoring engine, matrix LUT, ARGB4 frog → tearing fix
 - 27.1 Match-3 «синие шары вместо gap» — критический bug
 - 27.2 Scoring engine 1:1 с HD-ref Statistics.c
 - 27.3 Точный fill_px для прогресс-бара
 - 27.4 FT_ScissorXY клобает B — повторение pattern
 - 27.5 GaugeShown animated LERP — плавный бар
 - 27.6 Pre-baked rotation matrix LUT
 - 27.7 OK button hit-test + Fire trigger
 - 27.8 MENU sprite skip baked при idle
 - 27.9 Главный фикс tearing: ARGB4 + NEAREST для frog
 - 27.10 Outcome дня
- Глава 28. Mr.Gluk RTC чтение и часы на экране (2026-05-20)
 - 28.1 Постановка

- 28.2 Диагностика
- 28.3 Правильная процедура чтения Mr.Gluk
- 28.4 Применение к Zuma VDAC2
- 28.5 Часы в нижней рамке
- 28.6 Фикс TIME в Game Over dialog
- 28.7 Что в memory
- Глава 29. Когда эмулятор сам с багом — горький урок (2026-05-20)
 - 29.1 Симптомы
 - 29.2 Корень проблемы — Python эмулятор фазово drift'ил от ASM
 - 29.3 Кто закрыл — Codex через RAM dump
 - 29.4 Применённые ASM-фиксы
 - 29.5 Что синхронизировано в Python эмуляторе
 - 29.6 Урок
 - 29.7 Память на будущее
- Глава 30. FAT32 с SD-карты в TS-Conf: от WC ZiFi к собственному драйверу RawPak (CMD17 + LFN)
 - 30.0 Зачем эта глава
 - 30.1 Историческая развилка: почему отказались от WC ZiFi
 - 30.2 RawPak: собственный FAT32-ридер на прямом CMD17
 - 30.3 BPB и открытие тома — RawPak_OpenRoot
 - 30.4 LFN-сопоставление пути /Games/Zuma Deluxe VDAC2/ZUMALVL.PAK
 - 30.5 FAT-цепочка: FatNext + ловушка AdvanceOne
 - 30.6 Таблица секторов: LBA = PakLba + N (допущение непрерывности)
 - 30.7 Двухфазная загрузка: FT812 и SD на одной SPI-шине
 - 30.8 Трек на 2 страницы (#06 + #0F) — верхние уровни
 - 30.9 Инструмент: локальный Z80-харнесс FAT (не гонять хост зря)
 - 30.10 Pak-формат ZUMALVL.PAK
 - 30.11 Миф «размер SPG ломает загрузку» — опровергнут
 - 30.12 Ловушки (anti-patterns)
 - 30.13 Build pipeline
 - 30.14 Связано
- Глава 31. Adventure-режим: уровни из таблицы, перенос счёта, Win/Pause (v035–v041)
 - 31.1 Поток adventure
 - 31.2 Таблица параметров уровня → геймплей
 - 31.3 Отсечка хвоста (gauge) — два пойманных бага
 - 31.4 Накопительный счёт adventure
 - 31.5 Win-flow «LEVEL DONE»
 - 31.6 Пауза-fade
 - 31.7 Интро уровня: динамичный «LEVEL X-X»
 - 31.8 Заметки по бюджету страниц
 - 31.9 Связано

- Глава 32. Опрос клавиатуры (Mr.Gluk PS/2) и единый глобальный модуль ввода (v044, 2026-05-27)
 - 32.1 Железо: расширенная PC-клавиатура через Mr.Gluk Z-контроллер
 - 32.2 Архитектура: один резидентный модуль Input.asm
 - 32.3 Схема управления и объединение источников
 - 32.4 Лягушка: убрали хрупкий хак с FM_EN
 - 32.5 Навигация по меню: детектор фронта Input_EdgeZ
 - 32.6 Раскладка по сценам
 - 32.7 Связано
- 33. General Sound на TS-Config: музыка MOD и SFX из PAK
 - 33.1. Порты и базовый handshake
 - 33.2. Загрузка MOD музыки
 - 33.3. Возврат в меню без повторного BASS_MusicLoad
 - 33.4. SFX pack и почему нельзя сбрасывать GS перед gameplay
 - 33.5. Частоты sample и скорость проигрывания
 - 33.6. Устранение щелчков/хвоста sample
 - 33.7. Практические правила

Учебник «Программирование TS-Conf + VDAC2 для ZX-Evo»

Накопительный конспект материала. По мере разбора датшитов, источников и собственных экспериментов — здесь оседают факты, формулы, код-примеры, схемы и решения. Из этого вырастет полноценный учебник.

Целевая аудитория: разработчики на Z80 (sjasmplus / asm), знакомые с TS-Conf, расширяющие свой код на работу с FT812 через VDAC2.

Источники, осмысленные на сегодня (локальные копии — в [Docs/](#); ссылки — на оригиналы): - [VDAC2 #2 - Первые шаги.docx](#) — русскоязычный учебник #2 (порты ZX-Evo, SPI-обвязка, Z80 asm-функции FT_RD/FT_WR). Железо VDAC2 — TS-Labs / ZX-Evolution. - [FT81x.pdf](#) — Bridgetek **FT81X Embedded Video Engine Datasheet** (memory map, регистры, RGB-тайминги): https://brtchip.com/wp-content/uploads/Support/Documentation/Datasheets/ICs/EVE/DS_FT81x.pdf - [FT81X_Series_Programmer_Guide.pdf](#) — Bridgetek **FT81X Series Programmer Guide** (opcode-таблицы DL/coprocessor): https://brtchip.com/wp-content/uploads/Support/Documentation/Programming_Guides/ICs/EVE/FT81X_Series_Programmer_Guide.pdf - [BRT_AN_033_BT81X-Series-Programming-Guide.pdf](#) — Bridgetek **BT81X Series Programming Guide** (расширение для совместимой BT81x; ASTC, CMD_FLASH): <https://brtchip.com/wp-content/uploads/2023/12/BT81X-Series-Programming-Guide.pdf> - [AN_303_FT800_Image_File_Conversion.pdf](#) — конвертация изображений в FT-форматы (Bridgetek App Notes: <https://brtchip.com/document/application-notes/>). - AN_340 — Bridgetek App Note про DXT1-эмуляцию на EVE (основа глав 18–19; см. портал App Notes выше). - [The_Gameduino_2_Tutorial,_Reference_and_Cookbook](#) — J. Bowman, 2013 (высокоуровневое API EVE через Gameduino-обёртку + Cookbook): PDF https://excamera.com/files/gd2book_vo.pdf, исходник книги <https://github.com/jamesbowman/gd2-book>. - [TSLib](#) — готовая asm-библиотека для ZX-Evo + FT812 (DeadlyKom). Лежит локально в [Docs\TSLib\](#) (исходники + полная FTDI-документация в [Docs\TSLib\FT812\Docs\](#)). Главный практический референс. FT812-SDK для ZX-Evo: <https://hype.retrosce.org/blog/734.html>; общий репозиторий ZX-Evolution: <https://github.com/tslabs/zx-evo>. - Bridgetek EveApps* — официальные демо + эмулятор EVE (используется в главе 25): <https://github.com/Bridgetek/EveApps>.

Структура учебника

Учебник состоит из двух частей. Подробное оглавление со ссылками генерируется автоматически (раздел «Содержание» выше в HTML/PDF).

Часть I. Основы FT812 / VDAC2 (главы 1–11) — фундамент, выведенный из датшитов и TSLib: аппаратная связка ZX-Evo+VDAC2, SPI-протокол, memory map, видеотайминги 640×480, Display List, bitmap-форматы, производительность/DMA, главный цикл рендера и карта TSLib API. Читается линейно как введение.

Часть II. Журнал разработки Zuma (главы 12–32) — практический опыт в хронологическом порядке: каждая глава фиксирует конкретную задачу/баг/решение с датой и ссылками на код, baseline и память. Главы самодостаточны — можно читать выборочно по теме. Сквозные темы:

- **Рендеринг и оптимизация DL:** 12 (bitmap matrix/scale/paletted), 16 (persistent DL state), 17 (vsync-first), 20 (render-loop приёмы), 21 (адаптивная группировка матриц шаров), 23 (PALETTERD4444 шаров), 24 (бюджет строки FT812), 26 (BITMAP_HANDLE binding), 27 (matrix LUT, ARGB4 frog → fix tearing).
- **Фон уровня:** 18–19 (DXT1-эмуляция L2/L4), 24 (почему перешли на единый PALETTERD4444-проход).
- **Игровые объекты:** 13, 15 (композиция лягушки), 14 (RNG), 28 (RTC-часы).
- **Инструменты и методология:** 25 (эмулятор EveApps + дампы RAM_DL), 29 (когда эмулятор сам врёт — источник истины = RAM dump).
- **Загрузка с SD:** 30 (FAT32-драйвер: эволюция от WC ZiFi к собственному CMD17+LFN).
- **Игровые системы (adventure):** 31 (выбор уровня, параметры из таблицы, перенос счёта, Win/Pause).
- **Ввод:** 28 (RTC через Mr.Gluk), 32 (опрос PC-клавиатуры через PS/2 FIFO + единый глобальный модуль Input.asm: клавиатура/Kempston/мышь, навигация меню).

Нумерация глав сквозная (1→32). Историческое примечание: ранние версии учебника использовали отдельные пометки `$N/$M/$R` и нумерацию журнала с 18 — в ревизии 2026-05-26 всё приведено к сплошной нумерации.

1. Аппаратная связка ZX-Evo + VDAC2

1.1. Что такое VDAC2

VDAC2 — расширительная плата для ZX-Evo, заменяющая стандартный 5-bit VDAC. Содержит чип **FT812** (FTDI Embedded Video Engine): - 1 MB graphics RAM (RAM_G) - Display List engine (8 KB RAM_DL) - Co-processor с командным буфером (4 KB RAM_CMD) - VGA-выход до 800×600 (для нас целевой режим — 640×480) - 8/8/8 RGB output - SPI-интерфейс к хосту (Z80) до 30 MHz (на ZX-Evo тактирование меньше)

В STATUS-регистре TS-Conf версия адаптера: - `000` — 2-bit VDAC + PWM - `001` / `010` / `011` — 3/4/5-bit VDAC - `111` — VDAC2 (FT812) ← **наш случай**

Sanity-check на старте программы:

```
IN A, (0xAF) ; STATUS
AND %00000111
CP %00000111
JR NZ, .no_vdac2 ; на этой плате FT812 нет → fallback на TS-Conf рендер
```

1.2. Порты TS-Conf для общения с FT812

Порт	Назначение	R/W	Биты
0xAF	STATUS	R	[2..0] версия видеоадаптера
0xAF	VCONFIG	W	[2] FT_EN (0=TS-Config / 1=FT812), [5] NO_GFX (1=отключить TS-Config gfx, освободить DMA-циклы)
0x77	SPI_CTRL	W	bit 0 — ZX-Evolution flag, bit 1 — SD CS (0=en/1=dis), bit 2 — FT812 CS (0=dis/1=en)
0x57	SPI_DATA	R/W	байтовый обмен с активным SPI устройством

Магические значения SPI_CTRL: - `0x03` — `SPI_FT_CS_OFF` (FT812 disable) - `0x07` — `SPI_FT_CS_ON` (FT812 enable)

1.3. VCONFIG для VDAC2-режима

```
LD A, %00100100 ; FT_EN=1 (bit 2), NO_GFX=1 (bit 5)
OUT (0xAF), A
```

`NO_GFX=1` отключает обычный TS-Config рендер пикселей. Это экономит **DMA-циклы**: лимит DMA на строку = 448 циклов, обычно расходуются на чтение VRAM для отрисовки спектр-экрана. С `NO_GFX=1` эти циклы целиком уходят CPU и DMA-пересылке байт в FT812. На бордюре чтения и так нет — там всегда полный лимит свободен.

2. SPI-протокол FT812

2.1. Три типа транзакций (по 2-битному префиксу)

Префикс	Тип	Структура
<code>00b</code>	Memory Read	2b prefix + 22b address + dummy byte + N data bytes
<code>10b</code>	Memory Write	2b prefix + 22b address + N data bytes
<code>01b</code>	Host Command	2b prefix + 6b cmd code + arg byte + 0x00
<code>11b</code>	(зарезервирован)	—

На уровне Z80 префикс — это два старших бита первого отправляемого байта адреса: - Read: первый байт = `addr[21:16]` (биты 7..6 = `00`) - Write: первый байт = `addr[21:16] OR 0x80` (биты 7..6 = `10`) - Host command: первый байт = `0x40 OR cmd[5:0]`

2.2. Каждая транзакция в обёртке CS

```
FT_ON          ; OUT (0x77), 0x07 — взвести CS
... последовательность OUT/IN через 0x57 ...
FT_OFF         ; OUT (0x77), 0x03 — снять CS
```

Внутри одной транзакции **адрес FT812 авто-инкрементируется** — длина блока не ограничена, если данные пишутся в непрерывную область памяти. Это позволяет одной транзакцией залить весь Display List или большой bitmap.

2.3. Готовые asm-функции (из учебника #2)

Макросы CS

```

FT_ON:    MACRO
    LD A, 0x07          ; SPI_FT_CS_ON
    OUT (0x77), A       ; SPI_CTRL
ENDM

FT_OFF:   MACRO
    LD A, 0x03          ; SPI_FT_CS_OFF
    OUT (0x77), A
ENDM

FT_VMODE: MACRO
    LD A, %00100100     ; FT_EN=1, NO_GFX=1
    OUT (0xAF), A
ENDM

```

FT_RD8 — чтение байта из RAM_REG

```

; In:  DE = addr[15..0] (адрес внутри RAM_REG, старший байт фиксирован = 0x30)
; Out: A = прочитанный байт
; Corrupts: AF
FT_RD8:
    FT_ON
    LD A, 0x30          ; FT_RAM_REG >> 16 = 0x30
    OUT (0x57), A       ; addr[21..16] (префикс 00b - read)
    LD A, D
    OUT (0x57), A       ; addr[15..8]
    LD A, E
    OUT (0x57), A       ; addr[7..0]
    OUT (0x57), A       ; dummy OUT (FT812 готовится)
    IN A, (0x57)         ; dummy IN (особенность чтения)
    IN A, (0x57)         ; реальные данные
    PUSH AF
    FT_OFF
    POP AF
    RET

```

Важно: для чтения **всегда** нужен один dummy OUT + один dummy IN после трёх байт адреса, иначе следующий IN вернёт мусор.

FT_RD16

```

; In:  DE = addr[15..0]
; Out: BC = прочитанное 16-битное значение (little-endian)
FT_RD16:
    FT_ON
    LD A, 0x30 : OUT (0x57), A
    LD A, D    : OUT (0x57), A
    LD A, E    : OUT (0x57), A
    OUT (0x57), A ; dummy OUT
    IN A, (0x57) ; dummy IN
    IN A, (0x57) : LD C, A ; младший байт
    IN A, (0x57) : LD B, A ; старший байт
    FT_OFF
    RET

```

FT_WR8 — запись байта в регистр

```

; In: DE = addr[15..0], A = записываемое значение
FT_WR8:
    PUSH AF
    FT_ON
    LD A, (0x30) OR 0x80 ; bit 7 = 1 → префикс 10b → write
    OUT (0x57), A
    LD A, D : OUT (0x57), A
    LD A, E : OUT (0x57), A
    POP AF
    OUT (0x57), A ; данные
    FT_OFF
    RET

```

FT_Write — блочная запись через OTIR

```

; In: HL = Z80-источник, BC = количество байт,
;     A = addr[21..16] (без bit 7), DE = addr[15..0]
; Out: HL += BC, ADE += BC (для chained-вызовов)
FT_Write:
    PUSH AF
    FT_ON
    POP AF
    PUSH AF
    OR 0x80 ; включаем bit 7 - write префикс
    OUT (0x57), A
    LD A, D : OUT (0x57), A
    LD A, E : OUT (0x57), A
    POP AF

    ; пересчитать адрес для следующего вызова: HL += BC, ADE += BC
    EX DE, HL
    ADD HL, BC
    EX DE, HL
    ADC A, 0
    PUSH AF

    ; основной OTIR loop (256-байтные пакеты)
    LD A, C : OR A ; есть младший хвост?
    LD A, B ; A = количество полных пакетов
    LD B, C ; B = младший байт count'a
    LD C, 0x57 ; SPI_DATA
    JR Z, .loop

    OTIR ; неполный пакет
    OR A
    JR Z, .exit

.loop:
    OTIR ; B=0 → 256 байт
    DEC A
    JR NZ, .loop

.exit:
    FT_OFF
    POP AF
    RET

```

FT_Read строится симметрично через **INIR**, без bit 7 в первом байте + dummy перед циклом.

2.4. Когда размер блока удобен

- Заливка bitmap в RAM_G — одна транзакция на килобайты
- Полный DL (до 8 KB) — одна транзакция
- Запись в RAM_CMD кольцевого буфера — порциями до wgr'a

3. Memory Map FT812

22-битное адресное пространство; всё mapped единообразно по SPI:

Диапазон	Размер	Имя	Назначение
0x000000-0x0FFFFF	1024 KB	RAM_G	Графика общего назначения (bitmaps)
0x1E0000-0x2FFFFB	1152 KB	ROM_FONT	Шрифты ROM
0x300000-0x301FFF	8 KB	RAM_DL	Display List
0x302000-0x302FFF	4 KB	RAM_REG	Регистры
0x308000-0x308FFF	4 KB	RAM_CMD	Co-processor command buffer (ring)

Endianness: little-endian для всех многобайтных значений (Z80-friendly).

3.1. Ключевые регистры RAM_REG

Адрес	Имя	Биты	Сброс	Назначение
0x302000	REG_ID	8 ro	0x7C	Сигнатура чипа (sanity-check)
0x302004	REG_FRAMES	32 ro	0	Счётчик кадров от reset
0x30200C	REG_FREQUENCY	28 rw	60000000	Тактовая частота в Hz
0x30202C	REG_HCYCLE	12 rw	0x224	Полное число PCLK на строку
0x302030	REG_HOFFSET	12 rw	0x02B	H-offset (front porch + sync + back porch)
0x302034	REG_HSIZE	12 rw	0x1E0	Видимая ширина в PCLK = пикселях
0x302038	REG_HSYNCo	12 rw	0x000	H-sync front porch
0x30203C	REG_HSYNC1	12 rw	0x029	H-sync front + pulse
0x302040	REG_VCYCLE	12 rw	0x124	Полное число строк на кадр
0x302044	REG_VOFFSET	12 rw	0x00C	V-offset
0x302048	REG_VSIZE	12 rw	0x110	Видимая высота в строках
0x30204C	REG_VSYNCo	10 rw	0	V-sync front porch
0x302050	REG_VSYNC1	10 rw	0x00A	V-sync front + pulse
0x302054	REG_DLSWAP	2 rw	0	Управление flip'ом DL (0/1/2)
0x302070	REG_PCLK	8 rw	0	Делитель PCLK (0=PCLK выкл)
0x30206C	REG_PCLK_POL	1 rw	0	Полярность PCLK
0x302100	REG_CMD_DL	13 rw		Co-processor pointer в DL

3.2. DLSWAP modes

Значение	Имя	Эффект
0	DLSWAP_DONE	Текущий swap завершён (read)
1	DLSWAP_LINE	Swap по строке
2	DLSWAP_FRAME	Swap в начале vsync (стандарт)

После записи нового DL → `FT_WR8(REG_DLSWAP, DLSWAP_FRAME)` — кадр обновится в следующем vsync.

4. Видеотайминги для 640×480

TSLib содержит выверенные таблицы для **трёх** режимов 640×480 (`Include\FT\81x Const.inc:359-391`):

4.1. VM_640_480_57Hz (PCLK 24 MHz, F_MUL=3) — целевой для Zuma

Параметр	H (Horizontal)	V (Vertical)
Front porch	16	11
Sync pulse	96	2
Back porch	48	31
Visible	640	480
Total	800	524

`F_MUL` — значение, которое TSLib пишет в `REG_PCLK`. На плате VDAC2 (внешний clock через `CLKEXT` + `CLKSEL #C0`) результирующий pixel clock получается как $8 \text{ МГц} \times F_MUL : 8 \times 3 = 24 \text{ МГц}$ (а при `F_MUL=4` → 32 МГц, §4.2). Это подтверждается независимо: $HCYCLE \times VCYCLE \times \text{refresh} = 800 \times 524 \times 57 \approx 24 \text{ МГц}$.

⚠ Уточнение семантики (важно при переносе). « $8 \times F_MUL$ » — мнемоника **ИМЕННО** нашей настройки clock на VDAC2, а не общая семантика регистра. По даташиту FT81х `REG_PCLK` — это **ДЕЛИТЕЛЬ** системного clock: $PCLK = f_{sys} / REG_PCLK$ (не множитель и не «база 8 МГц»). У FT81х системный clock задаётся `CLKSEL` (по умолчанию 60 МГц; варианты 24/36/48; 72 МГц — уже у BT81х), и после смены clock надо обновить `REG_FREQUENCY`. Переносите на другой clock — считайте PCLK по формуле делителя из даташита, не по « $\times 8$ ».

TSLib-константа: `F0_MUL=3, H0_FPORCH=16, H0_SYNC=96, H0_BPORCH=48, H0_VISIBLE=640, V0_FPORCH=11, V0_SYNC=2, V0_BPORCH=31, V0_VISIBLE=480`.

4.2. VM_640_480_74Hz (PCLK 32 MHz, F_MUL=4) — повышенная частота

`F1_MUL=4, H1_FPORCH=24, H1_SYNC=40, H1_BPORCH=128, V1_FPORCH=9, V1_SYNC=3, V1_BPORCH=28`.

4.3. VM_640_480_76Hz (PCLK 32 MHz, F_MUL=4)

`F2_MUL=4, H2_FPORCH=16, H2_SYNC=96, H2_BPORCH=48, V2_FPORCH=11, V2_SYNC=2, V2_BPORCH=31`.

4.4. Соответствие регистрам FT812

TSLib `FT_ModeTab` (`81x Const.inc:460`) укладывает значения в следующие регистры FT812:

Регистр	Формула	для VM_640_480_57Hz
REG_HSYNCo	H_FPORCH	16
REG_HSYNC1	H_FPORCH + H_SYNC	112
REG_HOFFSET	H_FPORCH + H_SYNC + H_BPORCH	160
REG_HSIZE	H_VISIBLE	640
REG_HCYCLE	H_FPORCH+SYNC+BPORCH+VISIBLE	800
REG_VSYNCo	V_FPORCH – 1	10
REG_VSYNC1	V_FPORCH + V_SYNC – 1	12
REG_VOFFSET	V_FPORCH+V_SYNC+V_BPORCH – 1	43
REG_VSIZE	V_VISIBLE	480
REG_VCYCLE	V_FPORCH+V_SYNC+V_BPORCH+V_VISIBLE	524
REG_PCLK	F_MUL (делитель клона, см. §4.1)	3 → PCLK 24 МГц
REG_PCLK_POL	0	0

4.5. Применение через TSLib-макрос

В TSLib переключение режима — одна строчка:

```
FT_RESOLUTION VM_640_480_57Hz, ResolutionWidthPtr
```

Где `ResolutionWidthPtr` — Z80-указатель на 2-байтную ячейку в RAM, куда макрос сохраняет ширину экрана для последующего использования (`Examples\2.HelloWorld\Include.inc:8`).

`FT_RESOLUTION` (`Include\FT\812 Macro.inc:354`) сам разворачивается в нужную таблицу + серию `FT_WR_REG16` по адресам HCYCLE/HOFFSET/HSIZE/HSYNCo/HSYNC1/VCYCLE/VOFFSET/VSIZE/VSYNCo/VSYNC1 + `FT_WR_REG8` `FT_REG_PCLK` со значением F_MUL.

4.6. Полная init-последовательность (TSLib `FT_BOOT_UP`)

```
Include\FT\812 Macro.inc:104 :
```

```

FT_BOOT_UP    macro
                FT_SEND_COMMAND FT_CMD_PWRDOWN_      ; #50 - power-down
                FT_DELAY 3
                FT_SEND_COMMAND FT_CMD_CLKEXT        ; #44 - внешний clock
                FT_DELAY 3
                LD B, FT_CMD_CLKSEL                  ; #61 - clock select
                LD C, #C0                            ; PLL range / MUL
                CALL FT.SendCommand.Param
                FT_SEND_COMMAND FT_CMD_ACTIVE        ; #00 - активировать
                FT_DELAY 15

.WaitIntReady  FT_RD_REG8 FT_REG_ID                  ; ждать REG_ID == 0x7C
                CP #7C
                JR NZ, .WaitIntReady

.WaitCPU_Reset FT_RD_REG8 FT_REG_CPURESET            ; ждать REG_CPURESET == 0
                OR A
                JR NZ, .WaitCPU_Reset

"ТИХО"
                FT_WR_REG8 FT_REG_PCLK, 0            ; PCLK выкл - тайминги настраиваются

                FT_WR_REG16 FT_REG_HCYCLE, 0x224     ; default тайминги
                FT_WR_REG16 FT_REG_HOFFSET, 0x02B
                ; ... HSYNC0/1, VCYCLE/OFFSET/VSYNC0/1, SWIZZLE, PCLK_POL ...
                FT_WR_REG16 FT_REG_HSIZE, 0x1E0
                FT_WR_REG16 FT_REG_VSIZE, 0x110
                FT_WR_REG16 FT_REG_CSPREAD, 0x001
                FT_WR_REG16 FT_REG_DITHER, 0x001
                FT_WR_REG16 FT_REG_OUTBITS, 0x000
                FT_WR_REG16 FT_REG_GPIOX_DIR, 0xFFFF ; все GPIOX выходы
                FT_WR_REG16 FT_REG_GPIOX, 0xFFFF   ; включить DISP
                FT_WR_REG8 FT_REG_PCLK, 2           ; PCLK on (временное значение)
            endm

```

`REG_PCLK=2` в конце `FT_BOOT_UP` — **временное** значение: оно лишь включает развёртку с default-таймингами, чтобы видеовыход «ожил». Рабочий PCLK для 640×480 задаёт следующий шаг — `FT_RESOLUTION` (`REG_PCLK = F_MUL = 3` или `4`). Прежний комментарий «60/2 = 30 МГц» был прикидкой по даташит-формуле делителя при условном sys-clock 60 МГц (см. уточнение в §4.1).

После `FT_BOOT_UP` нужный режим выставляется через `FT_RESOLUTION VM_640_480_57Hz`. Финальный шаг — переключить выход TS-Conf на FT812 и отключить обычный gfx:

```
Video_Setting VID_FT812 | VID_NOGFX ; OUT (0xAF), %00100100
```

`Video_Setting` — TSLib-макрос (`Include\Cache\Macro.inc` и др.), эквивалент учебника #2.

5. Display List

5.1. Структура

DL = массив 32-bit команд в RAM_DL (`0x300000` .. `0x301FFF`). Максимум 2048 команд. Каждая команда — 4 байта little-endian. Последняя команда обязана быть `DISPLAY()`.

После записи DL → `REG_DLSWAP=DLSWAP_FRAME` → следующий vsync покажет новый кадр.

5.2. Базовые opcodes (минимальный набор для Zuma)

Команда	Opcode	Формат
<code>DISPLAY()</code>	<code>0x00 << 24</code>	конец DL
<code>BEGIN(prim)</code>	<code>0x1F << 24 prim</code>	старт примитива
<code>END()</code>	<code>0x21 << 24</code>	конец примитива
<code>CLEAR_COLOR_RGB(r,g,b)</code>	<code>0x02 << 24 (r<<16) (g<<8) b</code>	цвет очистки
<code>CLEAR(c,s,t)</code>	<code>0x26 << 24 (c<<2) (s<<1) t</code>	очистка буферов
<code>COLOR_RGB(r,g,b)</code>	<code>0x04 << 24 (r<<16) (g<<8) b</code>	цвет рисования
<code>COLOR_A(a)</code>	<code>0x10 << 24 a</code>	альфа
<code>BLEND_FUNC(src,dst)</code>	<code>0x0B << 24 (src<<3) dst</code>	смешивание
<code>POINT_SIZE(s)</code>	<code>0x0D << 24 s</code>	радиус точки в 1/16 px
<code>LINE_WIDTH(w)</code>	<code>0x0E << 24 w</code>	толщина линии 1/16
<code>BITMAP_HANDLE(h)</code>	<code>0x05 << 24 h</code>	активный handle (0..31)
<code>BITMAP_SOURCE(addr)</code>	<code>0x01 << 24 addr</code>	источник в RAM_G
<code>BITMAP_LAYOUT(fmt,stride,h)</code>	<code>0x07 << 24 (fmt<<19) (stride<<9) h</code>	формат + stride
<code>BITMAP_SIZE(filter,wrx,wry,w,h)</code>	<code>0x08 << 24 (filter<<20) (wrx<<19) (wry<<18) (w<<9) h</code>	визуальный размер
<code>CELL(c)</code>	<code>0x06 << 24 c</code>	номер cell в атласе
<code>VERTEX2II(x,y,h,c)</code>	<code>0x80000000 (x<<21) (y<<12) (h<<7) c</code>	вершина integer + handle + cell
<code>VERTEX2F(x,y)</code>	<code>0x40000000 (sx<<15) sy</code>	subpixel вершина (1/16)
<code>SCISSOR_XY(x,y)</code>	<code>0x1B << 24 (x<<11) y</code>	clip origin
<code>SCISSOR_SIZE(w,h)</code>	<code>0x1C << 24 (w<<12) h</code>	clip size
<code>SAVE_CONTEXT()</code>	<code>0x22 << 24</code>	стек контекста push
<code>RESTORE_CONTEXT()</code>	<code>0x23 << 24</code>	pop

`prim` для `BEGIN`: - 1 BITMAPS, 2 POINTS, 3 LINES, 4 LINE_STRIP, 5 EDGE_STRIP_R/L/A/B (6,7,8,9), 9 RECTS

5.3. Минимальный «Hello World» DL

Заливаем экран синим цветом:

```
0x02_00_00_64 ; CLEAR_COLOR_RGB(0,0,100) [синий]
0x26_00_00_07 ; CLEAR(1,1,1) [color, stencil, tag]
0x00_00_00_00 ; DISPLAY()
```

Записать 12 байт в `0x300000`, затем `REG_DLSWAP = 2` → синий экран на следующем vsync.

5.4. Цепочка спрайтов из атласа

Для рендера 240 шаров Zuma из атласа (handle 0, 6 cells × 40×40):

```
SAVE_CONTEXT()
BITMAP_HANDLE(0)
BEGIN(BITMAPS)
; per-ball loop:
;   COLOR_RGB(255,255,255)      ; без тинта
;   VERTEX2II(x, y, 0, color)   ; одна 32-bit команда на шар
END()
RESTORE_CONTEXT()
DISPLAY()
```

Для 240 шаров = ~244 32-bit команды = ~976 байт DL (вмещается в 8KB).

5.5. Co-processor (RAM_CMD) — для удобства

Командный буфер `0x308000` + — кольцевой, читается co-processor'ом FT812. Команды: - `cmd_dlstart` — открыть новый DL - `cmd_swap` — REG_DLSWAP - `cmd_loadimage` — JPEG/PNG → RAM_G - `cmd_text`, `cmd_number` — рендер текста (DL команды генерируются автоматически) - `cmd_rotate`, `cmd_translate`, `cmd_scale`, `cmd_setmatrix` — матричные трансформации

Запись в RAM_CMD управляется парой `REG_CMD_WRITE` (наша запись) и `REG_CMD_READ` (читает FT812). Wrapping — 4KB. После записи → `REG_CMD_WRITE = новый offset`. Ждать пока FT812 не прочитает: `REG_CMD_READ == REG_CMD_WRITE`.

6. Bitmap-форматы для FT812

Формат	Бит/ пиксель	Описание	Применение
ARGB1555	16	5R 5G 5B 1A	Спрайты с 1-bit маской
L1	1	Чёрно-белый	Биткарты
L2	2	4 уровня серого	Полупрозрачные текстуры (есть нюанс — см. Bowman 15.2)
L4	4	16 серых	Шрифты
L8	8	256 серых	Маски
RGB332	8	3R 3G 2B	Фон без точности
ARGB2	8	2A 2R 2G 2B	Лёгкая прозрачность
ARGB4	16	4 на канал	Полупрозрачные спрайты
RGB565	16	Стандарт без альфы	Backgrounds, спрайты без прозрачности
PALETTED4/8/565	4/8/8	Через CRAM	Наши шары/фон с экономией памяти
BARGRAPH	спес.	Гистограммы	UI

Для Zuma 640×480 рекомендации: - Background: PALETTED8 (256 цветов) → 640×480 = 307KB; либо RGB565 → 614KB. - Шары 6 цветов × 40×40: PALETTED8 → 9.6KB атлас; либо RGB565 → 19.2KB. Маска 1-bit для прозрачности отдельным L1-bitmap'ом. - Жаба 128×128: ARGB4 (есть альфа) → 32KB. - Курсор 32×32: ARGB1555 → 2KB. - Killzone 64×64 (1 frame): ARGB4 → 8KB.

Итого ~360 KB из 1024 KB RAM_G — есть запас на анимации.

7. Производительность и DMA

7.1. Оценка bandwidth Z80 → FT812 через SPI

- ZX-Evo Z80 на ~7 MHz
- **OTIR** = 21 такта/байт = ~333 KB/s максимум
- Через DMA TS-Conf (если задействована) — выше, до ~1 MB/s

7.2. Полный кадр при 60 fps

- Frame budget: 16.7 ms
- DL обновление 240 шаров: ~1 KB → 3 ms через OTIR
- Background не обновляется каждый кадр (статичен в RAM_G после init)

7.3. NO_GFX=1 экономит DMA

С **NO_GFX=1** лимит DMA на строку (448 циклов) полностью доступен для FT812-передач, а не делится с TS-Config рендером. Это критично для частых обновлений RAM_G (анимация фона).

8. План разработки Zuma VDAC2 (контекст)

1.  Базовая 360×288-версия Zuma собирается в папке проекта.
2.  Python-эмулятор VDC масштабирован под 640×480 (визуальная отладка).
3.  Init-последовательность FT812: detect → power → тайминги 640×480 → REG_PCLK.
4.  Первый «hello DL» — синий экран через VDAC2.
5.  Загрузка тестового bitmap в RAM_G + рендер одной точкой.
6.  Атлас шаров → BITMAP_HANDLE → VERTEX2II loop по slots[].
7.  Background (палитра + bitmap из конвертера).
8.  Жаба + cursor + killzone как handles 1/2/3.
9.  Sync VDC engine 360×288 ↔ FT812 рендер 640×480 (scale ×2 в координатах).

9. Главный цикл рендера (Hello World pattern из TSLib)

Examples\2.HelloWorld\Core\MainLoop.asm — образцовая структура кадра:

```

.Loop      FT_CMD_Start      ; начать собирать команды в буфер RAM Z80
          FT_DL_Start        ; команда DLSTART для co-processor

          FT_ClearColorRGB32 0x000000 ; чёрный фон
          FT_ClearAll        ; clear color + stencil + tag

          CALL ShowText      ; <- наш контент
          CALL Fizz

          FT_Display         ; конец DL
          FT_CMD_Write       ; залить буфер из RAM Z80 в RAM_CMD FT812

          FT_WR_REG8 FT_REG_DLSWAP, FT_DLSWAP_FRAME ; запросить swap

.WaitIntSwap FT_RD_REG8 FT_REG_INT_FLAGS ; дождаться SWAP interrupt
          AND FT_INT_SWAP
          JR Z, .WaitIntSwap

          FT_DELAY 2
          JP .Loop

```

9.1. Что делает FT_CMD_Start / FT_CMD_Write

FT_CMD_Start — устанавливает Z80-указатель **FT.Coprocessor.BufferPtr** в начало **локального буфера** **CMD_ADDRESS_PTR** в RAM Z80 (см. **BufferMacro.inc:5**). Все последующие **FT_CMD_BUF** / **FT_ClearAll** / **FT_Begin** / **FT_Vertex2ii** — это запись 4-байтных команд в этот локальный буфер (через **LD (HL), E : INC HL × 4**).

FT_CMD_Write — считает длину буфера (текущий ptr – начало) и блочно отправляет всё в RAM_CMD FT812 через **FT.Coprocessor.Write** (вызывает **FT_Write** / OTIR-цикл).

Это ключевой паттерн: **DL собирается в RAM Z80**, затем **одной транзакцией** уходит в FT812. Гораздо эффективнее чем командировать FT812 по одной команде за раз.

9.2. FT_DLSWAP_FRAME / FT_INT_SWAP

После записи команд **REG_DLSWAP = FT_DLSWAP_FRAME (=2)** запрашивает swap в начале ближайшего vsync. **REG_INT_FLAGS** бит **FT_INT_SWAP** поднимается когда swap состоялся — это сигнал «можно начинать новый кадр». Без ожидания будут «глитчи»: писать в DL пока FT-engine ещё рендерит — undefined.

FT_INT_MASK и **FT_INT_EN** нужно настроить в init (Hello World делает: **FT_REG_INT_MASK = FT_INT_SWAP**, **FT_REG_INT_EN = 1**).

10. TSLib API — карта макросов

10.1. Низкий уровень: `Include\FT\812 Macro.inc`

Макрос	Описание
<code>FT_ON</code> / <code>FT_OFF</code>	CS управление (= OUT 0x77)
<code>FT_VMODE</code>	OUT (VCONFIG), VID_FT812
<code>FT_ACTIVE</code>	host command #00 → выйти из standby
<code>FT_BOOT_UP</code>	полная init-последовательность (см. §4.6)
<code>FT_CMD_RESET</code>	сброс co-processor (CMD_READ/WRITE = 0)
<code>FT_SEND_COMMAND</code>	host command (3 байта)
<code>FT_DELAY Count?</code>	NOP-задержка
<code>FT_RD_REG8</code> / <code>FT_RD_REG16</code> / <code>FT_RD_REG32</code>	чтение регистра
<code>FT_WR_REG8</code> / <code>FT_WR_REG16</code> / <code>FT_WR_REG32</code>	запись регистра
<code>FT_RESOLUTION VM_*, RefPtr</code>	переключение видеорежима

10.2. Прямой Display List в RAM_DL: `Include\FT\DL Macro.inc`

Каждый макрос разворачивается в `DEFD <opcode>` (4 байта в текущем месте сборки). Используется когда DL зашит в постоянную область (например, статическая графика уровня), не строится каждый кадр.

Макрос	Opcode	Назначение
<code>FT_DISPLAY</code>	0x00	Конец DL (обязателен)
<code>FT_BITMAP_SOURCE Address?</code>	0x01	Источник в RAM_G
<code>FT_CLEAR_COLOR_RGB R,G,B</code>	0x02	Цвет очистки
<code>FT_TAG</code>	0x03	Тег для touch
<code>FT_COLOR_RGB R,G,B</code>	0x04	Цвет рисования
<code>FT_BITMAP_HANDLE H</code>	0x05	Активный handle (0..31)
<code>FT_CELL c</code>	0x06	Cell в атласе
<code>FT_BITMAP_LAYOUT fmt,stride,h</code>	0x07	Формат + stride
<code>FT_BITMAP_SIZE filter,wx,wy,w,h</code>	0x08	Размер для рендера
<code>FT_ALPHA_FUNC</code>	0x09	Альфа-тест
<code>FT_STENCIL_FUNC</code>	0x0A	Stencil-тест
<code>FT_BLEND_FUNC src,dst</code>	0x0B	Смешивание
<code>FT_POINT_SIZE s</code>	0x0D	Радиус точки 1/16
<code>FT_LINE_WIDTH w</code>	0x0E	Толщина линии
<code>FT_COLOR_A a</code>	0x10	Альфа
<code>FT_BITMAP_TRANSFORM_A..F</code>	0x15-1A	Матрица 2D трансформации
<code>FT_SCISSOR_XY x,y</code>	0x1B	Clip origin
<code>FT_SCISSOR_SIZE w,h</code>	0x1C	Clip size
<code>FT_BEGIN prim</code>	0x1F	Старт примитива
<code>FT_END</code>	0x21	Конец примитива
<code>FT_SAVE_CONTEXT</code>	0x22	Push контекст
<code>FT_RESTORE_CONTEXT</code>	0x23	Pop контекст

`prim` для `BEGIN`: 1=BITMAPS, 2=POINTS, 3=LINES, 4=LINE_STRIP, 5/6/7/8=EDGE_STRIP_*, 9=RECTS.

10.3. Сборка DL через co-processor: `Include\FT\Coprocessor\BufferMacro.inc`

Те же команды, но макросы пишут не `DEFD`, а `FT_CMD_BUF` (накапливают в RAM Z80 для последующего `FT_CMD_Write`). Используется в `MainLoop` каждый кадр.

Макрос	Что делает
<code>FT_CMD_Start</code>	Сбросить указатель Z80-буфера
<code>FT_DL_Start</code>	Команда CMD_DLSTART (открыть новый DL)
<code>FT_ClearColorRGB32 RGB?</code>	Цвет очистки 0xRRGGBB
<code>FT_ClearAll</code>	Clear all (color + stencil + tag)
<code>FT_Clear C,S,T</code>	Selective clear
<code>FT_Begin prim</code> / <code>FT_End</code>	Примитивы
<code>FT_Vertex2f X,Y</code>	Вершина float (1/16 px)
<code>FT_Vertex2ii X,Y,H,C</code>	Вершина integer + handle + cell в одной команде
<code>FT_PointSize s</code>	Радиус точки
<code>FT_LineWidth w</code>	Толщина линии
<code>FT_ColorRGB</code> / <code>FT_ColorRGB32</code>	Цвет
<code>FT_ColorA a</code>	Альфа
<code>FT_BitmapHandle H</code>	Активный handle
<code>FT_BitmapSource addr</code>	Указать на bitmap в RAM_G
<code>FT_BitmapLayout fmt, stride, h</code>	Формат
<code>FT_BitmapSize filter, wx, wy, w, h</code>	Размер
<code>FT_Cell c</code>	Cell в атласе
<code>FT_BlendFunc src, dst</code>	Смешивание
<code>FT_ScissorXY</code> / <code>FT_ScissorSize</code>	Clip
<code>FT_SaveContext</code> / <code>FT_RestoreContext</code>	Стек контекста
<code>FT_Tag t</code> / <code>FT_TagMask</code>	Touch теги
<code>FT_VertexFormat frac</code>	Точность Vertex2f (бит 0..7 = 1/2..1/256 px)
<code>FT_VertexTranslateX/Y</code>	Смещение всех последующих Vertex
<code>FT_PaletteSource</code>	Палитра PALETTED-форматов
<code>FT_FGColor</code> / <code>FT_BGColor</code> / <code>FT_GRADColor</code>	Цвета для widgets
<code>FT_Text X,Y,Font,Opt</code>	Текст
<code>FT_String addr, len</code>	Строка для FT_Text
<code>FT_Gradient x1,y1,rgb1,x2,y2,rgb2</code>	Градиент
<code>FT_Display</code>	Конец DL
<code>FT_CMD_Swap</code>	CMD_SWAP (через co-processor)
<code>FT_CMD_Interrupt ms</code>	CMD_INTERRUPT

10.4. Coprocessor функции (`Include\FT\Coprocessor\Buffer.asm` + `Cmd.asm`)

Дополнительные runtime-функции: - `FT.Coprocessor.PointSize` — `LD DE, size` → пишет POINT_SIZE в буфер - `FT.Coprocessor.ColorRGB` — `LD C, R : LD D, G : LD E, B` - `FT.Coprocessor.ColorA` — `LD E, A` - `FT.Coprocessor.Vertex2f` — `LD HL, X : LD DE, Y` (subpixel) - `FT.Coprocessor.WaitFlush` — ждать пока FT прочитает RAM_CMD - `FT.Coprocessor.GetPtr` — получить текущий REG_CMD_DL (для return-адресов в DL) - `FT.Coprocessor.IsFault` — проверка ошибки co-processor - `FT.Coprocessor.Inflate` — распаковать deflate-blob в RAM_G

10.5. Прочее

- `Include\Cache\Macro.inc` — `Cache_Setting EN_0000 | EN_4000 | EN_8000` — TS-Conf cache
- `Include\DMA\Macro.inc` — TS-Conf DMA helpers (для блочных копий, в т.ч. в FT812 — но про DMA-FT учебник #3+)
- `Include\Input\Kempston\Mouse\*` — мышь Kempston (готовое)
- `Include\Math\F16\*`, `Fixed\18.14\*`, `Fixed\2.14\*`, `Lerp.asm`, `Mul/*`, `Div/*` — fixed-point/F16 математика

10.6. Готовый Init_Video для Zuma VDAC2

Реализовано в `Init_Video.asm` в корне проекта. Собирается под sjasmplus (--syntax=ab) с TSLib без ошибок (smoke-build см. `_test_init_video.asm`).

Зависимости (порядок важен):

```

DEVICE ZXSPPECTRUM4096
define MAPPING_REGISTERS                ; Video_Setting через FMADDR

include "Docs/TSLib/Include/TSConf.inc"
include "Docs/TSLib/Include/Video/Macro.inc"
include "Docs/TSLib/Include/FT/81x Const.inc"
include "Docs/TSLib/Include/FT/DL Macro.inc"
include "Docs/TSLib/Include/FT/812 Macro.inc"
module FT
include "Docs/TSLib/Include/FT/812 Func.asm"
endmodule

ResolutionWidthPtr    EQU #40F3                ; Z80-RAM ячейки (FT_RESOLUTION пишет
туда W/H)
ResolutionHeightPtr   EQU #40F5

include "Init_Video.asm"

```

Сама логика (см. файл):

1. **Sanity-check VDAC2:** `IN A,(STATUS) : AND %111 : CP %111` — если бит-маска ≠ 111, возврат с Z=0 (нет VDAC2 на плате).
2. **FT_BOOT_UP** — полная init FT812: PWRDOWN→CLKEXT→CLKSEL #Co→ACTIVE, ждать REG_ID=0x7C, default тайминги, GPIOX=0xFFFF, REG_PCLK=2 → видеовыход активирован.
3. **FT_CMD_RESET** — обнулить REG_CMD_READ/WRITE (на случай висящих команд).
4. **FT_RESOLUTION VM_640_480_57Hz, ResolutionWidthPtr** — переключить тайминги: PCLK=24 МГц (F_MUL=3), HCYCLE 800, VCycle 524, HSIZE/VSIZE 640×480.

5. **Залить пустой DL** (12 байт = `CLEAR_COLOR_RGB(0,0,0); CLEAR(1,1,1); DISPLAY()`) в RAM_DL через `FT.WriteDL`, потом `REG_DLSWAP=2` — чёрный экран до первого MainLoop-кадра.
6. `REG_INT_MASK = FT_INT_SWAP, REG_INT_EN = 1` — разрешить swap-interrupt для синхронизации MainLoop'a.
7. `Video_Setting VID_FT812 | VID_NOGFX = OUT (0xAF), %00100100` — переключить TS-Conf выход на FT812 + отключить TS-Config gfx (освобождает 448 DMA-циклов/строку).

Возврат: A=0/Z=1 на успех, A=1/Z=0 если VDAC2 не обнаружен (caller выбирает fallback).

После Init_Video можно входить в MainLoop с FT_CMD_Start/FT_CMD_Write/DLSWAP паттерном (§9).

10.7. Готовый MainLoop для Zuma VDAC2 (каркас)

Реализовано в `MainLoop.asm` в корне проекта. Собирается без ошибок (см. `_test_init_video.asm` — там полная цепочка include'ов и `Init_Video → MainLoop` точка входа).

На текущем этапе MainLoop — **proof-of-life каркас**: тёмно-синий фон + одна оранжевая точка 16 px радиуса, отскакивающая от краёв 640×480. По мере добавления game-state'a сюда подключатся VDC engine update, цикл по `slots[]`, frog/cursor/score.

Структура одного кадра (6 шагов):

```
.Loop          ; 1. Открываем DL, заливаем общую очистку
               FT_CMD_Start
               FT_DL_Start
               FT_ClearColorRGB32 0x102030
               FT_ClearAll

               ; 2. Контент кадра
               CALL ZL_DrawFrame          ; PointSize + ColorRGB + Begin POINTS +
Vertex2f + End

               ; 3. Закрытие DL и заливка в FT812
               FT_Display
               FT_CMD_Write              ; OTIR-блок RAM Z80 → RAM_CMD FT812

               ; 4. Запросить swap при следующем vsync
               FT_WR_REG8 FT_REG_DLSWAP, FT_DLSWAP_FRAME

.WaitIntSwap   ; 5. Заблокироваться до swap-interrupt'a
               FT_RD_REG8 FT_REG_INT_FLAGS
               AND FT_INT_SWAP
               JR Z, .WaitIntSwap

               ; 6. Update game state (между swap'ом и следующим DL)
               CALL ZL_UpdateGame
               JP .Loop
```

Важно про порядок в одном кадре: - `FT_CMD_Start` сбрасывает указатель локального буфера в RAM Z80 (`CMD_ADDRESS_PTR`, по умолчанию `#C000`). - Все макросы группы `FT_*` из `BufferMacro.inc` **только пишут в этот буфер** — пока не вызван `FT_CMD_Write`, ничего на FT812 не уходит. - `FT_CMD_Write` — **одна** OTIR-транзакция в `REG_CMDB_WRITE` (FT_RAM_CMD). Эффективнее команд по одной. - `REG_DLSWAP=FT_DLSWAP_FRAME` запрашивает swap. Без ожидания `FT_INT_SWAP` следующий DL может начать строиться поверх ещё рендерящегося → артефакты. - `Update` после `WaitIntSwap` — пока FT-engine отрисовывает только что засвопленный кадр, Z80 свободен для физики. Это **естественный double-buffering**: кадр N+1 готовится пока кадр N показывается.

Точка состояния (`ZL_PointX` / `ZL_PointY` etc.) хранится в коде через `DEFW 0` — после загрузки `.bin` это валидные ячейки, `MainLoop` при первом входе явно их инициализирует на `(SCR_W/2, SCR_H/2)` и скорость `(3, 2)` px/frame в 1/16-формате (`VertexFormat=4`, по умолчанию).

`ZL_DrawFrame` использует runtime-функции `FT.Coprocessor.ColorRGB` / `PointSize` / `Vertex2f` (из `Coprocessor/Buffer.asm`) — они принимают значения в регистрах (BC/DE), а не immediate, что нужно для динамической позиции.

`ZL_UpdateGame` — bouncing: `X += VelX`, если `X >= MAX_X` или `X < MIN_X` → clamp + `VelX = -VelX` через мини-helper `ZL_NegateW`. То же по Y.

Smoke-test: `_test_init_video.asm` — точка входа `Init_Video → MainLoop`, при сборке через `sjasmpplus 1.18.3` даёт `Errors: 0, warnings: 0, compiled: 4370 lines`. Не запускался на реальном hardware/эмуляторе — это следующий шаг (нужен `spgbl` + правильные `SAVEBIN` директивы под ZX-Evo memory layout).

11. Открытые вопросы / TODO для углубления учебника

- ✓ ~~Точные тайминги VGA 640×480~~ — закрыто (TSLib `81x Const.inc:359-391`, см. §4).
- ✓ ~~Полный список opcodes DL~~ — закрыто (TSLib `DL Macro.inc` + `BufferMacro.inc`, см. §10).
- ⌚ RAM_CMD wrapping и синхронизация с REG_CMD_READ/WRITE — есть пример в TSLib `Coprocessor/Buffer.asm`, разобрать и перенести в учебник.
- ⌚ Touch-engine — нам не нужен, но в учебник для полноты: описать `REG_TOUCH_*`.
- ⌚ Аудио через FT812 — есть моно PCM/ADPCM; в Zuma можно подключить sfx (clicks при insert/match).
- ⌚ DMA-передача в FT812 (упомянута в учебнике #2, но детали в #3+) — раздел дописать после.
- ⌚ Bitmap-конвертация PNG → FT-формат: разобрать `AN_303 FT800 Image File Conversion.pdf` + изучить `Examples/3.Bitmap/Core/TexturesCharacter.inc` / `TexturesParallax.inc` — паттерн как загружают спрайты в RAM_G.
- ⌚ Tilemap для backgrounds — `Examples/Game/Core/Tilemap_DL.asm` (1116 байт) разобрать и перенести в учебник.
- ⌚ FT81X_Series_Programmer_Guide.pdf (4 МБ) — извлечь в txt и дописать недостающие детали (особенно секции про blend modes, color formats, optimization tips).
- ⌚ BRT_AN_033 BT81X Programming Guide (4.8 МБ) — BT81x совместим с FT81x по DL, но добавляет ASTC bitmap formats и CMD_FLASH* — может быть полезно если когда-нибудь будет VDAC3.

Глава 12. Bitmap rendering — matrix transform, scale, paletted formats (опыт 2026-05-09)

12.1 Главный урок: BITMAP_TRANSFORM работает на bitmap UV, не на screen position

В FT81x матрица `BITMAP_TRANSFORM_A..F` (set через `cmd_setmatrix` после `cmd_loadidentity` + операций) трансформирует **bitmap UV-coordinates** (= какой пиксель bitmap читать), не screen position.

Render-formula: pixel at screen `(vertex_pos.x + u, vertex_pos.y + v)` reads bitmap at `M * (u, v)`.

Из этого следует: - `cmd_translate(X, Y)` **сдвигает источник** читаемых пикселей. Для UV outside bitmap → BORDER возвращает transparent → sprite **невидим**. - **Screen position спрайта** задаётся через `Vertex2f((X-half)*16, (Y-half)*16)` (subpixel coords), не через matrix. - **Matrix используется только для transformations внутри sprite-rect**: rotation вокруг центра, scale, shear.

Pattern для rotated sprite (rotation around center)

Sprite size 56×56, центр (28, 28):

```
CALL ZL_EmitLoadId
LD HL, 28 : LD DE, 28 : CALL ZL_EmitTranslate      ; UV center to origin
LD A, (tangent) : CALL ZL_EmitRotate              ; rotate UV around (0,0) which is sprite
center
LD HL, -28 : LD DE, -28 : CALL ZL_EmitTranslate    ; restore offset
CALL ZL_EmitSetMatrix                             ; emits BITMAP_TRANSFORM_A..F (6 DL cmds)
FT_BitmapHandle 0 / FT_BitmapSource ...
FT_Begin FT_BITMAPS
LD A, cell : CALL FT.Coprocessor.Cell
LD BC, (X-28)*16 : LD DE, (Y-28)*16 : CALL FT.Coprocessor.Vertex2f
FT_End
```

Matrix формула: $M = T(28, 28) * R(\text{angle}) * T(-28, -28)$. Combined rotations (tangent + spin) можно складывать в одну: $R(\text{tangent} + \text{spin}) \rightarrow \text{один cmd_rotate}$.

12.2 cmd_scale convention

`cmd_scale(sx, sy)` где `sx, sy` — f16.16 fixed-point. `scale(N, N)` **отображает bitmap в N раз больше** на экране (= sprite displayed at N× native size), не наоборот. Counter-intuitive потому что matrix transforms UV.

Пример: bg хранится 400×300 в RAM_G, нужно отобразить 640×480. Scale factor = 640/400 = 480/300 = 1.6. `cmd_scale(0x1999A, 0x1999A)` (= 1.6 in f16.16).

12.3 BITMAP_SIZE при upscale

`FT_BitmapSize filter, wrap_x, wrap_y, screen_width, screen_height` определяет **output area on screen** (clipping bounds). При upscale указываем целевой размер 640×480, не native размер bitmap.

`FT_BitmapLayout format, stride_bytes, native_height` определяет storage в RAM_G — `stride = native_width × bpp`, `height = native_height` (= 300 для 400×300 RGB565).

Filter `FT_BILINEAR` (vs `FT_NEAREST`) даёт smooth interpolation между native pixels при upscale — обязательно для качественного render scaled bitmap.

12.4 Memory budget для bg (1 MB RAM_G FT812)

bg 640×480 в разных форматах: | Format | Size | Quality | Эмулятор Unreal | |---|---|---|---| | RGB565 (full) | 614 KB | high | ✓ | | RGB565 + scale 0.5x (320×240) | 154 KB | средне | ✓ | | RGB565 + scale 0.625x (400×300) | 240 KB | хорошее (compromise) | ✓ | | RGB332 (1 byte/px) | 307 KB | плохо | ✓ | | L8 grayscale | 307 KB | grayscale only | ✓ (диагностика) | | PALETTED8 (1 byte index + 1 KB ARGB8 palette) | 308 KB | хорошее | X серый фон | | PALETTED4 / PALETTED4444 | 154 KB | 16 colors | X серый фон |

Unreal эмулятор НЕ реализует palette-formats — серый фон при попытке. На реальном железе ZX-Evo+FT812 PALETTED должен работать (стандарт FT81x).

12.5 Asymmetric downscale (X≠Y)

Можно хранить bg с разными scale по осям. Пример: 480×240 RGB565 (X=0.75×, Y=0.5×) = 230 KB. $\text{cmd_scale}(640/480, 480/240) = \text{cmd_scale}(1.33\times, 2\times)$. Полезно если detail неравномерно: больше горизонтально (Y blur приемлем) или вертикально.

Для типичных Zuma backgrounds (rotational symmetry — спираль, swirley) — detail изотропен, симметричный downscale (320×240, 400×300) лучше.

12.6 spgbld page-padding gotcha

Каждая spgbld page = 16384 байт. Если data меньше → padding zeros. При upload через циклы `FT.WriteMem 16384` → padding zeros пишутся в RAM_G **после** реальных данных.

Опасность: если RAM_G layout плотный (data1 immediately followed by data2), padding data1 затирает начало data2. Решения: 1. Order: data2 ПОСЛЕ data1 (padding идёт после data2 в свободную область). 2. Gap: ≥16384 байт между блоками. 3. Точный last-page byte count: передавать `BC = real_size mod 16384` для last page.

См. также Главу 12 (root cause flicker chain 2026-05-09: bg-padding затирал atlas).

12.7 Compression: PNG/JPEG

GPU FT812 рендерит **только uncompressed pixel buffer** в RAM_G (нужен random pixel access). PNG/JPEG как source — только для compression в spg-файле: - `cmd_loadimage` (coprocessor): JPEG/PNG → uncompressed RAM_G (single-pass decode). - `cmd_inflate` (coprocessor): zlib → uncompressed RAM_G.

Это уменьшает spg-file, но НЕ RAM_G. RAM_G всегда хранит uncompressed.

12.8 Финальный выбор для Zuma VDAC2 (level 1 spiral)

`make_bg_level01.py`: source `levels/level_src_<NN>.png` (clean 640×480) → resize 400×300 LANCZOS → RGB565 LE. `MainLoop.asm` ZL_DrawFrame: `cmd_loadidentity + cmd_scale(0x1999A, 0x1999A) + cmd_setmatrix + bg setup + Begin/Vertex2ii(0,0,1,0)/End`.

Memory: 240 KB bg + ~310 KB atlas + freedom для дальнейших assets (frog, score, particles).

12.8.1 Полный pipeline компрессии bg (нюансы практики)

Workflow `make_bg_level01.py` → spg → RAM_G:

1. **Source PNG** — `levels/level_src_<NN>.png` (clean 640×480). НЕ jpeg оригинал, потому что jpg-artifacts усиливаются после downscale + bilinear upscale в FT812.
2. **Downscale 640×480 → 400×300** (LANCZOS) на Z80-стороне через Python. **Важно** — LANCZOS, не BICUBIC: на резких границах spirale Zuma BICUBIC ringing artifacts.
3. **RGB565 LE pack** — каждый пиксель 2 байта `((g>>2 & 7)<<13) | (b>>3) | ...` little-endian. На FT812 LE — нативный порядок.
4. **Запись в .bin файл** размером 240 000 байт.
5. **spgbld pack** — `Block = #0000, #07..#15, bg_level01_pNN.bin` (15 pages × 16384 = 245 760 байт, padding 5760 zero-bytes на последней page).
6. **Z80 upload-loop** в `Initialize:` ставит page в slot 2, копирует через `FT.WriteMem` 16384 байт за раз в RAM_G начиная с `BG_RAMG_ADDR=#010000`.

7. **DL render** в `ZL_DrawFrame`: `loadidentity` + `cmd_scale(0x1999A, 0x1999A)` + `setmatrix` + `BITMAP_LAYOUT FT_RGB565, ZL_BG_W*2, ZL_BG_H` (stride 800 байт, height 300) + `BITMAP_SIZE FT_BILINEAR, BORDER, BORDER, 640, 480` + `Begin BITMAPS / Vertex2ii(0,0,1,0) / End`.

12.8.2 Bug retro: bg-padding затирает atlas (#0A6000..#0A8000)

Эта история относится к §12.6 page-padding gotcha. **Хронология:** - bg первый раз грузился ПОСЛЕ atlas. atlas в `#050000..#0A6000` (302 KB старая версия 6×8 frames). bg в `#010000..#04A800` (240 KB). - spgbld bg padding = `0A8000 - 04A800 = 5C800` нулей. Они шли в `#04A800..#0A8000`, **затирая первые 8 KB atlas** (`#0A6000..#0A8000`) — это были последние пара cells, рендерились как пустые → flicker chain. - Fix: bg грузится **первым**, atlas — вторым. Atlas pages пишут поверх bg-padding в `#0A6000+` свежими данными → atlas цел.

Универсальное правило для FT812-проектов: **порядок upload pages = обратный к RAM_G layout** (старший адрес последним), либо gap ≥16 KB между блоками.

Глава 13. Frog composition: HD-стиль pipeline (опыт 2026-05-09/10)

Композиция лягушки в Zuma-Deluxe (HD-версия github.com/GalaxyShad/Zuma-Deluxe-HD) — multi-sprite с rotation matrix. В VDAC2 реализуется через FT812 multiple BITMAP_HANDLE + matrix manipulation.

13.1 Источники подспрайтов в `frog.png`

`frog.png` (324×648 RGBA) — sprite-sheet. Координаты 1:1 из `Zuma-Deluxe-HD/src/zuma/ResourceStore.c`:

Sprite	crop (X,Y,W,H)	Назначение
<code>SPR_FROG</code>	(0, 0, 162, 162)	body (frog с открытым ртом)
<code>SPR_FROG_TONGUE</code>	(162, 0, 162, 162)	язык (накладывается над body)
<code>SPR_FROG_PLATE</code>	(162, 162, 162, 162)	круглый диск-подставка
<code>ANIM_FROG_BLINK[0..2]</code>	(0, 162N, 162, 162)	моргание (frames N=1,2,3)
<code>ANIM_FROG_BALLS</code> ×6	(234, 633, 15, 15) горизонтальный strip	индикаторы цвета next-ball

Resize 162→122 (LANCZOS) даёт scale ≈ 0.753. Соотношение body/ball = 122/40 ≈ 3.05 (HD соотношение 162/48 = 3.375; -10% — компромисс под 640×480).

13.2 Render pipeline (Frog_Draw порядок)

Из HD `Frog.c`:


```

Frog_Draw:
    DrawSprite(plate)                // no rotation
    DrawSetAngle(angle - π/2)
    DrawSprite(body)                // rotated
    DrawSprite(tongue, pos + tongueExpand·dir) // rotated
Frog_DrawTop:
    DrawSprite(currentBall, pos + ballExpand·dir) // rotated
    DrawSetScale(1.5)
    DrawSprite(nextBallIndicator, pos - 40·dir) // rotated
    DrawSprite(blinkAnim, pos)      // rotated

```

VDAC2 эквивалент в `Frog.asm`: 1. `Frog_DrawPlate` — handle 4, no matrix (обнулять matrix не нужно если предыдущий блок identity). 2. `Frog_DrawBody` — handle 2, matrix `T(61,61) · R(angle-64) · T(-61,-61)` (где 61 = sprite_W/2, -64 = -π/2 для native face=south). 3. `Frog_DrawTongue` — handle 5, та же matrix как body + offset Vertex2f на `tongueExpand·dir`. (на текущем этапе отключён до реализации recoil).

13.3 Rotation matrix pattern (см. также §12.1)

```

Frog_DrawBody:
    CALL ZL_EmitLoadId
    LD HL, 61 : LD DE, 61 : CALL ZL_EmitTranslate ; T(+61,+61)
    LD A, (Frog_Angle) : CALL ZL_EmitRotate ; R(angle-64), -64 встроен в EmitRotate
    LD HL, -61 & 0xFFFF : LD DE, -61 & 0xFFFF
    CALL ZL_EmitTranslate ; T(-61,-61)
    CALL ZL_EmitSetMatrix ; emit BITMAP_TRANSFORM_A..F

    FT_BitmapHandle 2
    FT_BitmapSource FROG_RAMG_ADDR
    FT_BitmapLayout FT_ARGB4, 122*2, 122
    FT_BitmapSize FT_BILINEAR, FT_BORDER, FT_BORDER, 122, 122
    FT_Begin FT_BITMAPS
    LD BC, FROG_VTX_X : LD DE, FROG_VTX_Y : CALL FT.Coprocessor.Vertex2f
    FT_End

    ; reset → identity для последующих ops в DL
    CALL ZL_EmitLoadId : CALL ZL_EmitSetMatrix
    RET

```

`FROG_VTX_X = FROG_X*16 - 61*16` (subpixel top-left). `Frog_Angle` — raw BRAD 0..255 (0=east, 64=south, 128=west, 192=north). `ZL_EmitRotate` сам делает `ADD A, 192` (= -64) для коррекции native face direction.

Ключевой урок: matrix НЕ задаёт screen position (это делает Vertex2f), matrix трансформирует **UV-чтение** внутри bitmap-rect. `T(+61,+61)` переносит UV-origin в центр sprite, `R(angle)` вращает UV вокруг этого origin, `T(-61,-61)` возвращает; результат — rotated bitmap внутри своего фиксированного screen-rect.

13.4 Atan2 от курсора → angle

Источник: `c:\z80\zuma\zuma_new_spg.asm:793 ComputeFrogAngle` (TS-Conf версия, скопирован 1:1 в `Frog.asm`). Алгоритм:

1. `dx = SmoothMouseX - FrogX`, `dy = SmoothMouseY - FrogY` (16-bit signed).
2. **Флаги октанта** (3 бита): `b0=dx<0`, `b1=dy<0`, `b2=swap` (если `|dy|>|dx|`).
3. `|dx|`, `|dy|` через CPL+INC. Swap так чтобы C = max, E = min.
4. `t = E*128 / C` (16-bit/8-bit деление). 128, не 32 — даёт 4× разрешение и плавность на диагоналях.
5. **Atan LUT[129]**: `atan(i/128) × 256/(2π)`, i=0..128, выход 0..32 BRAD.

6. **Mirror at 90°** если был swap: `A = 64 - A`.

7. **Apply квадрант** по флагам dx/dy: Q1 → A, Q2 → 128-A, Q3 → 128+A, Q4 → 256-A.

Возвращает BRAD 0..255: 0=east, 64=south, 128=west, 192=north.

13.5 Hybrid follow для плавного rotation

Прямое присваивание `Frog_Angle = computed` даёт jitter при mouse-jitter (kempston через Hyper-V — особенно): - Big diff (≥ 4 BRAD = $> 5.6^\circ$) → snap - Small diff (1..3 BRAD) → ± 1 BRAD/frame ramp - Diff = 0 → no-op - Deadzone: `max(|dx|, |dy|) < 5` → не менять (курсор в frog-center)

Subjective результат: при медленном движении мыши лягушка плавно догоняет, при быстром — мгновенно прыгает. То же самое было в TS-Conf версии, проверено годами.

13.6 Tongue bbox (для будущего расчёта tongueExpand)

Native tongue (162×162 region из frog.png): - bbox непрозрачных пикселей: x=59..103, y=53..132 (45×80) - centroid (81, 89), sprite center (81, 81)

Это значит native tongue: язык чуть выше центра sprite до низа. После resize 162→122 bbox переходит в y=40..99. Если рендерить tongue в той же position что и body, язык **физически выше центра body** (до y=40 после resize).

В HD `tongueExpand=24` (idle) сдвигает tongue по `dir·24` вниз по native-face=south — язык легализуется под подбородком body. На рендере в VDAC2 (где rotation atan2-driven) это значит:

```
tongueX = FROG_X + (24*cos_lut[Frog_Angle]) >> 4
tongueY = FROG_Y + (24*sin_lut[Frog_Angle]) >> 4
Vertex2f((tongueX-61)*16, (tongueY-61)*16)
```

- та же matrix что и body. Реализуем когда дойдём до recoil/fire анимации.

Глава 14. RNG: LFSR Galois + bias + RTC-scramble (опыт 2026-05-10)

14.1 LFSR Galois 16-bit

Базовый PRNG, периодом 65535 (на любом non-zero seed):

```
LFSR16:                                ; state в HL
    LD A, L : AND 1                      ; LSB
    SRL H : RR L                          ; HL >>= 1
    JR Z, .no_xor
    LD D, #B4 : LD E, 0                  ; poly 0xB400 (CRC-16-IBM reverse)
    LD A, H : XOR D : LD H, A
    LD A, L : XOR E : LD L, A
.no_xor:
    RET                                  ; HL = новое state
```

Альтернативные полиномы `#D008`, `#A005` — те же свойства period-65535.

14.2 Bias-ловушка: **AND N + clamp** на не-степени двойки

Распространённая ошибка для распределения LFSR-output на N значений:

```
LD A, L
AND 7           ; 0..7
CP 6
JR C, .ok
SUB 6           ; 6→0, 7→1
.ok:
RET             ; A в 0..5
```

Проблема: distribution неравномерное. Для NUM=6: - Values 0, 1: вероятность $2/8 = 25\%$ **каждое** - Values 2..5: вероятность $1/8 = 12.5\%$ **каждое**

Visible эффект: на экране **в 2 раза больше синих и красных шаров** (если color 0=blue, 1=red).

14.3 Mul-then-shift: равномерное распределение

```
LD A, L : XOR H           ; смешать обе половины LFSR (8 бит entropy)
LD H, 0 : LD L, A
LD D, 0 : LD E, A
ADD HL, HL                ; HL = A*2
ADD HL, DE                ; HL = A*3
ADD HL, HL                ; HL = A*6 (A * NUM_COLORS=6)
LD A, H                   ; A = (A*N) >> 8 → 0..N-1
RET
```

Distribution: $256/N$ не делится нацело → bias $\leq 1/N$. Для N=6 максимальное отклонение $2 / 256 = 0.78\%$.

Generic вариант для произвольного N:

```
LD E, N                   ; multiplier из RAM
LD HL, 0
LD B, 8
.loop:
  ADD HL, HL
  SLA A
  JR NC, .skip
  ADD HL, DE
.skip:
  DJNZ .loop
LD A, H                   ; (A * N) >> 8
RET
```

14.4 RTC-scramble seed (для разнообразия per launch)

LFSR с фиксированным seed → одна и та же последовательность каждый запуск. Решение — scramble через TS-Conf RTC секунды:

```

ReadRTCSeconds:
    LD BC, #DFF7 : XOR A : OUT (C), A      ; reg 0 = seconds
    LD BC, #BFF7 : IN A, (C)              ; A = BCD seconds
    LD B, A
    AND $0F : LD C, A                    ; low nibble
    LD A, B : AND $F0
    SRL A : SRL A : SRL A : SRL A        ; high nibble
    LD B, A
    ADD A, A : ADD A, A : ADD A, B        ; *5
    ADD A, A                               ; *10
    ADD A, C                               ; +low → 0..59 binary
    RET

VDC_Init:
    LD HL, #ACE1 : LD (VDC_LfsrSeed), HL
    CALL ReadRTCSeconds
    OR A : JR NZ, .have : LD A, 17        ; защита если RTC=0
.have:
    LD D, A : LD E, A                    ; multiplier
    LD HL, (VDC_LfsrSeed) : LD A, L       ; A = low_byte(seed)
    LD HL, 0 : LD B, 8
.mul:
    ADD HL, HL : SLA A : JR NC, .skip      ; HL = low_byte * RTC_sec через 8x mult
    ADD HL, DE
.skip:
    DJNZ .mul
    LD A, H : OR L : JR NZ, .ok
    LD HL, #1234                          ; protection если результат=0
.ok:
    LD (VDC_LfsrSeed), HL
    RET

```

Каждая секунда (RTC ticks) = разный multiplier → разное seed → разная LFSR-цепочка цветов в каждом запуске.

Глава 15. Frog с полной HD-композицией (2026-05-10)

15.1 Render order (HD-1:1)

plate (no rotation)	– диск под лягушкой
body (rotation matrix)	– frog с лапами + face/mouth
tongue (rotation matrix)	– язык, position = pos + tongueExpand·dir
ball-now (no rotation)	– выстреливаемый шар, position = pos + ballExpand·dir
next-ball (no rotation)	– индикатор на спине, position = pos - 28·dir
overlay (rotation matrix)	– face без лап (HD blink frame 0), маскирует корни tongue

Все 4 rotated спрайта (body, tongue, overlay) — **одного размера 122×122**. Это критично для **feature alignment**: после rotation eyes body и eyes overlay должны совпадать → они должны быть на одинаковых **относительных** pixel-offsets от sprite centra. Разный размер → разные относительные offsets → "moon-like" дрейф features при rotation.

15.2 RAM_G layout (1 МБ, baseline 2026-05-10)

```
#010000..#04C000  bg (15 pages, 400×300 RGB565 + scale 1.6 upscale)
#04C000..#04E000  killzone (1 page real, в bg padding zone)
#050000..#09C000  balls atlas (19 pages - 6 colors × 8 phases × 56×56 ARGB4)
#09C000..#0A4000  body 122×122 ARGB4 (2 pages)
#0A4000..#0AC000  plate 122×122
#0AC000..#0B4000  tongue 122×122
#0B4000..#0BC000  overlay 122×122 (HD blink frame 0)
свободно          272 КБ для будущих assets
```

Balls atlas сжат с 16 phases до 8 — освободило 18 pages для overlay full-size. Chain spin formula поменялась: `& 7` вместо `& 15`, `cell = color*8` вместо `*16`.

15.3 Tongue — pos + tongueExpand·dir (HD orbit)

В отличие от tight-cropped sprite (32×80) с pivot (16, 29) — full 122×122 sprite даёт ту же rotation pattern что body:

```
; Frog_DrawTongue:
;   matrix = T(61, 61) · R(angle + 192) · T(-61, -61)
;   Vertex2f at (TmpX-61, TmpY-61), screen rect 122×122
;   TmpX = PosX + cos·tongueExpand/128
;   TmpY = PosY + sin·tongueExpand/128
```

`tongueExpand = 24` idle (HD), 0..24 при выстреле. Tongue native асимметричный (stripe внутри 162×162 native занимает y=53..133), поэтому при rotation вокруг centra (61, 61) tongue tip "выходит" из mouth area body.

15.4 Ball-now / Next-ball через chain atlas (handle o)

Используется **тот же** atlas что и chain rendering. Cell = `color*8 + 0` (frame 0, не вращается). Native размер 56×56, рендерится без cmd_scale.

```
; Frog_DrawBallNow:
;   no rotation matrix (identity).
;   BITMAP_HANDLE 0 / SOURCE BALLS_RAMG_ADDR / LAYOUT 56*2/56 / SIZE 56/56.
;   Cell(ballColor*8) → frame 0 selected color.
;   Vertex2f((TmpX-28)*16, (TmpY-28)*16), centred at TmpX, TmpY.
;   TmpX = PosX + cos·ballExpand/128 (idle = 24).
```

Next-ball аналогично, но `pos - NEXT_OFFSET·dir` (= -28·dir, на спине после rotation body).

15.5 Recoil cycle (HD-style fire animation)

ЛКМ rise-edge → `isFire=1, recoilTick=0, ballExpand=0, ballColor=nextBallColor, nextBallColor=random(0..3)`.

Каждый кадр: - `recoilTick += 10 BRAD` (≈0.245 rad, HD = 0.25). - `recoil = sin(recoilTick)` (signed byte, -127..127). - Пока recoil ≥ 0: - `tongueExpand = 24 - (recoil·24)>>7` → язык втягивается в рот (24→0). - `ballExpand += 2` (cap 24) → шар выезжает. - `pos = posStart - (cos·recoil)/2048, posStart - (sin·recoil)/2048` → тело откатывается на ~8 px max. - recoil < 0 → end fire, всё в idle.

Полу-цикл синуса = 13 кадров (≈260ms на 50fps), полное возврат ballExpand до 24 — ещё ~5 кадров.

15.6 FT81x cmd_scale: matrix хранит INVERSE

Param scale = visual ratio = output/native: - bg upscale 400→640: `cmd_scale(1.6)` = 0x1999A. Matrix внутри $S(1/1.6) = S(0.625)$. UV = 0.625·screen → UV(640) = 400. ✓ samples full bg. - ball downscale 56→32: `cmd_scale(0.5714)` = 0x9249. Matrix $S(1.75)$. UV = 1.75·screen → UV(32) = 56. ✓ samples full ball.

Документация FT81x неоднозначна — проверять empirically через bg upscale.

15.7 Critical bugs found and fixed (2026-05-10 session)

Bug 1 — Frog_ComputeAngle truncate без clamp. `LD C, L` для `|dx| > 255` обрезает high byte H, остаётся младший байт. E.g., dx=313=0x139 → C=0x39=57. Swap-логика `|dy| > |dx|` инвертируется → frog резко крутится у краёв экрана.

Fix:

```
.dx_pos:
    LD    A, H
    OR    A
    JR    Z, .dx_clamped
    LD    L, 255                ; saturate to 255 if H ≠ 0
.dx_clamped:
    LD    C, L                ; true 8-bit clamp
```

То же для `.dy_pos`.

Bug 2 — Frog_DrawNextBall забывал cmd_scale. DrawBallNow применял scale matrix, DrawNextBall пропускал → next-ball рендерился at native 56×56 в screen rect 32×32, центрирован через 16-px half → визуально "огромный шар" с неправильной позицией.

Fix: либо добавить scale matrix в DrawNextBall, либо (как в baseline 2026-05-10) убрать scale из обеих функций и рендерить native 56×56.

Bug 3 — Multi-sprite feature alignment. Body 122 + overlay 80 → eyes на разных pixel-offsets от sprite centra → после rotation eyes body и overlay расходятся → "две точки вращения как Луна".

Fix: все спрайты с одинаковыми features ОДНОГО размера. Required: balls atlas 16→8 phases для освобождения RAM_G.

15.8 Python visual_emulator.py — prototype-first workflow

Прототипирование parameters (rotation formula, pivot, offsets, recoil curve) в `visual_emulator.py` (tkinter+PIL) даёт X30-X100 ускорение vs цикла sjasmplus → spgbld → Unreal. Параметры подбираются интерактивно через keys (стрелки, `[ / ]`, `, / .`, `r`), затем переносятся в asm как численные EQU.

Visual emulator не симулирует FT812 cmd_translate/rotate/scale 1:1 — но даёт визуальный **target behavior** для asm transfer. Различия rendering pipeline (PIL bilinear vs FT812 BILINEAR + cmd_scale convention) могут давать ±1-2 px смещения, но архитектурные параметры (radii, pivots, formulas) переносятся точно.

Ключевые количественные приёмы:

- **scale 1.6** = `0x1999A` в f16.16 ($0.6 \times 65536 \approx 0x9999$, целая 1 = 0x10000). Не `0x19999`, не `0x1A000` — точное значение.
- **stride** = **native_width** × **bpp**, HE display_width. Для 400×300 RGB565 stride = 400×2 = 800. Если поставить 1280 (= 640×2 для display) — битмап читается из неправильных адресов в RAM_G, на экране каша.

- **BITMAP_SIZE.W/H = display, BITMAP_LAYOUT.height = native.** Это обязательная асимметрия: SIZE определяет output rect (для clipping), LAYOUT — storage в RAM_G.
- **FT_BILINEAR обязательно** для качества. С **FT_NEAREST** 400×300→640×480 даёт ступеньки на диагоналях.

Что НЕ использовали и почему:

Approach	Причина отказа
Full 640×480 RGB565 (614 KB)	занимает 60% RAM_G, не оставляет места под atlas (300+ KB)
320×240 RGB565 + scale 2× (154 KB)	заметная потеря деталей на детализированной spirale
RGB332 (307 KB)	работает, но 256 цветов + dithering = грязный gradient на воде
PALETTED8 (308 KB + 1 KB palette)	Unreal эмулятор не поддерживает — серый экран. На реальном железе должно работать (стандарт FT81x), но без возможности отладки на эмуляторе — не используем.
cmd_loadimage JPEG decode	RAM_CMD coprocessor decode 640×480 на загрузку (overhead ~2 сек), нужен cmd_inflate для zlib потоков; spg-файл становится меньше, но RAM_G всё равно uncompressed. Не оправдано когда spg ёмкость не критична.

Полученный bg memory layout:

```
RAM_G:
#000000..#040FFF → reserved (DL/FONT/HANDLES area FT812)
#010000..#04A8FF → bg_level01 (240 000 bytes RGB565 400×300)
#04A900..#04FFFF → bg padding (~5 KB) + free
#050000..#0E4FFF → balls atlas (602 112 bytes ARGB4 6×16×56×56)
#0E5000..#0FCFFF → frog body/plate/tongue (3×30 KB ARGB4 122×122)
#0FD000..#0FEFFF → killzone (8 KB)
```

Дальше по AvailableRamG ещё ~6 KB до 1 MB конца — запас для score, particles.

Глава 16. FT81x DL persistent state — Cell, BITMAP_HANDLE и ловушки наследования (2026-05-10)

После сборки полной HD-композиции лягушки (глава 18) проявился неприятный интермиттент-баг: «крышка» (face overlay) иногда исчезала на N кадров после выстрела. Видимое поведение — после fire ~75% случаев overlay пропадает до следующего fire, ~25% случаев overlay виден.

Что мы исключили (типичные кандидаты, оказавшиеся неверными)

1. **Координата overlay (recoil-сдвиг)** — **Frog_PosX/Y** смещаются на ±8 px во время recoil. Заменяли вычисление overlay-вершины на **Frog_PosStartX/Y** (статика). Баг **остался** → координаты ни при чём.
2. **Cmd-buffer overflow** — буфер CMD_ADDRESS_PTR=#C000 на 16 KB, фактически используется ~2.5 KB за кадр. Не близко к лимиту.
3. **Coprocessor exception** — после exception coprocessor останавливается, всё что после игнорируется. Но overlay рендерится ПЕРЕД chain block, и chain рендерится корректно → coprocessor жив.

4. **DL пострадал** — снимок Z80 RAM (F12-dump) показал что DL для overlay полностью корректный: handle=6, source=#0B4000, ARGB4 244×122 BILINEAR, matrix valid, vertex (266, 170) внутри 640×480.
5. **Matrix corruption** — overlay использует **ту же matrix** что body ($T(61) \cdot R(\text{angle}+192) \cdot T(-61)$). Body не пропадает, overlay пропадает → matrix не виновата.
6. **RAM_G corruption** — overlay area #0B4000..#0BB740 не имеет writers после Initialize (никто туда не пишет). Layout правильный, padding tongue заканчивается ровно на #0B4000 (overlay start), не наезжает.

Root cause — Cell как persistent DL state

Frog block рендерит спрайты в порядке:

plate	handle 4	Vertex2f	без Cell	→ cell наследован
body	handle 2	Vertex2f	без Cell	→ cell наследован
tongue	handle 5	Vertex2f	без Cell	→ cell наследован
ball-now	handle 0	Cell(BallColor*8)	+ Vertex2f	→ cell ставится
next-ball	handle 0	Cell(NextBallColor*8)	+ Vertex2f	→ cell перезаписывается
overlay	handle 6	Vertex2f	без Cell	→ cell НАСЛЕДОВАН от next-ball!

Перед frog-блоком идёт bg, который рендерится через `Vertex2ii(0, 0, 1, 0)`. Vertex2ii — **специальная** компактная команда, которая включает в себя **handle и cell прямо в опкоде** (поля 7 бит handle, 7 бит cell). Она ставит DL state cell=0 как побочный эффект.

После bg DL state: **cell=0**. Killzone, plate, body, tongue читают этот cell=0. Когда ball-now эмитит `Cell(BallColor*8)` — DL state cell меняется. Next-ball аналогично. После next-ball cell = NextBallColor*8.

Overlay не эмитит Cell перед своим Vertex2f → **наследует cell от next-ball**.

Overlay = 122×122 ARGB4 stride 244 = 29768 байт = 1 cell в layout. FT81x вычисляет адрес pixel-data:

```
addr = BITMAP_SOURCE + cell * cell_size_bytes
      = OVERLAY_RAMG_ADDR + cell * 29768
      = #0B4000 + cell * 0x7448
```

Для NextBallColor=1 → cell=8 → addr = #0B4000 + 829768 = #EE200. Это далеко за пределами реального overlay sprite в RAM_G (overlay-data заканчивается на #0BB740 < #EE200). Зона #EE200 — не используется, в RAM_G там zeros. ARGB4 нулевые байты = alpha=0 для всех пикселей → overlay полностью прозрачный → невидим*.

Когда NextBallColor=0 (= 25% случаев в randomize 0..3) → cell=0 → читаем правильный overlay из #0B4000 → виден. Отсюда **интермиттент**.

Fix

```
Frog_DrawFaceOverlay:
    ; ... matrix setup ...
    FT_BitmapHandle 6
    FT_BitmapSource OVERLAY_RAMG_ADDR
    FT_BitmapLayout FT_ARGB4, FROG_SPR_W * 2, FROG_SPR_W
    FT_BitmapSize    FT_BILINEAR, FT_BORDER, FT_BORDER, FROG_SPR_W, FROG_SPR_W
    FT_Begin FT_BITMAPS
    XOR  A
    CALL FT.Coprocessor.Cell      ; <-- сброс cell в 0
    CALL Frog_EmitVertex2f_PosCentered
    FT_End
    ...
```

`FT.Coprocessor.Cell` = TSLib helper, эмитит DL command `0x06000000 | (cell & 0x7F)`.

Универсальное правило: какой DL state в FT81x persists

Команда	Persists	Scope
BITMAP_HANDLE	да	global
BITMAP_SOURCE	да	per-handle
BITMAP_LAYOUT/SIZE	да	per-handle
BITMAP_TRANSFORM_A..F	да	global
CELL	да	global
COLOR_RGB	да	global
COLOR_A	да	global
BLEND_FUNC	да	global
LINE_WIDTH	да	global
POINT_SIZE	да	global
SCISSOR_XY/SIZE	да	global

Practical rule: **любой Vertex2f, идущий после atlas-блока (где Cell≠0 был эмитен), должен явно эмитить нужный Cell** (даже Cell(0) для single-cell sprite). Не полагайся на наследование = 0 by default.

Нюанс `VERTEX2II` vs `VERTEX2F`. Ловушка наследования `CELL` касается прежде всего `VERTEX2F`: он берёт cell из persistent-состояния. У `VERTEX2II` номер ячейки (cell, биты 0..6) и handle зашиты **прямо в 32-битную команду**, поэтому такой вершине «унаследованный» `CELL` не страшен. **НО** `VERTEX2II` при этом сам **перезаписывает** глобальный `CELL` для последующих команд — так что если дальше в кадре идёт `VERTEX2F`, он подхватит cell от предыдущего `VERTEX2II`. Правило «эмить `CELL` явно» остаётся в силе именно из-за этого взаимодействия.

Методология поиска

Ловушка для одиночного отладчика — баг локализован в DL pipeline state, который **не виден в дампе Z80 RAM** (DL state живёт в FT81x регистрах). Дамп показывал все правильные команды; вычислить

наследование Cell можно только мысленным прохождением DL.

Diagnostic A (изолировать координату): заменить Vertex источник на статику (Pos→PosStart). Баг не ушёл → не координаты.

Главная подсказка пришла от пользователя: «чем крышка отличается от остальных слоёв спрайта» — заставило сесть и **последовательно сравнить** overlay с другими frog-спрайтами по всем атрибутам. Различие в **позиции в DL pipeline относительно atlas-блока** (overlay = единственный single-cell sprite ПОСЛЕ atlas-блока) и привело к Cell.

Похожие ловушки могут возникнуть с **любым** persistent DL state. При добавлении новых sprite в frame — пройти по всем persistent settings и проверить, что текущий sprite их не наследует случайно (или явно ресетит).

Глава 17. Vsync-first sync: race между Z80 build и FT812 render (2026-05-10)

После сборки полного gameplay loop (chain physics + bullet + match-3) на реальном железе ZX-Evo + FT812 проявился класс артефактов: «цветной мусор / линии посередине экрана при ≥ 30 шарах в цепи». На эмуляторе Unreal x64 артефакт **минимален или отсутствует**. На железе — линейно нарастает с числом шаров и **усиливается при движении мыши**.

Гипотезы и проверка

Гипотеза 1 — RAM_CMD overflow (4 KB ring). Решение TSLib `FT.Coprocessor.Write` уже опрашивает `REG_CMDB_SPACE` перед каждой SPI-записью и ждёт пока coprocessor освободит место. **Overflow невозможен** через TSLib API. Гипотеза отклонена.

Гипотеза 2 — RAM_DL overflow (8K commands = 32 KB). Подсчёт DL команд на кадр: bg ~24 + killzone ~14 + frog 6 спрайтов с matrix ~75 + chain N шаров $\times 8$ + cursor ~14 + bullet $1 \times 8 \approx 149 + 8N$. При 60 шарах ≈ 629 DL — **далеко от 8192 лимита**. Подтверждено визуально через красную полосу внизу экрана (диагностика, потом убрана). Отклонено.

Гипотеза 3 — cmd_swap через CMD-FIFO вместо REG_DLSPWAP. По FT81x документации `cmd_swap` = «coprocessor сам выполнит swap когда DL готов». Заменили manual `REG_DLSPWAP=FRAME` на `FT_CMD_Swap`. **Программа зависла** (deadlock в CMD-FIFO). Откат, гипотеза отклонена.

Гипотеза 4 — vsync-first sync (HighLander). Кадровый sync с FT812 vsync **перед** SPI write. Идея: write попадает в vblank window, не накладывается на render. На железе **частично помогло** — артефакт исчез при стационарной мыши, остался при mouse motion.

Корень оставшегося артефакта: тяжёлый build при mouse motion. `ZL_AimUpdate` детектит motion → `Frog_ComputeAngle` запускается с atan2 LUT[129] + hybrid follow + 8-octant logic = десятки сложений/делений. Build удлиняется, SPI write **выходит за vblank window** в render time → race с FT812 RAM_DL read.

Решение — parallel build + vsync-first write

Перестраиваем main loop:

```
.Loop      ; --- 1. Input + game state + Build DL в Z80 buffer ---
           ; ВЫПОЛНЯЕТСЯ ПАРАЛЛЕЛЬНО с FT812 рендером prev frame.
CALL Input.Mouse.UpdateMouseState
CALL ZL_AimUpdate           ; mouse/keyboard → Frog_Angle
CALL ZL_SmoothMouse
CALL Frog_Update            ; ComputeFrogAngle + recoil
CALL VDC_Update
CALL Bullet_Update
CALL Bullet_CheckCollision
FT_CMD_Start                ; reset Z80 buffer ptr
FT_DL_Start                 ; cmd_dlstart
FT_VertexFormat 4
FT_ClearColorRGB32 0x102030
FT_ClearAll
CALL ZL_DrawFrame           ; bg + frog + chain + cursor + bullet
FT_Display

.WaitIntSync ; --- 2. Sync с FT812 vsync ПОСЛЕ build ---
FT_RD_REG8 FT_REG_INT_FLAGS
AND FT_INT_SWAP
JR Z, .WaitIntSync

.WaitDLSwap FT_RD_REG8 FT_REG_DLSWAP
AND 3
JR NZ, .WaitDLSwap

           ; --- 3. Burst write Z80 buffer → FT812 RAM_CMD (в vblank window) ---
FT_CMD_Write
CALL FT.Coprocessor.WaitFlush
FT_WR_REG8 FT_REG_DLSWAP, FT_DLSWAP_FRAME
JP .Loop
```

Ключевое отличие от предыдущей схемы: **wait FT INT_SWAP** перенесён **в середину loop**, между build и write. Z80 cycles на input + game state + DL build идут **в параллель** с тем, что FT812 рендерит предыдущий кадр. Когда Z80 готов — ждёт vsync, затем SPI burst попадает строго в vblank.

Почему это работает

Pipeline	Build location	Write location	Race
Старая (wait в начале)	После vblank	В render time	Mouse motion → race
HighLander (wait в начале v2)	После vblank	В render time	Mouse motion → race
Parallel + vsync write	Параллельно с render	Vblank window	Нет

При mouse motion build занимает ~3-5 ms (atan2 в `ZL_KbdAimUpdate` + matrix calc для frog). FT812 render @ 57Hz занимает ~15.5 ms из 17.5 ms кадра, vblank ~2 ms. В старой схеме build ел кусок vblank → write попадал в render. В новой — build делается в render time, write строго в vblank.

Урок (универсальный)

Sync на vsync должен быть ПЕРЕД сторонним I/O write, не ПОСЛЕ. Z80-only работа (input read из port, game state update в RAM, DL build в Z80 buffer) НЕ трогает FT812 → может идти в любое время, в т.ч. параллельно с render.

Только I/O в FT812 (`FT_CMD_Write` , `FT_WR_REG`) требует vblank window. Поэтому правильный sync = «build в любое время, sync прямо перед I/O burst».

Открытое (TODO для следующей итерации)

- Подтверждение fix mouse-motion artifact на железе. Текущая версия — кандидат на полный fix.
- Hemisphere insert (target = i vs i+1 по ближайшему neighbour).

Глава 18. DXT1-эмуляция на FT812: компрессия фона до 0.5 байт/пикс через L2-mask + RGB565 blend (2026-05-12)

Задача

Фон уровня 640×480 в нативном RGB565 занимает **614 400 байт** в RAM_G FT812 — 60% от всего 1 МБ. Для multi-level игры (22 уровня Zuma Deluxe) это неприемлемо: 22 × 614 400 = 13.5 МБ — нужен какой-то стриминг или сжатие.

Раньше использовали трюк «400×300 RGB565 + cmd_scale(1.6) NEAREST до 640×480»: 240 000 байт, но качество ступенчатое (см. `reference_zuma_vdac2_bg_compression.md`). Хочется честные 640×480 при минимальном объёме.

Block-compressed форматы (DXT, ETC, ASTC) FT812 не поддерживает hardware'но. Список `BITMAP_LAYOUT.format` (FT81X PG Table 7): только ARGB1555, L1/L2/L4/L8, RGB332, ARGB2/4, RGB565, TEXT8X8, TEXTVGA, BARGRAPH, PALETTE565/4444/8. Никаких DXT/S3TC. `BITMAP_EXT_FORMAT` (под ASTC) появился только с BT815/816.

Идея

DXT1 кодирует 4×4 пиксельный блок 8 байтами: - 2 байта со endpoint (RGB565) - 2 байта с1 endpoint (RGB565) - 4 байта = 16 × 2-битных индексов выбора цвета

Декодирование на лету: для каждого пикселя индекс 0..3 определяет цвет: - `0` → `c0` - `1` → `c1` - `2` → $(2 \cdot c0 + c1) / 3$ ($\approx \frac{2}{3}c0 + \frac{1}{3}c1$) - `3` → $(c0 + 2 \cdot c1) / 3$ ($\approx \frac{1}{3}c0 + \frac{2}{3}c1$)

FT812 умеет каждый из этих кусков по-отдельности: - **со и с1 endpoint цвета** = два RGB565 цвета на блок 4×4 = массив `(W/4)×(H/4)` RGB565 - **Индекс выбора** = 2 бита на пиксель = формат `FT_L2` `W×H` - **Интерполяция между со и с1** через индекс → реализуется аппаратным **alpha-blending'ом**: L2 пишет alpha канал, со/с1 рисуются с `DST_ALPHA` / `ONE_MINUS_DST_ALPHA` blend

Это классический трюк из EVE Application Note **AN_340** (DXT1 emulation, Bridgetek). Конвертер `ft812_dxt_convert.py` (автор — TS-Labs) раскладывает обычный DXT1 в нужный layout.

Формат raw файла

```
+-----+ offset 0
| c0 plane   | RGB565, (W/4) × (H/4)
| 38400 bytes | для 640×480 → 160 × 120 cells × 2 байта
+-----+ offset 38400
| c1 plane   | RGB565, (W/4) × (H/4)
| 38400 bytes |
+-----+ offset 76800
| L2 mask    | 2 бит/пикс, W × H
| 76800 bytes | для 640×480 → 640 × 480 / 4 = 76800
+-----+ offset 153600
```

Всего: 153 600 байт для 640×480 ровно 0.5 байт/пикс — теоретический минимум среди форматов FT812 (PALETTED8 = 1 байт/пикс минимум). Экономия **75%** vs raw RGB565.

L2 alpha mapping (нелинейный)

Эмпирически FT812 декодирует 2-битный raw L2 в 8-битную alpha по таблице (0, 255, 85, 170) для (raw 0, 1, 2, 3). **Не линейно** — raw=1 → alpha=255, а не 85.

```
L2_ALPHAS = (0, 255, 85, 170)
```

Конвертер использует эту таблицу при выборе selector-ов так, чтобы итоговый композит после blend = c0 * (1-A/255) + c1 * A/255 давал:

sel	alpha	финальный цвет	смысл DXT1
0	0	c0	endpoint c0
1	255	c1	endpoint c1
2	85	$\frac{2}{3}c0 + \frac{1}{3}c1$	интерполяция
3	170	$\frac{1}{3}c0 + \frac{2}{3}c1$	интерполяция

Это **точно** DXT1 декомпрессия, без потерь относительно стандартного DXT1.

Display List — 3 прохода

```

FT_CMD_BUF (ZL_DL_SAVE_CONTEXT)
CALL ZL_EmitLoadId
CALL ZL_EmitSetMatrix

; handle 1: RGB565 color cells (cell 0=c0, cell 1=c1)
FT_BitmapHandle 1
FT_BitmapSource ZL_BG_COLOR_ADDR
FT_BitmapLayout FT_RGB565, ZL_BG_COLOR_STRIDE, ZL_BG_BLOCK_H
FT_BitmapSize FT_NEAREST, FT_BORDER, FT_BORDER, ZL_BG_W, ZL_BG_H

; handle 8: L2 mask на full resolution
FT_BitmapHandle ZL_BG_L2_HANDLE
FT_BitmapSource ZL_BG_L2_ADDR
FT_BitmapLayout ZL_FT_L2, ZL_BG_L2_STRIDE, ZL_BG_H
FT_BitmapSize FT_NEAREST, FT_BORDER, FT_BORDER, ZL_BG_W, ZL_BG_H

FT_Begin FT_BITMAPS

;--- Pass 1: L2 → alpha канал dst.A ---
FT_CMD_BUF (ZL_DL_COLOR_MASK | ZL_COLOR_MASK_A) ; только A
FT_CMD_BUF (ZL_DL_BLEND_FUNC | (ZL_BLEND_ONE << 3) | ZL_BLEND_ZERO)
FT_CMD_BUF (ZL_DL_COLOR_A | 255)
FT_Vertex2ii 0, 0, ZL_BG_L2_HANDLE, 0

;--- готовимся к color planes ---
FT_CMD_BUF (ZL_DL_COLOR_MASK | ZL_COLOR_MASK_RGB) ; только RGB
CALL ZL_EmitLoadId
FT_CMD_BUF FT_CMD_SCALE
FT_CMD_BUF #00040000 ; sx = 4.0
FT_CMD_BUF #00040000 ; sy = 4.0
CALL ZL_EmitSetMatrix

;--- Pass 2: c1 plane с DST_ALPHA blend ---
FT_CMD_BUF (ZL_DL_BLEND_FUNC | (ZL_BLEND_DST_ALPHA << 3) | ZL_BLEND_ZERO)
FT_Vertex2ii 0, 0, 1, 1 ; cell 1 = c1, out = c1 * A

;--- Pass 3: c0 plane с ONE_MINUS_DST_ALPHA сверху ---
FT_CMD_BUF (ZL_DL_BLEND_FUNC | (ZL_BLEND_ONE_MINUS_DST_ALPHA << 3) | ZL_BLEND_ONE)
FT_Vertex2ii 0, 0, 1, 0 ; cell 0 = c0, out = c0*(1-A) + dst

FT_End
FT_CMD_BUF (ZL_DL_RESTORE_CONTEXT)

```

Математика итогового пикселя:

```

после pass1: dst.A = L2_ALPHAS[selector] (∈ {0, 255, 85, 170})
после pass2: dst.RGB = c1 * dst.A / 255
после pass3: dst.RGB = c0 * (1 - dst.A/255) + dst.RGB * 1
               = c0 * (1 - A/255) + c1 * (A/255)

```

Подводные камни (на отладку ушёл вечер)

1. sjasmpplus parsing макро-аргументов с |

В `---syntax=ab` запись `FT_CMD_BUF ZL_DL_COLOR_MASK | 15` парсится криво — в макрос приходит **только первый operand** (`ZL_DL_COLOR_MASK = #20000000`), а `| 15` пропадает.

Результат: COLOR_MASK эмитится с битами `0000` (всё запрещено к записи), все последующие draw-ы становятся по-ор-ами, экран = clear color.

Лечение: ВСЕГДА оборачивать в скобки.

```
FT_CMD_BUF (ZL_DL_COLOR_MASK | 15)      ; правильно
FT_ColorMask 1, 1, 1, 1                  ; или штатный TSLib-макрос
```

2. FT_BitmapSize уже эмитит BITMAP_SIZE_H

```
FT_BitmapSize macro Filter?, WrapX?, WrapY?, Width?, Height?
    FT_CMD_BUF ((0x29 << 24) | ((W>>9)<<2) | (H>>9))    ; BITMAP_SIZE_H
    FT_CMD_BUF ((0x08 << 24) | ... | (W & 511) | ...)    ; BITMAP_SIZE
endm
```

Передаём 640/480 **напрямую** в макрос. Если попытаться вручную предварительно эмитить `FT_CMD_BUF (ZL_DL_BITMAP_SIZE_H | hi)` + потом `FT_BitmapSize` с младшими `W_LO, H_LO` — макрос **затирает** ручной SIZE_H своим (с нулевыми hi-битами, потому что W_LO=128, H_LO=480 укладываются в 9 бит). Высокие биты теряются → BITMAP_SIZE становится 128×480, draws обрезаются.

Также `FT_BitmapLayout` сам эмитит `BITMAP_LAYOUT_H` для `linstride > 1023 / height > 511`.

3. BITMAP_SIZE = screen extent, не source

Для co/c1 cells источник 160×120 + `cmd_scale(4,4)` → screen draws 640×480. `FT_BitmapSize` должен быть **640×480** (final screen extent после matrix), не source 160×120. Иначе draws обрезаются до 160×120 в верхнем-левом углу.

L2 plane (handle 8) — source уже 640×480 нативно, scale identity → BITMAP_SIZE тоже 640×480.

4. Vertex2ii max 511×511

`VERTEX2II` имеет 9-битные поля координат (max 511). Для рисования full-screen 640×480 надо использовать `Vertex2f` с `VertexFormat` 0 (1 px) или 4 (1/16 px).

В нашем случае все draws начинаются с (0,0), поэтому Vertex2ii ОК — позиция ноль помещается, а размер контролируется через BITMAP_SIZE.

Сравнение объёмов 640×480

Формат	Байт	vs DXT1-эмул
Raw RGB565	614 400	4.0×
ARGB4	614 400	4.0×
400×300 RGB565 + scale 1.6 (старый bg)	240 000	1.56×
DXT1-эмуляция (co+c1+L2)	153 600	1.0×
320×240 RGB565 + scale 2.0	153 600	1.0× (мыло)
PALETTED8	308 224	2.0×
L8 (grayscale)	307 200	2.0×

Когда использовать

ОК Фотореалистичный фон (level background, splash screen) ОК Текстуры с плавными цветовыми переходами ОК Когда RAM_G сильно ограничен (multi-level игра)

NOT Спрайты с резкими краями и небольшим количеством цветов — артефакты на границах (DXT1 теряет alpha, плохо ловит тонкие линии). Для шаров/frog эффективнее ARGB4. NOT Текст и UI — здесь DXT1 даёт «лесенки» из-за грубых endpoint цветов.

Конвертер ft812_dxt_convert.py

Опции качества (effort `-e 0..10`): `-e 0` — быстро, шумный (видны блоки 4×4 на градиентах) - `-e 3` — почти неотличим от оригинала (рекомендация TS-Labs) - `-e 6+` — perceptual weights + seam smoothing + residual diffusion, медленно

Базовый запуск:

```
python ft812_dxt_convert.py level01.png -o out/level01 -f l2 -t raw -e 3 -p
```

Выход: - `out/level01_l2.raw` — 153 600 байт raw в формате co|c1|L2 (грузим в RAM_G как есть) - `out/level01_l2.h` — C-заголовок с offset-ами/strides (для интеграции) - `out/level01_l2_preview.png` — реконструкция (для визуальной оценки качества)

Multi-level в Zuma — что меняется

22 уровня × 153 600 = 3.4 МБ DXT1-эмуляции vs 13.5 МБ raw RGB565. Сейчас один уровень упаковывается в 10 spgblt-страниц по 16 КБ. При переключении уровней upload bg = ~150 КБ через SPI ≈ 70 мс на 14 МГц Z80 (вполне допустимая пауза при level transition).

Объёмы по сравнению с zlib (`cmd_inflate` план): - DXT1-эмуляция: 153 КБ uncompressed, ~70 мс upload, hardware decode - ZX0/zlib: ~100 КБ compressed → ~150 КБ uncompressed, ~120 мс upload + decode

DXT1-эмуляция выигрывает по uncompressed size (тот же объём в SPI transfer), проще в реализации (нет decode-кода), и качество фотореалистичных фонов визуально приемлемое начиная с `-e 3`.

Источники

- **EVE Application Note AN_340** (Bridgetek, "Compressing texture using DXT1 with EVE2/EVE3 chipsets") — оригинальная идея трюка.
- `ft812_dxt_convert.py` — реализация конвертера, автор TS-Labs.
- `reference_zuma_vdac2_dxt1_emulation_l2_blend.md` — компактная памятка по технике.
- `feedback_sjasmplus_macro_or_parens.md` — про скобки в FT_CMD_BUF.

Глава 19. Апгрейд DXT1-эмуляции с L2 до L4: +50% SPI за фотокачество (2026-05-12)

Зачем понадобился L4

Глава 18 описала DXT1 на L2-маске: 0.5 байт/пикс, 153 600 байт на 640×480. Объёмно идеально, но на каменной текстуре фона `level_src_01` оставалась заметная **блочность 4×4**.

Корень: L2-маска даёт всего **4 уровня** между endpoints (`{0, 85, 170, 255}`) → четыре цвета: c_0 , $\frac{1}{2}c_0 + \frac{1}{2}c_1$, $\frac{1}{2}c_0 + \frac{3}{2}c_1$, c_1). На гладких градиентах внутри блока 8 уникальных оттенков в исходнике вынуждены коллапсировать в 4 → видна ступенька в каждом блоке.

Чтобы оценить «насколько лучше» — переходим на L4: - **16 уровней** маски (линейный ramp `0..255` шагом 17) - 4×4 блок цветов c_0/c_1 тот же, размер endpoint planes не меняется - **mask** 4bpp вместо 2bpp → +76 800 байт (76800 → 153600) - Итого raw: **230 400 байт** vs 153 600 = +50%

Пиксельный «бюджет фона» 200 КБ был принятой границей бюджета SPI/RAM_G. 230 КБ — чуть выше потолка, но bg уже **по-настоящему фотореалистичен**.

Сравнение L2 vs L4 в одном блоке

оригинал блока 4×4:	цвета на пиксель
<pre> +---+---+---+---+ A A B B A A B B C C D D C C D D +---+---+---+---+ </pre>	<pre> A = (200, 90, 60) B = (210, 130, 80) C = (180, 100, 70) D = (170, 110, 90) </pre>
L2 (4 уровня): endpoints: $c_0=A$, $c_1=D$ selectors per pixel: A→0 B→2 ($\%A+\%D$) C→3 ($\%A+\%D$) D→1 ошибка перекраски: B → $\%A+\%D$ отличается от B → видимый шов между блоками	L4 (16 уровней): endpoints: $c_0=A$, $c_1=D$ selectors per pixel: A→0 B→5 ($a\sim 85$) C→10 ($a\sim 170$) D→15 ошибка перекраски: B → $a*A+(1-a)*D$ с лучше подбираемым a → плавная интерполяция, шов невидим

Что меняется в raw layout

Только размер маски и её stride:

<pre> +-----+ c0 plane 38400 bytes +-----+ </pre>	offset 0 RGB565, (W/4) × (H/4) для 640×480 → 160 × 120 cells × 2 байта
<pre> +-----+ c1 plane 38400 bytes +-----+ </pre>	offset 38400 RGB565, (W/4) × (H/4)
<pre> +-----+ L4 mask 153600 bytes +-----+ </pre>	offset 76800 4 бит/пикс, W × H (вместо 2 бит/пикс) для 640×480 → 640 × 480 / 2 = 153600 ← x2 от L2 offset 230400

Изменения в asm (минимально)

main.asm: 10 → 15 spgbld pages

BG_FIRST_PAGE	EQU 7	
; было:		
; BG_PAGE_COUNT	EQU 10	; DXT1-decomp 640×480 (c0/c1/L2 = 153600)
; стало:		
BG_PAGE_COUNT	EQU 15	; DXT1_L4 640×480 (c0/c1/L4 = 230400, last padded)

RAM_G layout не меняется: BG занимает `#010000..#04C000` = 245 760 байт (230400 реальных + 15 360 padding из последней spgblt-страницы). Killzone сидит ровно на `#04C000` — без overlap.

MainLoop.asm: формат маски и stride

```
; было:
; ZL_BG_L2_STRIDE EQU ZL_BG_W / 4           ; FT_L2 = 2bpp → 4 пикс/байт
; ZL_FT_L2      EQU 17                     ; format code FT_L2

; стало:
ZL_BG_L2_STRIDE EQU ZL_BG_W / 2           ; FT_L4 = 4bpp → 2 пикс/байт
ZL_FT_L2      EQU FT_L4                  ; format code FT_L4 (=2)
```

`FT_L4` = 2, `FT_L2` = 17 — две разные ячейки в `BITMAP_LAYOUT.format` (см. FT81x PG §4.7.7, Table 7). Stride для 4bpp = `(W+1)/2`.

DL pipeline — без изменений

```
;--- Pass 1: маска → dst.A через ONE/ZERO blend ---
FT_CMD_BUF (ZL_DL_COLOR_MASK | ZL_COLOR_MASK_A)
FT_CMD_BUF (ZL_DL_BLEND_FUNC | (ZL_BLEND_ONE << 3) | ZL_BLEND_ZERO)
FT_CMD_BUF (ZL_DL_COLOR_A | 255)
FT_Vertex2ii 0, 0, ZL_BG_L2_HANDLE, 0 ; теперь L4 mask

;--- Pass 2/3: c1/c0 с DST_ALPHA blend — те же команды ---
```

L4 декодируется FT812 в **линейный** 8-битный alpha: `raw_value × 17` (значения 0, 17, 34, ..., 255). В отличие от L2 (`{0, 255, 85, 170}`), L4 без перестановок — selector `k` даёт alpha $\approx k/15 * 255$. Конвертер автоматически использует правильное соответствие.

Финальный blend `dst.RGB = c0*(1-A) + c1*A` алгебраически одинаков — просто `A` теперь имеет 16 значений вместо 4.

Подводный камень: CPU энкодер на Windows нежизнеспособен

Для 640×480 = 19 200 блоков 4×4. Локальный энкодер (без GPU):

Режим	Результат
<code>-j 0</code> (auto = 6 cores)	BrokenProcessPool (OOM при effort 8 / L4)
<code>-j 1</code> (single-process)	~3 мин до 2% при effort 4 → ~2.5 часа total
<code>-j 2</code> effort 6	~2 мин до 0%, не дождалось

Multiprocessing у `concurrent.futures.ProcessPoolExecutor` на Windows **не shared memory**: каждый воркер получает копию `blocks` через pickle. Для 230 КБ blocks × 6 воркеров = 1.4 МБ × Python overhead ~50× = ~70 МБ накапливается; через несколько итераций OOM на 4 ГБ VM.

Single-process работает стабильно, но 19 200 блоков × ~0.5 сек/блок (effort 4 с perceptual weights) = 160 мин. Эта длительность была подтверждена эмпирически на CPU `2 × Xeon Gold 6132` под Hyper-V.

Решение: запускать энкодер на хост-машине с GPU через pyopencl.

```
python ft812_dxt_convert.py level_src_01.png -o out -f l4 -t raw -x -p -e 8
```

На AMD `gfx1032` весь pipeline (initial pair generation + hybrid refine + write) проходит **менее чем за 30 секунд** на effort 8. Готовые файлы (`out/level_src_01_l4.raw` 230 400 байт) копируются в проект, режутся на страницы, собираются.

Сплит в spgbld pages

```
# split_l4.py
PAGE = 16384
data = open('level_src_01_l4.raw', 'rb').read()
assert len(data) == 230400
n_pages = (len(data) + PAGE - 1) // PAGE    # = 15
for i in range(n_pages):
    chunk = data[i*PAGE:(i+1)*PAGE]
    if len(chunk) < PAGE:
        chunk += b'\x00' * (PAGE - len(chunk))    # padding zeros
    open(f'bg_l4_p{i:02d}.bin', 'wb').write(chunk)
# wrote 15 файлов, последний с 1024 реальных байт + 15360 нулей
```

`spgbld_vdac2.ini` :

```
Block = #0000, #07, bg_l4_p00.bin
Block = #0000, #08, bg_l4_p01.bin
...
Block = #0000, #15, bg_l4_p14.bin
Block = #0000, #16, killzone_p00.bin    ; следом, без overlap
```

Итоговый бюджет

Формат	Байт	vs Raw	Качество
Raw RGB565 640×480	614 400	1.00×	reference
400×300 RGB565 + scale 1.6 NEAREST	240 000	0.39×	ступенька 1.6×
DXT1_L2 (Глава 18)	153 600	0.25×	блочность 4×4
DXT1_L4 (эта глава)	230 400	0.38×	фоторовно
ARGB4 native 640×480	614 400	1.00×	reference

L4 даёт **2/3 объёма** native RGB565 при визуально неотличимом качестве — ровно та точка цена/качество, которая нужна для multi-level Zuma: 22 × 230 КБ = 5 МБ vs 13.5 МБ raw. Помещается в обычный TR-DOS + spgbld.

Когда выбирать L2 vs L4

- L2**: tile-фон, splash-screen с большими flat-зонами, ограниченный RAM_G. Если 75% экономии важнее минимальной блочности — берём L2.
- L4**: фотореалистичные уровни, фоны с плавными градиентами (наш случай), splash-screen с тонкой детализацией. +50% к L2, но качество скачком вверх.

Источники

- `ft812_dxt_convert.py` — light версия (1197 строк) после автора; hybrid GPU/CPU pipeline через ruopencil + numpy.

- `reference_zuma_vdac2_baseline_2026-05-12_bg_dxt_l4.md` — опорный baseline после интеграции.
- FT81x PG §4.7.7 — таблица `BITMAP_LAYOUT.format` (FT_L1/L2/L4/L8 codes).

Глава 20. Render-loop оптимизации и DL-emit ловушки (2026-05-17)

Главы 15-19 закрыли визуальную часть Zuma. Эта глава — три приёма, которые выжали из FT812 ещё несколько процентов и закрыли тонкий баг рендера. Появились в процессе финального полировок kill-zone (плавное поглощение шаров) и frog-композиции.

20.1 Bucket-grouped tangent rotation: 32 cmd_rotate → 16, а потом обратно

VDC выдаёт каждому шару в цепи `tangent` 0..255 — направление трека в точке. HD-источник вращает каждый шар своим `cmd_rotate(angle)`, но на FT812 это N call'ов `cmd_loadidentity → cmd_translate → cmd_rotate → cmd_translate → cmd_setmatrix` на каждый шар. При длине цепи 85 шаров это ~30% бюджета DL.

Bucket-grouping — группировка шаров по углу:

1. Pre-pass: для каждого шара вычисляем `bucket = (tangent + N/2) >> log2(N)` и кешируем (bucket, cell, Vx, Vy) в RAM.
2. Outer loop по N бакетам: emit matrix для `bucket * (256/N)`, inner scan — все шары с этим bucket'ом → Cell + Vertex2f.

При N=32: шаг 11.25° (256/32 = 8 BRAD = 11.25°). Шар получит visually-acceptable rotation, плюс цены matrix-emit'а только 32 раза за кадр.

```
; 32-bucket scheme: bucket = (tangent+4) >> 3
LD  A, (VDC_LastTangent)
ADD A, 4                                ; round-nearest
RRCA : RRCA : RRCA                      ; >> 3
AND 31                                  ; mod 32
LD  (cache_bucket), A
; ...позже, в outer loop:
LD  A, (current_bucket)
ADD A, A : ADD A, A : ADD A, A          ; bucket * 8
CALL ZL_EmitRotate                     ; A = BRAD 0..255
```

Lesson: 16 vs 32. Изначально 32 бакета считались избыточными — попробовали 16 (шаг 22.5°). На статичных шарах выглядело норм, но на быстродвигающихся по крутой кривой (вход в killzone, головной шар) проявился **визуальный jitter** — глаз ловит ступеньки. Откатили обратно в 32. Урок: не оптимизируй "на глаз" в статике; смотри на самые быстрые моменты gameplay.

20.2 Per-sprite alpha fade через COLOR_A — плавное поглощение

FT812 имеет команду `COLOR_A(alpha)` — умножает alpha-канал последующего bitmap'а на 0..255. Это позволяет делать **dissolve-эффект на спрайте без изменения текстуры**.

В нашем случае: head-шар цепи во время Game Over absorb должен плавно исчезать в kill-zone, а не пропадать дискретно. Алгоритм:

```

; Каждый тик absorb (state=1):
LD  A, (VDC_HSub)           ; HSub 0..31 in cell
ADD A, A : ADD A, A : ADD A, A ; * 8 (max 31*8 = 248)
CPL                               ; alpha = 255 - HSub*8
LD  (VDC_HeadAbsorbAlpha), A ; смыкается с 255 до 7 за цикл

; В .BInner bucket-loop, перед Vertex2f head-шара:
LD  A, (VDC_HeadAbsorbAlpha)
LD  E, A
CALL FT.Coprocessor.ColorA    ; emit COLOR_A(alpha)
LD  C, (IX+2) : LD B, (IX+3)   ; перезагрузить BC (Cell/ColorA уничтожили)
LD  E, (IX+4) : LD D, (IX+5)
CALL FT.Coprocessor.Vertex2f
LD  E, 255
CALL FT.Coprocessor.ColorA    ; восстановить для остальных шаров

```

Ловушка: COLOR_A — **persistent state DL**. Если не восстановить до 255, все последующие спрайты в этом кадре будут полупрозрачные.

Identify the target sprite: head-шар = первая запись в кеше bucket-prepass по адресу `ZL_BALL_CACHE_ADDR`. В .BInner проверяем `IX == ZL_BALL_CACHE_ADDR` (PUSH IX / POP HL / CP HIGH / CP LOW) — это slot[o]. COLOR_A применяется только к этому одному `Vertex2f`.

20.3 Cell/ColorA корраптят BC/DE — координаты грузить ПОСЛЕ, а не ДО

Все одноаргументные DL-команды TSLib (`Cell`, `ColorA`, `Tag`, `LineWidth` ...) эмитятся через `Command_BCDE` — формируют 4 байта опкода в BC/DE и пишут в буфер. **После такого CALL'a BC и DE мусор.**

Из этого следует жёсткое правило для пары Cell+Vertex2f:

```

; WRONG — баг, который у нас прятался месяц в DrawKillzoneDual:
LD  BC, x_scaled           ; BC = X
LD  DE, y_scaled           ; DE = Y
XOR  A
CALL FT.Coprocessor.Cell    ; BC, DE corrupted!
CALL FT.Coprocessor.Vertex2f ; uses corrupted BC, DE → sprite в ?,?

; RIGHT — Cell первым, координаты после:
XOR  A
CALL FT.Coprocessor.Cell
LD  BC, x_scaled
LD  DE, y_scaled
CALL FT.Coprocessor.Vertex2f

```

Bug-symptom при wrong ordering: спрайт рисуется в верхнем-левом углу или вообще не виден — Cell оставляет в BC значение `0x0600` (опкод Cell), Vertex2f интерпретирует это как $X*16 = 1536$, что выходит за разумный экраный диапазон, либо clip.

Эвристика: если sprite появляется не там где ожидаешь, или мигает, или "то ли есть, то ли нет" — **первое что проверить:** между LD BC,coords и Vertex2f нет ли промежуточного CALL'a к Cell/ColorA/Tag/etc. Если есть — переставить порядок.

20.4 Скип лишнего DL: bg-baked = overlay не нужен

Иногда самый быстрый рендер — **не рисовать вообще**. Kill-zone "закрытый череп" уже запечён в bg-арте (golden 8-pointed sun); рисовать overlay поверх в idle-state — двойная работа.

```
DrawKillzoneDual:
    LD    A, (VDC_KzFrame)
    CP    2
    RET    C                ; KzFrame=0/1 (idle / final GO) → bg сам показывает
    ; ...emit Cell + Vertex2f только когда KzFrame >= 2 (анимация)
```

Это экономит **~10 байт DL × 60 FPS = 600 байт/сек** трафика SPI, который освобождает Z80 cycles для chain physics + input + sound. Микротимизация, но накладывается на каждый "статичный" sprite в render-loop'e.

20.5 Continuous-motion absorb через HSub-advance (mirror of fast-spawn)

Last optimization-pattern: **используй существующий механизм движения, не пиши свой**. Игра уже умеет двигать цепь плавно — в fast-spawn phase chain двигается HSub++ × **VDC_FAST_ADVANCE** = 12 раз за тик. Это даёт плавное скольжение шаров по треку.

Для Game Over absorb разумно использовать **тот же механизм с другими параметрами**:

```
VDC_UpdateAbsorb:
    LD    B, VDC_ABSORB_ADVANCE ; e.g., 8 (32/8 = 4 ticks/cell)
.aa_loop:
    PUSH BC
    CALL .ua_move_once          ; HSub++; on wrap → array shift, HSA capped
    POP   BC
    DJNZ .aa_loop
    ; alpha рассчитывается из HSub → синхрон с motion
```

.ua_move_once :

```
LD    A, (VDC_HSub)
INC    A
CP     VDC_CELL_SIZE
JR     C, .save                ; HSub < CS → просто save
XOR    A                        ; wrap: HSub=0
LD     (VDC_HSub), A
; remove slot[0] (array shift), HSA capped → новый head в том же clamped
; последнем track sample → lpx continuity jump (invisible)
```

Эффект: tail-шары плавно скользят (sub-pixel HSub), head clamped на последнем сэмпле трека, alpha fade ↔ HSub progress. При wrap — array shift И сброс alpha в 255. **Visual continuity = 1 px разрыв** вместо discrete cell-jump.

Аналогичный паттерн можно применить к: уменьшению цепи после match-3 cascade, выбросу bonus-шаров, любым "цепь сжимается/растягивается" анимациям.

Источники

- **releases/baseline_2026-05-17_killzone_smooth_absorb/** — production-ready baseline после применения всех пяти приёмов.
- **Source/ASM/MainLoop.asm** : **.BInner** — bucket loop с per-head COLOR_A inject.

- `Source/ASM/main.asm`: `DrawKillzoneDual`, `VDC_UpdateAbsorb` — Cell-order fix + skip-in-idle + HSub-based absorb.
- FT81x PG §4.5 — `COLOR_A` opcode + persistent DL state.

Глава 21. Per-ball matrix с per-slot hysteresis и grouped emit (2026-05-18)

21.1 Постановка задачи

Шары цепи Zuma вращаются по тангенсу трека: на изгибе спрайт повернут так, чтобы рисунок (рельеф/блик) шёл по направлению движения, а не «лежал на боку». На каждый шар нужна BITMAP_TRANSFORM с углом = `tangent_at_track[i]`.

В лоб через FT812 это:

```
; per ball: 5 coproc-commands → 6 BITMAP_TRANSFORM_X DL entries
CALL ZL_EmitLoadId           ; cmd_loadidentity
LD HL, ZL_BALL_HALF
LD DE, ZL_BALL_HALF
CALL ZL_EmitTranslate        ; cmd_translate(+16, +16)
LD A, (cache+0)
CALL ZL_EmitRotate           ; cmd_rotate(tangent_byte)
LD HL, -ZL_BALL_HALF
LD DE, -ZL_BALL_HALF
CALL ZL_EmitTranslate        ; cmd_translate(-16, -16)
CALL ZL_EmitSetMatrix        ; cmd_setmatrix
```

Translate(+16) → Rotate(θ) → Translate(-16) — стандартная связка чтобы повернуть sprite вокруг центра bitmap (16,16) для атласа 32×32, а не вокруг угла (0,0).

Стоимость на цепь 35 шаров: **175 coproc-команд + 210 DL-записей BITMAP_TRANSFORM**.

FT812 coproc'у на это не хватает vblank-окна даже на 74Hz → **тиринг на реале**.

21.2 Альтернатива #1: бакеты — почему не подошло

Классический способ дешево покрыть N шаров: разбить tangent диапазон 0..255 BRAD на K корзин (buckets), назначить каждому шару ближайшую корзину, и в outer-loop эмитить матрицу 1 раз на корзину, а внутри обходить все шары своей корзины.

```
матрицы за кадр = K (фиксированно)
DL записи       = K × 6 transform + N × (cell + vertex)
```

K=32 → 11.25° на bucket, ~6× быстрее чем per-ball. Так и было сделано до 2026-05-18.

Проблема: «глобальный flip». Если raw tangent шара трамплинит между двумя бакетами кадр-к-кадру (например, из-за округления track-данных), его поворот скачет на 11.25°. И — что хуже — поскольку соседние шары находятся в **одной с ним** корзине (общая матрица), они визуальнo мигают **сегментом цепи целиком**. Глаз ловит «волну» на изгибах.

Это была реальная жалоба пользователя за всю прошедшую неделю работы.

21.3 Альтернатива #2: чистый per-ball — почему сломалось на реале

Прямой переход к per-ball matrix (для каждого шара свой `cmd_setmatrix`) убирает эффект «сегмент мигает» начисто — каждый шар вращается независимо. Visual quality максимальный.

Но соргос-нагрузка взлетела в ~6 раз. На баре эмуляторе (Unreal x64) кадр строился, на реальном FT812 при 74Hz и DL ≥ 300 записей **vblank-окна не хватало**: коприйцессор не успевал обработать команды до следующего DLSWAP — экран рвало.

Симптом: верхняя половина — frame N, нижняя — frame N-1, с горизонтальной чертой разрыва. Появляется в самых нагруженных моментах (длинная цепь + жаба + bullet).

21.4 Гибрид: per-slot hysteresis + run-length grouped emit

Идея: **хранить tangent per-ball независимо** (это уже даёт per-slot stability — flicker нет), но **эмитить матрицу только когда у соседних шаров в цепи tangent действительно поменялся**.

На спирали Zuma соседние шары цепи находятся на одной дуге трека, поэтому их tangent'ы очень близки. С разумной квантизацией (16 BRAD, на длинной цепи 32 — адаптивно, см. §21.4.2) адъяцентные шары часто попадают в **одинаковую дольку** — для них достаточно одной матрицы.

21.4.1 Per-slot byte-level hysteresis

Pre-pass для каждого шара хранит **свой** «стабильный» tangent в page-5 RAM (`#4100 + slot_idx`), обновляется только когда raw отличается на ≥ THR=8 BRAD:

```
; D = raw tangent (preserved). HL = state addr чеpez H = STATE_HI, L = slot.
LD  A, (VDC_LastTangent)
LD  D, A
LD  A, C                      ; slot index
LD  H, ZL_BALL_TANGENT_STATE_ADDR >> 8 ; #41 (low byte STATE_ADDR = 0 заведомо)
LD  L, A
LD  A, (HL)                   ; prev stable
LD  E, A
LD  A, D                      ; raw
SUB  E                        ; (raw - prev) mod 256
JP  P, .stab_pos              ; signed sign-bit check
NEG
.stab_pos: CP  ZL_BALL_TANGENT_HYSTERESIS_THR ; = 8
JR  NC, .stab_update          ; |delta| >= THR → update
LD  A, E                      ; else keep prev
JR  .stab_done
.stab_update: LD  A, D
LD  (HL), A
.stab_done:                   ; A = stable tangent (raw if updated, prev else)
```

Почему именно 8 BRAD threshold: должен быть ≥ ширине квантизационной корзины (8 BRAD), иначе raw, осциллирующий на границе ±4, заставит stable скакать между двумя бакетами. С 8: stable меняется только если raw уехал заметно в новую область → stable settles в одной корзине.

Почему ёлки H=STATE_ADDR>>8, L=slot (а не `LD HL,...+LD DE,slot+ADD HL,DE`): выбрали ZL_BALL_TANGENT_STATE_ADDR= `#4100` с low-byte=0 специально, чтобы 8-bit slot index ставился прямо в L без сложения. Сохраняет регистр D (с raw tangent) от затирания через `LD DE, addr`.

21.4.2 Адаптивная квантизация: грубее по мере роста цепи

В per-ball loop **квантуем** stable tangent к корзине и сравниваем с tangent'ом, для которого мы УЖЕ эмитили матрицу. Совпал — пропускаем эмит матрицы.

Адаптивная грубость по длине цепи (ключевой приём — `MainLoop.asm:.PerBallLoop`): ширина корзины зависит от `ZL_BallCount` (= длина цепи). На короткой цепи запас DL-бюджета большой → можно квантовать мелко (плавнее поворот); на длинной цепи бюджет на исходе → квантуем грубее, чтобы число эмитов матриц не росло линейно с числом шаров:

- `BallCount < 70` → `AND #F0` = **16 корзин** (шаг 16 BRAD = 22.5°);
- `BallCount ≥ 70` → `AND #E0` = **8 корзин** (шаг 32 BRAD = 45°) — грубее.

```
.ChainDraw:    LD    A, #01                      ; sentinel (не кратен 16/32)
               LD    (ZL_TmpLastTangent), A
               LD    A, (ZL_BallCount)
               LD    B, A                      ; loop count
               LD    IX, ZL_BALL_CACHE_ADDR
.PerBallLoop:  LD    A, (IX+1)                  ; cell (+1) = 0xFF marks gap
               CP    #FF
               JP    Z, .PBSkip                 ; (JR out of range - body grew)
               PUSH  BC
               LD    A, (IX+0)                  ; stable tangent
               LD    D, A
               LD    A, (ZL_BallCount)
               CP    70                        ; адаптивный порог
               LD    A, D
               JR    C, .PBQuant16
               AND    #E0                      ; длинная цепь (≥70): 8 корзин - грубее
               JR    .PBQuantDone
.PBQuant16:    AND    #F0                      ; обычно: 16 корзин
.PBQuantDone:  LD    HL, ZL_TmpLastTangent
               CP    (HL)
               JR    Z, .PBNoMatrix             ; та же корзина → переиспользуем
матрицу
               LD    (HL), A                   ; новая корзина → save
               ; матрица из ПРЕДРАСЧЁТНОГО LUT (минус соргос, см. §27.6)
               LD    A, (ZL_TmpLastTangent)
               CALL  ZL_EmitBallMatrixFromBRAD
.PBNoMatrix:   ; ... handle, cell, vertex2f для текущего шара (без матрицы) ...
```

Два рычага вместе: **(1) предрасчёт** — матрица не строится соргос-командами на лету, а копируется готовой из `ZL_ChainMatrixLUT` (32×24 байта, генератор `make_chain_matrix_lut.py`, см. §27.6); **(2) адаптивная группировка** — соседи по дуге попадают в одну корзину, эмит делается раз на корзину, а ширина корзины растёт с длиной цепи. Итог — число матриц на кадр почти не зависит от длины цепи.

Sentinel `#01` гарантирует что первый шар всегда триггерит эмит матрицы: реальные quantized tangent'ы кратны 16 (или 32), значению 1 никогда не равны.

21.4.3 Что в итоге

На спирали с 35 шарами цепи статистически на цепь приходится ~8–15 уникальных quantized buckets, и balls внутри bucket'a лежат подряд (соседи по track) → matrix emit срабатывает ~8–15 раз вместо 35. **3–4× падение соргос-нагрузки**. Адаптивная грубость (§21.4.2) удерживает это число и на длинных цепях: при `BallCount ≥ 70` корзины вдвое шире (8 вместо 16), так что эмитов не больше.

Доводка во30 (§27.6): сам эмит матрицы позже перевели на предрасчётный LUT (`ZL_EmitBallMatrixFromBRAD`) — матрица копируется готовой, coproc-команды на построение матрицы больше не тратятся вообще (строка `coproc-cmd/frame` ниже относится к реализации 2026-05-18 до LUT).

Метрики (snapshot 2026-05-18, до LUT):

	Bucketed (старое)	Per-ball naive	Per-ball + grouped
matrix/frame	32	35	8-15
coproc-cmd/frame	160	175	40-75
DL entries (chain)	294	315	~150
flip-flicker	YES	NO	NO
vblank ok @ 74Hz	YES	NO (tear)	YES

21.5 Ловушки реализации

Регистр-сейв (B-clobber)

Helpers `FT_BitmapLayout`, `FT_BitmapSize` — макросы, разворачивающиеся в инлайн через `FT_CMD_BUF`, который **клобает BCDE**. Поэтому паттерн «сохрани цвет в B → emit setup macros → возьми обратно из B» молча даёт мусор:

```
LD A, (Bullet_Color)
LD B, A                      ; "save"
... CALL ZL_EmitBallHandle ...
FT_BitmapLayout ...          ; ← кладёт B = 0x07 (opcode)
FT_BitmapSize ...            ; ← кладёт B = 0x08
LD A, B                      ; ← A не цвет! → cell wrong
AND 3
CALL Cell                    ; рисует случайный цвет
```

Симптом был: жаба стреляет одним цветом, в цепь вставляется другой. Потому что **в памяти** `Bullet_Color` корректный (`VDC_InsertAt(Bullet_Color)`), а **на экране** во время полёта пуля рисовалась мусорным cell.

Фикс: перечитать color из памяти после макросов, не из регистра:

```
LD A, (Bullet_Color)
CP 4
LD A, 0
JR C, .h0
LD A, 9
.h0: CALL ZL_EmitBallHandle
FT_BitmapLayout ...
FT_BitmapSize ...
LD A, (Bullet_Color)          ; re-read - macros clobbered registers
AND 3
ADD A,A : ... *32
CALL Cell
```

Аналогично для chain draw, но там есть `IX → (IX+1)` cache pointer — Cell читаем оттуда, IX через хелперы сохраняется.

Sentinel выбор

`ZL_TmpLastTangent` инициализируется `#01`, а не `#FF` — потому что после `AND #F8` реальные quantized tangent'ы могут быть `0, 8, 16, ..., 248`. Значение `#FF` после AND F8 даёт `#F8` (валидный bucket), и если у первого шара quantized = 248 = `#F8`, он бы совпал с sentinel и **пропустил matrix emit** — а матрицы ещё нет (FT812 uninitialized state) → шар нарисуетс с identity matrix. Sentinel `#01` гарантированно не совпадает ни с одним quantized = multiple-of-8.

JR vs JP — out of range

После добавления matrix-skip логики body цикла вырос. `JR Z, .PBSkip` (2 байта, ±127 диапазон) перестал доставать. Заменял на `JP Z, .PBSkip` (+1 байт но absolute address). Уроки прошлых сессий: при росте кода всегда чекать JR-distances через `--lst`.

21.6 RNG bias как побочный bug (2026-05-18)

Параллельно с per-ball рефакторингом расширил `VDC_NUM_COLORS` с 4 до 6 (атлас уже содержал colors 4-5: white + yellow). Жёлтый не появлялся в цепи COBCEM.

LFSR Galois с polynomial `#B400`:

```
LD HL, (VDC_LfsrSeed)
LD A, L
AND 1 ; bit out
SRL H : RR L ; shift HL right
JR Z, .no_xor
LD A, H : XOR #B4 : LD H, A ; feed back via poly
.no_xor:
LD (VDC_LfsrSeed), HL
LD A, L
XOR H ; 8-bit "random"
AND 7 ; → 0..7
CP NUM_COLORS ; reject если >= NUM
JR NC, retry
RET
```

Скрытая корреляция битов: для конкретного poly `#B400`, после XOR $L \oplus H$ и маски `AND 7` результат покрывает почти исключительно `{0,1,3,4}`, а значения `{2,5,6,7}` встречаются ~1 раз на 1000. Rejection (`CP NUM_COLORS`) отсекает 6,7 — а 2 и 5 он не лечит. Цвета 2 (фиолетовый) и 5 (жёлтый) выпадают почти никогда.

Замер на baseline: 1000 вызовов `VDC_RandomColor` дали `[306, 231, 2, 230, 230, 1]`.

Фикс — mul-then-shift вместо bit-masking:

```
LD A, L
XOR H ; A = 8-bit raw rand
LD L, A
LD H, 0 ; HL = rand byte
LD A, VDC_NUM_COLORS ; A = 6
CALL ZL_Mul16x8 ; HL = rand * NUM (max 6*255=1530, <16-bit)
LD A, H ; A = (rand * NUM) >> 8 = 0..NUM-1
RET
```

Принцип: любое значение rand 0..255 распределяется по NUM_COLORS bucket'ов пропорционально размеру bucket'a. Bias ≤ 1.4% даже при равномерном rand, и НЕ требует, чтобы определённые биты были некоррелированы.

После замера: `[166, 111, 110, 57, 222, 334]` — все 6 цветов появляются. Дистрибуция всё ещё неравномерна из-за самой неравномерности LFSR-байта, но **кolor 5 теперь в игре**.

21.7 Применимость в других случаях

Паттерн «per-slot hysteresis + run-length grouped emit» обобщается:

1. **Условие применимости:** есть много объектов, которым нужно индивидуальное состояние (color, scale, rotation), но в смежных объектах состояние часто одинаково.
2. **Шаг 1:** state per-object с byte-level hysteresis (storage = N байт RAM, threshold $\geq$ quantization step).
3. **Шаг 2:** в draw-loop сравнивай с last-emitted state, пропускай emit при совпадении.

Кандидаты на это в Zuma VDAC2: - Spin frame (cell number): соседние шары на одной фазе rolling — сейчас они **уже** имеют разные cell индексы из-за `t × K`, group skip = no-op. - Bitmap handle 0 vs 9 (для colors 4-5 split): группировать по color group — уже работает (handle меняется только при cell ≥ 128).

Источники

- `Source/ASM/MainLoop.asm` : `.ChainDraw` / `.PerBallLoop` — финальная реализация.
- `Source/ASM/VDC.asm` : `VDC_RandomColor` — mul-then-shift fix.
- `releases/baseline_2026-05-18_pre_per_ball_6_colors/` — pre-change snapshot для отката.
- FT81x PG §4.7 — BITMAP_TRANSFORM_A..F state, persistent across vertex2f.

Глава 22. Расщепление Core на main0 + main1 (slot 1 + slot 3) и невидимая ловушка CMD_ADDRESS_PTR (2026-05-18)

22.1 Проблема: лимит "считать байты Core" 9216

К v020 в Core (slot 1 page 5, ORG #5C00) набралось 9210 байт из 9216 — 6 байт запаса. Каждая новая фишка режется на размере: match-3 explode pass добавили с трудом, шрифт "GAME OVER" вкручен между функциями, ring log переехал в slot 0 ещё раньше. Дальше расти некуда — а впереди 22 уровня + стартовый экран + level select + сжатие данных.

22.2 Архитектура split

В соседнем TS-Conf проекте `~/Desktop/Zuma Deluxe` уже работает паттерн **main0 / main1**:

Сегмент	Что	Где	Зачем
main0	резидент: bootstrap, IM2, paging-helpers, общие helpers	slot 1 page 5 (ORG #5C00, до #7FFF = 9.2K)	всегда в памяти
main1	сценовый код (Init_Video + VDC + Frog + Bullet + MainLoop)	slot 3 page #04 (ORG #C000, до #FFFF = 16K)	можно иметь несколько разных main1 страниц под разные сцены и свопать через SetPage3

В VDAC2 main0 на 2026-05-18 ужался до **415 байт** (Start + Initialize + Init_Core + Init_Int + INT_Handler), main1_play занял **8795 байт** в 16K window'e. Лимит "считать байты Core" снят — теперь есть ~7.5K запаса в main1 и ~8.8K в main0.

```

; main0 (slot 1 page 5)
ORG EntryPoint ; #5C00
module Core
Start: LD SP, StackTop
CALL Initialize
JP MainLoop ; target #C000+ in slot 3 - works
; once Init_Core sets slot 3 = page 4

Initialize: CALL Init_Core
...

Init_Core: FMapAddrInit
SetPage1 5 ; slot 1 -> main0 page
SetPage2 6 ; slot 2 -> TrackData
SetPage3 #04 ; slot 3 -> main1_play page
RET

Init_Int: ... ; IM2 setup + first INT wait
INT_Handler: EI : RET
Main0_End: ; ОБЯЗАТЕЛЬНО после всего main0 кода

; main1_play (slot 3 page #04)
SLOT 3 : PAGE #04 : ORG #C000
Main1_Start:
include "Init_Video.asm"
include "VDC.asm"
include "Frog.asm"
include "Bullet.asm"
include "MainLoop.asm"

Main1_End:
endmodule

SAVEBIN "Core.bin", Core.Start, Core.Main0_End - Core.Start
SAVEBIN "main1_play.bin", Core.Main1_Start, Core.Main1_End -
Core.Main1_Start

```

В `spgbld_vdac2.ini` добавляется блок:

```
Block = #C000, #04, main1_play.bin
```

22.3 Cross-slot CALL работает без thunks

Z80 видит slot 1 (#4000..#7FFF) и slot 3 (#C000..#FFFF) **одновременно** — они разные адресные диапазоны, оба мапятся через TS-Conf mapping регистры. Когда `CALL Init_Video` (target #C000+) делается из main0 (#5C00+), Z80 толкает return-addr в стек (стек в slot 1, #40F2) и прыгает в slot 3. Main1 код выполняется, делает RET — return-addr из стека → возвращаемся в slot 1. **Никаких thunks не нужно**, пока сцена одна.

Thunks потребуются позже, когда добавим Title/LevelSelect: тогда main0 будет свапать slot 3 на разные main1-страницы и JP в общую entry-точку сцены.

22.4 Ловушка 1: Main0_End в неправильном месте

В первой попытке split метка `Main0_End:` стояла сразу после `RET` функции `Init_Core`. Но `Init_Int:` и `INT_Handler:` определены ниже в файле — **между** `Init_Core` и `SLOT 3` директивой. `SAVEBIN "Core.bin", ..., Main0_End - Start` покрыл только #5C00..#5D89 = 393 байта и **обрезал** `Init_Int` + `INT_Handler`.

На железе: 1. SPG-loader загружает Core.bin (393 байта) в page 5 → #5C00..#5D89 2. Остальная page 5 (#5D89..#7FFF) — нули (initialized) 3. Z80 на `CALL Init_Int` прыгает в #5D89 → читает 0 = NOP 4. NOPит через всю slot 1 → входит в slot 2 (#8000+) = TrackData 5. На байте 0xFF в TrackData выполняется `RST 38` → прыжок в #0038 (RST vector) 6. NOPит из #0038 до #1000 (4040 NOP-ов) 7. На #1000 в TSLib живёт функция `SetPage0:` (`LD (FMADDR_REGS+0x10), A : RET`) 8. A = 8 (последнее значение из upload loops) → slot 0 переключается на page 8 = bg_l4_p01 9. TSLib исчезает → следующая инструкция читается из bg-картинки = chaos → виснет

Урок: `Main0_End:` ВСЕГДА последняя строчка main0 секции. Любая код-строка ниже неё (но выше SLOT 3 директивы) выпадает из SAVEBIN.

22.5 Ловушка 2: TSLib CMD_ADDRESS_PTR = #C000 молча затирает main1

После исправления Main0_End игра запустилась — frog/bg/cursor видны. Но **цепочки шаров не появляются, фрог не стреляет, при попытке выстрела полностью виснет.**

В эмуляторе VDC_TrySpawn и Bullet_Spawn работают идеально на изолированных вызовах. Регресс физики PASS. То есть код ОК. Проблема — где-то между кадрами.

Дамп с реального железа показал: содержимое #C000 — это **FT812 display list команды** (CLEAR_COLOR_RGB, VERTEX_FORMAT, CLEAR), а не main1_play код.

Источник в TSLib: `Docs/TSLib/Include/FT/Coprocessor/Buffer.asm:5-7`:

```
ifndef CMD_ADDRESS_PTR
define CMD_ADDRESS_PTR #C000
endif
```

`FT_CMD_Start` макрос делает `LD HL, CMD_ADDRESS_PTR : LD (BufferPtr), HL`. Каждый `FT_CMD_BUF` пишет 4 байта на `BufferPtr++`. За кадр накапливается до `ZL_CMD_WARN_BYTES = #0E00` ≈ 3.5 KB DL-команд **поверх main1_play кода.**

До split этот буфер тоже жил на #C000, но slot 3 содержал бесполезный bg_l4_p01 (или что было default). Buffer перезаписывал данные, которые никто не читал. Безобидно.

После split slot 3 = main1_play → буфер переписывает VDC_Update, Bullet_*, ZL_DrawFrame и т.д. Первый кадр успевает отрендериться (потому что buffer ещё не достиг этих адресов), второй — VDC функций нет, цепь не спавнится, при выстреле Bullet функция уже мусор, Z80 уходит в random код, виснет.

Fix. Перед `include "FT/Coprocessor/Include.inc"` в main.asm:

```
; FT command buffer: TSLib дефолтит на #C000 (slot 3). После
; main0/main1 split main1_play код живёт в slot 3 → буфер
; перекрывает код. Перенесён в slot 1 free area после main0.
define CMD_ADDRESS_PTR #5E00
```

`#5E00` — slot 1 page 5 после main0 (415 байт = ends at #5D9F). До конца slot 1 (#7FFF) → 8.5 KB запаса, в избытке для 3.5 KB буфера.

22.6 Чек-лист split-проекта

Любой split, в котором код переезжает в slot 3 (#C000+), должен пройти:

1. `Main0_End:` **метка** — строго после всего main0 кода, перед `SLOT 3` директивой. `SAVEBIN "main0.bin", Start, Main0_End - Start` гарантирует что вся main0-логика попадает в bin.

2. **Override** `CMD_ADDRESS_PTR` — перед TSLib include-ами. Любое значение в writable RAM области, минимум 4 KB до границы window-a. Не #C000.
3. **Init_Core SetPage3 на main1-страницу** — например `SetPage3 #04` если main1_play лежит на page 4 в SPG.
4. `spgbld_vdac2.ini` **Block** — `Block = #C000, #04, main1_play.bin`.
5. **Cross-slot CALL без thunks** — JP MainLoop из main0 в #C000 работает после Init_Core. Внутри одной сцены никаких thunks не нужно.
6. **Paged simulator с FMADDR_REGS hook** — `zuma_full_z80_emulator.py` ловит memory writes в #0410..#0413 (mapping registers) и обновляет pages map. Без этого hook-а эмулятор не воспроизводит SetPage* в режиме MAPPING_REGISTERS. Также полезны: ring buffer 4096 PC + PC watchpoint на #1000 (SetPage0 функция) — за минуты находят misjumps.
7. **Дамп после F12 — RESET, не data.** В Unreal F12 = RESET. Содержимое после F12 не отражает состояние во время виса. Для диагностики hang-a — F12 не подходит, нужен paged simulator или hardware-marker.

22.7 Запас на будущее

После v021 split (2026-05-18):

- main0: 8801 байт свободно (415/9216)
- main1_play: 7589 байт свободно (8795/16384)
- На каждую новую сцену (Title, LevelSelect, разные уровни) можно выделить свою 16K страницу под main1_.bin без касания main0
- FT command buffer 4 KB в slot 1 (#5E00..#7FFF) — запас в 4.5 KB

Следующий шаг: подключение Dzx7Turbo из TS-Conf проекта для сжатия per-level данных (15 страниц bg L4 × 22 уровня = 330 страниц без сжатия, с ZX7 ~245 страниц).

Источники

- `Source/ASM/main.asm` — main0/main1 split + CMD_ADDRESS_PTR override.
- `Docs/TSLib/Include/FT/Coprocessor/Buffer.asm:5-7` — CMD_ADDRESS_PTR дефолт.
- `Source/OTHER/zuma_full_z80_emulator.py` — FMADDR_REGS hook + ring buffer + watchpoint.
- `~/Desktop/Zuma Deluxe/src/ASM/zuma_new_spg.asm` — main0/main1 паттерн в TS-Conf.
- `releases/v021-2026-05-18-main0-main1-split.spg` — рабочий baseline.

Глава 23. Экономия RAM_G шаров: путь к PALETTED4444 (v025)

23.1 Постановка задачи

Изначальный atlas шаров — 6 цветов × 32 spin-фазы × 32×32 px ARGB4 (2 байта/пиксель) = **384 KB**. Из 1 MB RAM_G на FT812 после bg, frog, killzone, destroy, text, cursor, sparkle остаётся ~530 KB. Добавление UI-фрейма (gameinterface) требует ещё ~80 KB, остаётся 65 KB — впритык.

Цель: ужать atlas минимум вдвое, **сохранив**: 6 цветов, 32 spin-фазы, native цвета из source PNG, per-pixel alpha (anti-aliased силуэт).

23.2 Опции форматов FT812

Формат	Bytes/px	6×32×32×32 atlas	Alpha
ARGB4	2	384 KB	прямой 4-bit per px
RGB565	2	384 KB	нет
L8	1	192 KB	= alpha mask (не intensity!)
L4	0.5	96 KB	= alpha mask
PALETTED8	1	192 KB	1024B RGBA8 палитра
PALETTED565	1	192 KB	нет
PALETTED4444	1	192 KB	512B ARGB4 палитра

L4/L8 на FT812 — alpha mask: output = `tint × pixel/255`. Это «силуэт + tint», тело шара теряется. Не годится для многоцветных балов с собственным shading.

23.3 Неудачные попытки (mid-may 2026)

Попытка 1: MONO ARGB4 + COLOR_RGB tint (64 KB). Один atlas silver-шара (R=G=B=L, A=alpha) + tint per ball. Maya-faces на всех цветах одинаковые (из silver-источника), нет белого specular highlight (silver L_max ≈ 230). Tint не работает для многоцветного shading.

Попытка 2: L8 + tint (32 KB). L8 на FT812 — alpha mask, не intensity. Шары полупрозрачные.

Попытка 3: PALETTED8 с BGRA byte order (192 KB + 1024B). FT812 читает палитру как RGBA, не BGRA. Все шары серые.

Попытка 4: PALETTED8 с RGBA byte order (192 KB + 1024B). На этом конкретном FT812 — по-прежнему серые. PALETTED8 трактуется как L8 (chip-revision quirk). Вывод после 4-х неудач — «PALETTED не работает». **Этот вывод был ложным.**

23.4 PALETTED4444: три условия

User-инсайт: PALETTED4444 (формат = 15) использует палитру **СТРОГО 512 байт** (256 × 2 ARGB4). Прошлые попытки попадали в одну из трёх ловушек:

- Размер палитры ≠ 512.** При 1024 (RGBA8) FT812 читает байты 512+ как PIXEL data за палитрой → corruption / hang. При меньшем размере → out-of-range index → hang.
- Формат записи ≠ ARGB4 LE.** Корректно: `python word = (a4 << 12) | (r4 << 8) | (g4 << 4) | b4 # 16-bit LE pal_bytes += word.to_bytes(2, "little")` 1024-байтная RGBA8 палитра ляжет как 256 пар бессмысленных ARGB4 → все цвета серые.
- Адрес не выровнен на 4 байта.** FT_PaletteSource берёт младшие 22 бита; FT812 требует 4-byte aligned. Невыровненный → junk.

23.5 Setup PALETTED4444 (baseline v025)

Palette generation (Python):

```

fake_q      =      Image.fromarray(opaque_rgb.reshape(-1,      1,      3),      "RGB")
.quantize(colors=255, method=Image.Quantize.MEDIANCUT)
pal = bytearray()
pal += b""      # idx 0 = transparent
for i in range(255):
    r, g, b = pal_rgb_flat[i*3:i*3+3]
    a4 = 0xF
    word = (a4 << 12) | ((r >> 4) << 8) | ((g >> 4) << 4) | (b >> 4)
    pal += word.to_bytes(2, "little")
assert len(pal) == 512      # СТРОГО

```

main.asm — upload:

```

BALLS_PALETTE_RAM EQU #080000      ; 16K-aligned (4-byte aligned тоже)

LD A, BALLS_PALETTE_PAGE
SetPage2_A
LD HL, #8000
LD BC, 512      ; ← не 1024!
LD A, (BALLS_PALETTE_RAM >> 16) & 0xFF
LD DE, BALLS_PALETTE_RAM & 0xFFFF
CALL FT.WriteMem

```

MainLoop.asm — render:

```

FT_PaletteSource BALLS_PALETTE_RAM      ; опкод 0x2A
LD A, 9 : CALL ZL_EmitBallHandle      ; dual handle (192 cells > 128)
FT_BitmapLayout FT_PALETTED4444, ZL_BALL_W, ZL_BALL_H
FT_BitmapSize FT_NEAREST, FT_BORDER, FT_BORDER, ZL_BALL_W, ZL_BALL_H
XOR A : CALL ZL_EmitBallHandle      ; handle 0
FT_BitmapLayout FT_PALETTED4444, ZL_BALL_W, ZL_BALL_H
FT_BitmapSize FT_NEAREST, FT_BORDER, FT_BORDER, ZL_BALL_W, ZL_BALL_H

```

23.6 Результаты v025 (HW-verified)

Аспект	v019 (original)	v025 (PALETTED4444)
Atlas	384 KB	192 KB + 512B палитра
Spin phases	32	32
Colors	6	6 (256 quantized)
HW status	works	works (юзер: «ничего не глючит»)

Экономия **192 KB**. 4 точки в коде (chain, bullet, frog hand, frog back) используют один FT_PaletteSource per frame.

23.7 Урок: «не работает» ≠ «формат не поддерживается»

Из 4-х неудач легко сделать вывод «PALETTED unsupported». Прежде:

1. Точно ли palette size совпадает со спекой формата? (565 = 512, 4444 = 512, 8 = 1024).
2. Точно ли byte order совпадает? FT81x: little-endian 16-bit для 565/4444, 32-bit для 8.
3. Выровнен ли адрес на 4 байта?
4. FT_PaletteSource эмитнут ДО Vertex2f в Begin BITMAPS?

Если 3 пункта проверены и **всё равно** не работает — попробовать другой PALETTED-вариант (4444 vs 8 — оказалось решением). На разных chip revisions FT812 поведение PALETTED8 / 4444 различается.

Источники

- `Source/OTHER/make_balls_atlas_paletted.py` — atlas + RGBA8 palette.
- `Graphics/Converted/balls_palette_argb4.bin` — 512 байт ARGB4 LE.
- `Source/ASM/{main.asm, MainLoop.asm, Bullet.asm, Frog.asm}` — 4 точки.
- `releases/v025-2026-05-19-paletted4444-fix.spg` + sources в `_baseline_v025-paletted4444_fix/`.

Глава 24. Почему отказались от псевдо-DXT фона (2026-05-19)

TL;DR

Псевдо-DXT (Глава 18–19) занимал ~80 КБ RAM_G и весил 0.5–0.75 байт/пикс — отличный компромисс по памяти. Но на реальном железе ZX-Evo вызывал то, что выглядело как «срыв строчной синхронизации»: дрожание строк, цветные полосы, мерцание. На Unreal x64 эмуляторе всё было идеально. Причина оказалась в **переполнении бюджета пиксельных клоков FT812 на строку** из-за трёх полноэкранных bitmap-проходов на каждый кадр. Решение: 400×300 PALETTED4444 + NEAREST upscale ×1.6 = **один** bitmap-проход. Канарейка v028-эпохи (2026-05-19) подтвердила на реале.

Что делал псевдо-DXT (краткое напоминание)

Из Глав 21–22: - Фон 640×480 раскладывался на три плоскости в формате DXT1-emulation: - `c0` — RGB565 цвет «низкий» (160×120 пикс, 38 КБ) - `c1` — RGB565 цвет «высокий» (160×120 пикс, 38 КБ) - `mask` — L2 или L4 (2 или 4 бита/пикс, 80×60 или 160×120 пикс) - Display List на каждый кадр: 1. Begin BITMAPS, source=c0, scale ×4, draw 640×480 full screen 2. Source=c1, BLEND_FUNC = DST_ALPHA, draw 640×480 3. Source=mask, sample as alpha mask, draw 640×480

Три полноэкранных прохода. Каждый проход = один bitmap fetch per pixel.

Симптом на реале — «срыв строчной»

Когда фон рисовался псевдо-DXT, на ZX-Evo: - Картинка дрожала по горизонтали (строки слегка прыгали) - Появлялись цветные полосы непредсказуемой ширины - При движении frog/шаров — артефакты усиливались - На Unreal x64 эмуляторе — **всё чисто**, никакого тиринга

Первая гипотеза была tearing — попробовали VM_640_480_74Hz (§4.2). Не помогло. Вторая — буфер DL переполняется. Замеры через REG_CLOCK профайлер показали что DL build не превышал vblank window. То есть Z80 ничего не зашкаливает.

Реальная причина — пиксельный клок-бюджет на строку

FT812 на 800 пикс/строка имеет ровно **800 пиксельных клоков** для всех операций рендера этой строки. При: - 1 bitmap-проходе с NEAREST + paletted4444 → ~1 такт/пикс, бюджет = 800 пикс, с большим запасом. - 1 bitmap-проходе с BILINEAR → ~2 такта/пикс, бюджет = 400 пикс ≈ половина строки. - **3 bitmap-прохода** (псевдо-DXT) → каждый берёт 1 такт/пикс минимум, итого 3 × 640 = 1920 тактов. **На 1120 тактов больше** чем строка содержит. FT812 не успевает закончить рендер строки до начала следующей → строка обрезается / смещается, что визуально и выглядит как срыв синхронизации.

Это **аппаратное ограничение per-line**, а не общий FPS-cap. Unreal x64 эмулирует DL семантически но не моделирует pixel-clock budget — поэтому там не было видно.

Решение — один полноэкранный проход

Перешли на формат `400x300 PALETTED4444` + аппаратный NEAREST upscale $\times 1.6$:

```
FT_BitmapHandle BG_HANDLE
FT_PaletteSource BG_PALETTE_RAMG
FT_BitmapSource BG_RAMG_ADDR
FT_BitmapLayout FT_PALETTED4444, 400, 300
FT_BitmapSize FT_NEAREST, FT_BORDER, FT_BORDER, 640, 480
CMD_SCALE 1.6 (16.16 fixed)
Vertex2f (0, 0)
```

Один bitmap fetch на пиксель. NEAREST = 1 такт/пикс. На 800-тактовую строку бюджет занят процентов на 50 — есть запас под другие операции (рамка, шары, frog).

На реале — чистая картинка, никакого дрожания.

Что потеряли по сравнению с псевдо-DXT

Метрика	Псевдо-DXT (L4)	400x300 PALETTED4444 NEAREST
RAM_G	~80 КБ	120 КБ + 512Б palette
Native разрешение	640x480 (после blend)	400x300 → upscale 640x480
Цветовая глубина	RGB565 (16 bpp эфф.)	256 цветов в палитре ARGB4
Bitmap-проходов на кадр	3	1
Pixel-clock budget на строку	переполнен на реале	ОК с запасом

- **Потеряли:** 640x480 native + 16-bit цвет. С NEAREST upscale картинка стала слегка «пиксельной» (~1.6× больше реальных пикселей). Цвета теперь ограничены 256-цветной палитрой (раньше было до ~50K при RGB565x2).
- **Получили:** рабочая стабильная картинка на реальном железе.
- **Стоимость по RAM_G:** +40 КБ (~50% больше) — терпимо, общая RAM_G FT812 = 1 МБ.

Почему не разделили на полосы

Можно было оставить псевдо-DXT, но рисовать не 640x480 за один кадр а полосами (скажем 8x60 px каждая), переключая bitmap source/scissor между ними. Это распределило бы pixel-clock budget — каждая полоса в своём DL «окне».

Не пошли потому что: - Требует scissor management + N draw calls = сложнее код - Не масштабируется для уровней — каждый фон уровня = свой trick - Бюджет всё равно >>1 такт/пикс если делать blend в одной полосе

Простой single-pass PALETTED4444 решает проблему **без архитектурной сложности**, ценой ~50% RAM.

Bigger picture: правило бюджета строки

Закон для FT812 на 800-тактовой строке:

$$\text{sum(такты/пикс на каждый bitmap-проход)} \times (\text{видимая ширина в пикс}) \leq 800$$

Для 640-pix экрана: - 1 NEAREST = 640 тактов → запас 160 - 1 BILINEAR = 1280 тактов → **уже переполнение** для full-screen sprite - 2 NEAREST = 1280 тактов → переполнение - 3 NEAREST (псевдо-DXT) = 1920 → грубое переполнение

Если нужно несколько проходов — они должны быть **не полноэкранными**, чтобы сумма ширин × такты ≤ 800. Маленькие sprites (frog, шары, курсор) проблемы не создают потому что их совокупная ширина по любой строке << 640.

Когда стоит вернуться к псевдо-DXT

Если на реале FT812 окажется что появилась дополнительная архитектура которая решает row-pixel-clock переполнение (например split rendering на 4 параллельных engine), или будет важна экономия 40 КБ RAM_G в ущерб качеству. В текущей VDAC2 архитектуре — нет.

Уточнение модели (ground-truth через эмулятор, см. главу 25)

Модель выше («800 тактов на строку, ~1 такт/пиксель на проход») — рабочее **первое приближение**: она верно предсказывает, что три полноэкранных прохода рвут картинку, а один — нет. Но позже, сняв через эмулятор EveApps **реальный RAM_DL** (глава 25) и сверившись с аппаратной моделью FT812 от TS-Labs, мы уточнили механизм:

- Движок FT812 **заново проходит весь Display List на КАЖДОЙ строке** раstra, тратя примерно **1 такт на DL-команду**. Этот «проход по командам» — а не заливка — часто и есть доминанта нагрузки.
- Заливка пикселей дешевле, чем казалось: **~16 пикс/такт** для не-палитровых форматов; палитра медленнее (lookup на пиксель); **BILINEAR** вдвое медленнее **NEAREST**.
- Реальный бюджет строки — это **REG_HCYCLE** (для нашего 640×480 ≈ **832 такта**), а не ровно 800.

Ground truth. Дамп RAM_DL тяжёлого кадра (frame 0650) показал **439 DL-команд**, из которых ~192 — матрицы разворота шаров (~44% бюджета). Итог ≈ **104–106%** от 832 → строка не успевает достроиться → tearing на верхней полосе. То есть tearing = **переполнение бюджета строки**, и виноват прежде всего объём DL, а не заливка как таковая.

Рычаги (без потери качества картинки): 1. Сократить **число DL-команд** — адаптивная группировка + предрасчётный LUT матриц шаров (§21.4) уже снижает их в разы. 2. Избегать **BILINEAR** / **PALETTED** на крупных спрайтах (§27.9 — frog). 3. Не дробить фон на полосы со scissor под полупрозрачной рамкой — фон виден сквозь неё, scissor исказил бы картинку.

⚠ Ограничение проекта (2026-05-26): матрицы разворота шаров по касательной **не убираются** — они заметно улучшают восприятие (решение пользователя). Поэтому дальнейшая tearing-оптимизация отложена: режим что угодно, кроме матриц шаров и читаемости фона.

Источники

- Memory: **reference_zuma_vdac2_ft812_line_budget_command_walk** — уточнённая модель: доминирует проход по командам DL (HCYCLE≈832, 16 пикс/такт), ground-truth через RAM_DL.
- Memory: **reference_zuma_vdac2_eve_emulator_dl_readback** — readback реального RAM_DL через EveApps (см. главу 25).
- Memory: **reference_zuma_vdac2_baseline_2026-05-19_bg400_killzone88_real_hw_ok** — финальный baseline с 400×300 PALETTED4444 фоном, верифицирован на реале.

- Чат.txt: `[2026-05-19 22:55] Codex -> BG 400x300 PALETTED4444 nearest upscale canary` — диагностика и переход.
- Глава 18 (DXT1 L2) + Глава 19 (L4 апгрейд) — описание того что отказались делать.

Глава 25. Bridgetek EveApps FT812 Emulator — настоящая эмуляция чипа (2026-05-19)

Зачем понадобился новый эмулятор

До сессии 2026-05-19 наша «эмуляция» Zuma выглядела так:

Слой	Инструмент	Что эмулирует
Z80 CPU	<code>zuma_full_z80_emulator.py</code> (kosarev/z80)	Только Z80 instructions + memory
VDC physics	Python <code>vdc_visual_emulator.py</code>	Chain логика 1:1 порт asm
Графика → screen	Unreal x64	Z80 + TS-Conf + FT812 (LIES — semantically only)

Проблема Unreal x64: он эмулирует FT812 **семантически**, выполняет DL команды и выводит картинку, но **не моделирует hardware constraints**: - pixel-clock budget per line (см. Главу 24) - BITMAP_HANDLE binding rules (см. Главу 26 ниже) - SPI bandwidth, DMA timing, BFLB swap latency

Поэтому на Unreal x64 всё работало, а на реальном ZX-Evo + VDAC2 — глюки.

Нужен был **настоящий эмулятор чипа FT812** для отладки таких багов БЕЗ необходимости загружать на реал каждый раз. Bridgetek предоставляет официальный — берём.

Что выбрали

Bridgetek EveApps MSVC Emulator — официальный эмулятор FT812 от производителя.

- GitHub: <https://github.com/Bridgetek/EveApps>
- Лицензия: MIT (open source)
- Точность: 100% (от FTDI/Bridgetek, той же команды что сделала чип)
- Windows + MSVC build

Альтернативы из `Docs/ft812_emulator_setup_guide.md`: - EVE Screen Editor — GUI, для прототипирования, не для авто-тестов - RudolphRiedel FT800-FT813 — кросс-платформа, требует USB-SPI bridge - CircuitPython_eve — через железо

Выбрали EveApps MSVC Emulator потому что: - Бесплатно + open source - Windows native (наша платформа разработки) - Поддержка ASTC/FT81x/BT81x — для будущих апгрейдов - Можно интегрировать через файлы (просто и надёжно)

Установка (Codex, 2026-05-19)

Шаг 1: Visual Studio Build Tools 2022 + MSVC v143

Без полного Visual Studio — достаточно «Build Tools for Visual Studio 2022» с компонентом «Desktop development with C++». Лёгкая установка ~3 GB.

Шаг 2: Клонирование EveApps

```
cd C:\Users\Администратор\Desktop
git clone https://github.com/Bridgetek/EveApps.git
```

Шаг 3: Адаптация SampleApp

Bridgetek даёт готовое `SampleApp\Bitmap` — простой sample который рисует bitmap на эмулированный экран. Скопировали как заготовку:

```
cd EveApps\SampleApp
xcopy /E /I Bitmap ZumaPlayback
```

Затем переименовали проект в Visual Studio в `ZumaPlayback_Emulator`, выбрали target = FT812 (через `EVE_GRAPHICS_TARGET` define), заменили `ZumaPlayback.c` на наш файл (см. ниже).

Шаг 4: Build

В Visual Studio: Configuration = Debug, Platform = MSVC_Emulator (не FT9XX!), Build → Build Solution. Результат:

```
C:\Users\Администратор\Desktop\EveApps\SampleApp\ZumaPlayback\Project\Msvc_Emulator\Debug\ZumaPlayback_Emulator.exe
```

Что делает ZumaPlayback.c

Эмулятор работает как **playback harness** — не интерпретирует Z80, а просто проигрывает заранее снятые «кадры» (RAM_G + cmd FIFO snapshots).

```

int main(int argc, char* argv[]) {
    // 1. Init FT812 эмулятор
    EVE_HalContext s_halContext;
    EVE_Hal_open(&s_halContext, ...);

    // 2. Load ram_g.bin → RAM_G FT812 (1 MB полный snapshot)
    char bundle_dir[256];
    strcpy(bundle_dir, argv[1]); // bundle path из CLI
    FILE* f = fopen(bundle_dir "/ram_g.bin", "rb");
    uint8_t ram[1024*1024];
    fread(ram, 1, sizeof(ram), f);
    fclose(f);
    EVE_Hal_wrMem(&s_halContext, 0, ram, sizeof(ram));

    // 3. Find all cmd_frame_*.bin in bundle, sorted
    // 4. For each frame: write cmd stream → EVE_Cmd_wr32, waitFlush, swap
    for (...) {
        FILE* cf = fopen("cmd_frame_XXXX.bin", "rb");
        // Read 32-bit commands one at a time
        uint32_t cmd;
        while (fread(&cmd, 4, 1, cf) == 1) {
            EVE_Cmd_wr32(&s_halContext, cmd);
        }
        EVE_Cmd_waitFlush(&s_halContext);
        EVE_Hal_wr8(&s_halContext, REG_DLSWAP, DLSWAP_FRAME);
        Sleep(500); // поддержать кадр на экране
    }

    // 5. Keep emulator window open
    while (1) Sleep(100);
}

```

Bundle export (Python side)

Bundle = папка с двумя видами файлов: - `ram_g.bin` — полный 1 MB snapshot FT812 RAM_G (assembled все assets в правильных адресах) - `cmd_frame_NNNN.bin` — snapshot CMD FIFO для конкретного frame number

Генератор: `Source/OTHER/export_ft812_bundle.py`. Делает:

1. **Парсит EQU константы** из `Source/ASM/main.asm` (RAM_G addresses всех assets)
2. **Собирает ram_g.bin** — кладёт каждый converted asset (bg paletted, balls, frog, frame strips, dialog_frame, fonts, palettes...) в свой EQU address
3. **Запускает full Z80 emulator** (`zuma_full_z80_emulator.py`) с реальным asm кодом
4. **Прогоняет N frames** через emulator (вызывает Frog_Update, VDC_Update, Bullet_Update, ZL_DrawFrame)
5. **На указанных frames** дампит CMD FIFO buffer → `cmd_frame_NNNN.bin`

CLI пример:

```
python Source\OTHER\export_ft812_bundle.py --out _ft812_bundle_test --frames 0,240,500,650
```

Опциональные флаги для тестирования специфичных состояний:

```
python Source\OTHER\export_ft812_bundle.py --out _ft812_bundle_dialog_test --frames 100 --force-dialog 1 --force-lives 2
```

`--force-dialog 1` — записать `VDC_DialogState=1` ПЕРЕД захватом frame, чтобы триггернуть рендер game-over диалога без необходимости проиграть партию.

Запуск эмулятора

```
$exe='C:\Users\Администратор\Desktop\EveApps\SampleApp\ZumaPlayback\Project\Msvc_Emulator\Debug\ZumaPlayback_Emulator.exe'
$bundle='C:\Users\Администратор\Desktop\Zuma Deluxe VDAC2\_ft812_bundle_dialog_test'
Start-Process -FilePath $exe -ArgumentList '"$bundle"' -WorkingDirectory (Split-Path $exe)
```

Открывается окно «BT8XX Emulator» с эмулированным экраном FT812 640×480. Кадры из bundle проигрываются по очереди с задержкой ~0.5 сек.

Что дало в v028

Глава 25 написана *после* того как эмулятор нашёл нам реальный баг которого Unreal x64 не видел.

Кейс: при рендере game-over диалога текст должен показывать "2 lives left", "GAME OVER" и т.д. В Unreal x64 текст рисовался корректно. На FT812 эмуляторе — **rainbow color noise** в позиции текста (Главу 26 см. ниже про сам баг).

Сценарий который сработал: 1. Написали DrawString routine, проверили в Unreal x64 — выглядит ОК. 2. Юзер запустил на реале → garbled. Не поверили — могли быть проблемы с реалом. 3. Запустили FT812 emulator → **то же garbled!** Стало ясно что баг в коде, не в реале. 4. Скриншот эмулятора через PowerShell `CopyFromScreen` → ВИДНО garbage rect ровно где должен быть текст. Position+size правильные, content = rainbow noise. 5. Гипотеза: BITMAP_SOURCE/SIZE применяются к WRONG handle. Подтверждено reading FT812 datasheet. Fix: эмитить BITMAP_HANDLE ПЕРЕД SOURCE/SIZE. 6. После fix → эмулятор показывает правильный текст. Юзер подтвердил на реале.

Без FT812 emulator мы бы либо: - Гоняли реал каждый цикл итерации (5-10 мин на цикл) - Или скипнули баг как «реальное hardware quirk» и оставили сломанным

С эмулятором цикл итерации ~30 сек, и баг был очевиден сразу.

Подход playback vs interactive

EveApps умеет и **interactive mode** — где C код напрямую обновляет DL каждый кадр через FT812 API. Это нужно для тестов с input/touch. Мы пока сделали только **playback** (RAM_G + cmd snapshots), потому что:

- Z80 логика — это наш asm, не C; портировать в C это дублирование
- Playback покрывает 90% случаев — рендер «застывшего» состояния
- Bundle экспорт за секунды, можно проверить любой state через `--force-*` флаги
- Snapshot не зависит от времени → reproducible

Interactive mode понадобится если будем тестировать timing-sensitive штуки (анимации, animations, transitions). Тогда напишем C harness который вызывает Z80 функции через ctypes или socket bridge.

Readback реального RAM_DL — ground-truth аудит Display List (доводка)

Главное, что дал этот эмулятор после первоначальной настройки, — **достоверный снимок настоящего Display List**. Наш Z80-код собирает кадр через co-processor (RAM_CMD FIFO); команды FIFO (`cmd_*`) — это НЕ готовый DL: копроцессор сам разворачивает их в реальные DL-команды. Поэтому «сколько команд

в DL» по нашему cmd-поток считать нельзя — нужно прочитать то, что копроцессор реально положил в `RAM_DL`.

`ZumaPlayback.c` пропатчен так: после `EVE_Cmd_waitFlush` (FIFO допроигран) читаем `RAM_DL` (адрес `0x300000`, **8 КБ**) и пишем в `dl_frame_NNNN.bin`, затем **headless-выход** (без удержания окна — для пакетного прогона). Сборка — `Debug|Win32` через MSBuild; в этом прогоне target эмулятора = **BT817** (совместимый EVE из того же EveApps). Bundle тот же (`ram_g.bin + cmd_frame_*`).

Парсер (`analyze_dl_*`) читает `dl_frame_*.bin` до команды `DISPLAY` и считает DL-команды по типам. Так мы впервые увидели реальные цифры бюджета строки (глава 24): тяжёлый кадр = **439 DL-команд**, ~192 из них — матрицы разворота шаров. **Только этот путь** разворачивает FIFO-копроцессорные команды в настоящий DL — Unreal x64 и наши Python-эмуляторы этого не делают.

Что осталось сделать

- ✓ ~Headless-режим для пакетного прогона~~ — сделано (DL-readback патч, см. выше): эмулятор пишет дампы и выходит без удержания окна.
- Авто-скриншот пикселей (PNG)** — сейчас картинку снимаем вручную через PowerShell `CopyFromScreen`; можно добавить `glReadPixels` → PNG в C-код.
- Diff-tests** — сравнивать скриншот/дамп DL с эталоном, fail если `diff > N`.

Ссылки

- Setup guide: `Docs/ft812_emulator_setup_guide.md` (раннее общее планирование)
- Source: `C:\Users\Администратор\Desktop\EveApps\SampleApp\ZumaPlayback\Src\ZumaPlayback.c`
- Bundle exporter: `Source/OTHER/export_ft812_bundle.py`
- Чат.txt: `[2026-05-19 13:42] Codex -> FT812 emulator playback harness ready` — Codex set-up note.
- Чат.txt: `[2026-05-20 02:40] VDAC2 -> VDC: v028 Game Over dialog` — описание bug который эмулятор нашёл.

Глава 26. BITMAP_HANDLE binding ловушка FT812 (2026-05-20)

TL;DR

`BITMAP_HANDLE` в FT812 это **selector** одного из 32 slots с per-handle bitmap state. Команды `BITMAP_SOURCE`, `BITMAP_LAYOUT`, `BITMAP_SIZE` записываются в **текущий** selected handle. Если эмитить `SOURCE` → `HANDLE` → `SIZE` → `Vertex2f`, то `SOURCE` попадает в OLD handle (тот что был selected до), и новый handle остаётся со старым source (garbage или previous frame leftovers). Symptom: **ЯРКИЙ** rainbow / multi-color noise ровно в bounding box того места где должна быть текстура.

Семантика BITMAP_HANDLE в FT812

FT812 имеет 32 "bitmap slot" (handles 0..31).

Каждый slot хранит свои:

- BITMAP_SOURCE (address в RAM_G)
- BITMAP_LAYOUT (format + stride + height)
- BITMAP_SIZE (filter + wrap + width + height)
- BITMAP_TRANSFORM_* (matrix coefficients)
- PALETTE_SOURCE (для PALETTED формата)
- ... остальные per-bitmap state regs

BITMAP_HANDLE(N) переключает "active slot" в N.

Все последующие BITMAP_* команды модифицируют active slot.

BEGIN(BITMAPS) + Vertex2II/Vertex2f — рисуют из active slot.

Bad pattern (rainbow noise!)

```
; Try to draw glyph with runtime BITMAP_SOURCE switch
LD HL, glyph_addr_lo
LD A, glyph_addr_hi
LD B, #01 : LD C, A : LD D, H : LD E, L
CALL Command_BCDE ; ← BITMAP_SOURCE emit (но какой handle active?)
FT_BitmapHandle GLYPH_HANDLE ; ← switch slot ПОСЛЕ source emit
FT_BitmapLayout FT_ARGB4, ...
FT_BitmapSize FT_NEAREST, ...
XOR A : CALL FT.Coprocessor.Cell
LD BC, x*16 : LD DE, y*16
CALL FT.Coprocessor.Vertex2f
```

Что происходит: - Active handle = previous (e.g. FRAME_HANDLE) - BITMAP_SOURCE записывается в previous slot (perverts state of frame!) - FT_BitmapHandle GLYPH_HANDLE — switches active - BITMAP_LAYOUT/SIZE записываются в GLYPH_HANDLE (правильно) - Vertex2f читает из GLYPH_HANDLE: source = whatever было раньше (или default) = читаем garbage RAM_G как ARGB4 → rainbow noise

Good pattern

```
FT_BitmapHandle GLYPH_HANDLE ; ← FIRST select slot
LD HL, glyph_addr_lo ; теперь setup runtime addr
LD A, glyph_addr_hi
LD B, #01 : LD C, A : LD D, H : LD E, L
CALL Command_BCDE ; ← BITMAP_SOURCE → GLYPH_HANDLE (correct)
FT_BitmapLayout FT_ARGB4, ...
FT_BitmapSize FT_NEAREST, ...
XOR A : CALL FT.Coprocessor.Cell
LD BC, x*16 : LD DE, y*16
CALL FT.Coprocessor.Vertex2f
```

Защита от случайной перестановки

В DrawString routine (vo28 baseline) специально сделана пара PUSH/POP вокруг HANDLE emit чтобы можно было запустить вычисленный addr ДО switch handle:

```

; HL = glyph addr lo, A = glyph addr hi уже вычислены
PUSH HL
PUSH AF
FT_BitmapHandle GLYPH_HANDLE ; макро clobbers BCDE — нужно сохранить HL/AF
POP AF
POP HL
; Теперь HL/AF сохранены, эмитим SOURCE
LD B, #01 : LD C, A : LD D, H : LD E, L
CALL FT.Coprocessor.Command_BCDE

```

Когда баг проявляется

- Switch font/atlas внутри DrawString loop (см. vo28 DrawDialogContent с SetFontNative + SetFontCancun8)
- Switch sprite state (normal/hover/pressed) внутри dialog button draw
- Любое место где runtime BITMAP_SOURCE emit'ится «не от того» handle

Почему Unreal x64 НЕ видит этот баг

Unreal x64 эмулирует FT812 как «один глобальный bitmap state» (упрощённо), не как 32 отдельных slot. Поэтому BITMAP_SOURCE применяется к НОВОМУ active handle сразу после BITMAP_HANDLE — независимо от порядка emit. Реальный FT812 + Bridgetek эмулятор моделируют per-slot state правильно → видят баг.

Урок: для bug'ов которые «вокруг handle binding» — Unreal x64 не подходит, нужен Bridgetek emulator или реальное железо.

Ссылки

- vo28 baseline: [releases/v028-2026-05-20-game-over-dialog.spg](#)
- Memory: [reference_ft812_bitmap_handle_binding](#)
- Чат.txt: [\[2026-05-20 02:40\]](#) bug section
- FT81x Programmers Guide §4.30 BITMAP_HANDLE — описание slot binding (но не явно про порядок emit'a — нашли через эмулятор debugging).

Глава 27. Сессия 2026-05-20: scoring engine, matrix LUT, ARGB4 frog → tearing fix

Серия оптимизаций и багфиксов за один день, кульминация — устранение tearing на реальном железе через смену формата frog слоёв.

27.1 Match-3 «синие шары вместо gap» — критический bug

Симптом: после анимации match-3 на месте удалённой группы появлялись 3 синих шара (color 0) вместо пустых ячеек.

Root cause: в `VDC_CheckMatch3.m3_have_marker` регистр В хранил marker (#FE GAP_STOP / #FD CASCADE), затем Codex добавил stats-блок ниже с подсчётом points. `LD B, A` в gauge-loop клобал В значением `TmpMCount` (=3). После DJNZ loop B=0, и потом `LD (HL), B` писал 0 в `ExplodeMarker[lb..rb]`. При финализации `Slots[idx]=0` = синий шар.

Fix: `PUSH BC` сразу после `.m3_have_marker:`, `POP BC` перед записью в `ExplodeMarker`.

Урок: при добавлении кода в существующую функцию — отметить все клобаемые регистры. В и С особо опасны в scoring/UI коде где много `LD B, A` для loop counter.

27.2 Scoring engine 1:1 с HD-ref Statistics.c

Реализована полная формула очков из `Statistics.c:37`:

```
points = count*10 + combo*100 + (100 + 10*(chain-5))
      ^^^ если chain>=5 AND combo==0
```

State в `VDC.asm`: `VDC_StatChainCount` (reset на miss-shot), `VDC_StatCombos`, `VDC_GaugeScore` (true), `VDC_GaugeShown` (animated LERP +8/frame), `VDC_BulletGapMinDist` / `Count` для gap_bonus.

Gap bonus в 1D VDC через новый entry `VDC_SlotPosAllowGap` (без skip-on-gap). На expire bullet off-screen без hit — `VDC_AwardGapBonus` начисляет очки, max 32/shot.

Spawn gate: `VDC_TrySpawn_NoHsubGate` блокируется при `VDC_GaugeFull != 0`.

Тесты: `test_gauge_score_z80.py` верифицирует формулы через реальный CALL `VDC_CheckMatch3`.

27.3 Точный fill_px для прогресс-бара

Заменял `score/16` на математически точную `(GaugeShown * 63) / 1000`. На Z80 нет 16-bit division, делим в два прохода через `ZL_Mul16x8` + `>> 3` + `VDC_DivHLbyA(125)`.

27.4 FT_ScissorXY клобает B — повторение pattern

Симптом: при score=30 бар показывал ~27 px вместо 1.

Cause: `LD B, A` (fill_px) → `FT_ScissorXY` (FT_CMD_BUF inline-ит LD BC, opcode_high → B = 0x1B) → `LD A, B` берёт клобнутое 27.

Fix: `LD (DhpFillPx), A` ПЕРЕД `FT_ScissorXY`, `LD A, (DhpFillPx)` ПОСЛЕ.

Правило ужесточено: **ВСЕ FT_* макросы клобают BCDE. Не держать критические данные в регистре через FT_CMD_BUF.**

Поймал через `test_draw_progress_z80.py` — Z80 тест вызывает CALL DrawHudProgress и парсит SCISSOR_SIZE opcode из emitted CMD buffer.

27.5 GaugeShown animated LERP — плавный бар

Score прыгает мгновенно (chain bonus +100), но бар анимируется +8 pts/frame через `VDC_TickGaugeShown` в конце `VDC_AnimateChain`. ~12 кадров до полного догона = ~0.2 сек = выглядит плавно.

27.6 Pre-baked rotation matrix LUT

CMD-цепочка (`LOADIDENTITY` → 2x `TRANSLATE` → `ROTATE` → `SETMATRIX`, 40 bytes + coproc work) заменена на **LUT 32 x 24 bytes = 768 bytes**. Генератор `make_chain_matrix_lut.py` считает Q8.8 cos/sin + Q23.8 translation.

Helper `ZL_EmitBallMatrixFromBRAD` (12 LOC) — LDIR 24 bytes из LUT в FT BufferPtr.

Sign convention: эмпирически инвертировано `B+=sin, D=-sin` + C/F пересчитаны как `cx*(1-cos)-cy*sin / cx*sin+cy*(1-cos)`. Совпало с CMD_ROTATE coprocessor output.

Применён в `Frog_DrawBallNow` — шар во рту лягушки крутится синхронно с frog aim.

Lazy BITMAP_HANDLE switching (`ZL_TmpLastHandle`) — skip emit если same handle.

DL byte savings: v028 ~2900 → v030 2232 bytes/frame (-23%).

27.7 OK button hit-test + Fire trigger

OK button рисуется при state ∈ {1, 2}. Click hit-test bounds [170..470, 315..349]. Также Fire trigger через SPACE/Kempston с rising edge debounce.

27.8 MENU sprite skip baked при idle

`DrawHudMenu RET Z if HudMenuState=0`. Idle state уже запечён в `frame_top.png`.

27.9 Главный фикс tearing: ARGB4 + NEAREST для frog

После всех оптимизаций tearing продолжался на реале. Pixel-clock analyzer показывал budget 14% — HE виноват.

Codex нашёл root cause: frog слои body/plate/tongue/overlay рисовались **PALETTED4444 + BILINEAR**.

Per output pixel cost:

Формат	Reads/px
ARGB4 NEAREST	1
ARGB4 BILINEAR	4 (4 taps)
PALETTED4444 NEAREST	2 (index + palette entry)
PALETTED4444 BILINEAR	8 (4 taps × 2 reads)

Frog был **PALETTED4444 + BILINEAR** = 8 reads/px × 122×122 × 4 layers ≈ 476K reads/кадр.

Fix: `FROG_ARGB4_ENABLED EQU 1` — переключение в **ARGB4 + NEAREST** = 1 read/px × 122×122 × 4 = 59.5K reads/кадр (**8× меньше**).

Tearing устранён. v031 опорная (`releases/v031-2026-05-20-argb4-frog-no-tearing.spk`).

Урок: мой analyzer использовал упрощённую модель (NEAREST=16 px/clock, BILINEAR=2 px/clock) и **не учитывал palette lookup overhead** PALETTED формата. Если tearing на реале — сначала проверить crucial sprites на BILINEAR/PALETTED и попробовать ARGB4+NEAREST.

27.10 Outcome дня

После Codex v028 baseline: - **v029**: scoring engine + match-3 blue fix + FT_ScissorXY clobber fix - **v030**: matrix LUT + lazy handle + frog ball rotates (-23% DL bytes) - **v031**: ARGB4 frog → tearing устранён (текущая опорная)

Все харнесс тесты PASS. Цель сессии достигнута — играбельный билд без видимого tearing на реальном железе ZX Evo + FT812 @ 74Hz.

Глава 28. Mr.Gluk RTC чтение и часы на экране (2026-05-20)

28.1 Постановка

После добавления cluster-RNG для chain spawn'a юзер заметил: на каждом запуске игры выпадает **одна и та же последовательность шаров**. RTC не влияет на seed.

Симптомы: - 2 кадра screenshot'ов с рестартом → идентичные цвета первых ~20 шаров - F12 dump показывал `VDC_LfsrSeed = 0x9624` для каждого запуска - Распределение синих шаров (color 0) почти нулевое

28.2 Диагностика

Шаг 1. Сохранил entropу-источники в фиксированную RAM область:

```
LD (#5008), A      ; raw RTC sec (parsed)
LD A, R
LD (#5009), A      ; R refresh register
LD HL, (ZL_FrameCounter)
LD (#500A), HL     ; FrameCounter at VDC_Init time
LD HL, (VDC_LfsrSeed)
LD (#500C), HL     ; final seed
```

F12 dump → парсер → результат:

```
#5008 RTC parsed = 165 (0xA5)
#5009 R register = 0
#500A FC         = FF00
#500C seed       = 9624
```

165 — расшифровка BCD от 0xFF: `high*10 + low = 15*10 + 15 = 165`. То есть порт #BFF7 возвращал **0xFF** (нет данных).

Шаг 2. Юзер указал на Wild Commander (показывает реальное время в Unreal). Значит RTC в эмуляторе работает. Бага в нашем коде.

Шаг 3. Изучил ZiFi source (`C:\Users\Администратор\Desktop\WC\ZiFi\zifi.asm:4206`) — рабочее чтение GLUK:

```
write_rtc
    ld a, #80
    ld bc, #eff7      ; <-- АКТИВАЦИЯ через #EFF7
    out (c), a
    ...
    ; работа через #DFF7 (адрес) / #BFF7 (данные)
    ...
    ld a, #00
    ld bc, #eff7
    out (c), a        ; <-- ДЕАКТИВАЦИЯ
    ret
```

Я использовал `#DEF7` (ошибочно из памяти, никогда не работало). Правильный порт активации — **#EFF7**.

28.3 Правильная процедура чтения Mr.Gluk

1. OUT #EFF7, #80 — enable Mr.Gluk (включает порты DFF7/BFF7)
2. OUT #DFF7, reg_idx — выбрать регистр (0=сек, 2=мин, 4=час, 7=день, 8=мес, 9=год)
3. IN A, (#BFF7) — прочитать BCD значение
4. OUT #EFF7, #00 — disable Mr.Gluk (вернуть порты другим устройствам)

BCD → binary: `value = (raw>>4)*10 + (raw&0xF)`.

28.4 Применение к Zuma VDAC2

После замены `#DEF7` на `#EFF7` в `ReadRTCSeconds` и `ReadRTCRegister`: - RTC возвращает реальное wall-clock значение - Каждый запуск с другой секундой → seed для chain spawn разный - `VDC_GameSeconds` правильно инкрементируется в `VDC_UpdateRtcElapsed`

28.5 Часы в нижней рамке

`DrawDebugClock` в `MainLoop.asm:204`:

```
DrawDebugClock:
; HH @ (28, 438)
LD A, 4
CALL ReadRTCRegister
LD C, A : LD B, 0
LD DE, 0
FT_NumberDEBC 28, 438, 26, 2
; ':' через FT_CMD_TEXT (ASCII 0x3A + NUL padding)
FT_Text 46, 438, 26, 0
FT_CMD_BUF #0000003A
; MM @ (52, 438)
LD A, 2
CALL ReadRTCRegister
LD C, A : LD B, 0
LD DE, 0
FT_NumberDEBC 52, 438, 26, 2
; ':' @ (70, 438)
FT_Text 70, 438, 26, 0
FT_CMD_BUF #0000003A
; SS @ (76, 438)
XOR A
CALL ReadRTCRegister
...
```

Геометрия: - Bottom frame занимает `Y=456..479` (24 px) - Left frame занимает `X=0..23` (24 px) - Часы на `Y=438` (18 px над bottom frame), `X=28` (за left frame + 4 px отступ) - FT812 ROM font handle 26 = 8×16 → "HH:MM:SS" ≈ 64 px width

`FT_NumberDEBC` принимает число в BC (low 16) + DE (high 16). `Options=2` = 2-digit padding (00..99).

`FT_Text` для двоеточия: после cmd word передаётся 4-byte aligned NUL-terminated строка через `FT_CMD_BUF #0000003A` (":\0\0\0" в LE).

28.6 Фикс TIME в Game Over dialog

`DrawTimeValue` раньше использовал `VDC_StatTimeFrames / 60` — это давало неверный результат при 74Hz видеорежиме (74 frames/sec ≠ 60). Заменено на `VDC_GameSeconds` (RTC-based секунды, pause

excluded).

28.7 Что в memory

- `[[feedback_zuma_vdac2_ft_cmd_buf_clobbers_bcde]]` — Mr.Gluk activation port = #EFF7.

Глава 29. Когда эмулятор сам с багом — горький урок (2026-05-20)

29.1 Симптомы

Два разных бага в match-3 одновременно: - **Ложный** match-3 после gap closure без реального схлопывания одноцветных шаров. - **Пропущенный** match-3 после half-cell insert: визуально 3 шара одного цвета подряд, match НЕ срабатывает.

Я пытался лечить через `Source/OTHER/vdc_visual_emulator.py` — это Python mirror VDC physics с 2D визуализацией. Тесты показывали зелёное, но на реальном железе симптомы оставались.

29.2 Корень проблемы — Python эмулятор фазово drift'ил от ASM

Python emulator имел старый update-order на тике:

```
try_spawn → move_chain → animate_chain
```

ASM фактически использовал: - **Fast phase** (BallsSpawned < LEVEL_START_BALLS): `12×MoveChain → AnimateChain → TrySpawn_NoHsubGate` - **Normal phase**: `MoveChain → AnimateChain → TrySpawn`

Из-за рассинхрона Python показывал правдоподобные, но **фазово неверные** цепочки: - spawn попадал на другой HSub - HSA/SlotsLen жили по другому расписанию - delayed match (когда insert half-cell match становится валидным через несколько кадров offset decay) в Python работал, в ASM — нет

Эмулятор как oracle подсовывал ЛОЖНЫЕ объяснения: я «чинил» симптомы в модели, не реальные invariants в RAM.

29.3 Кто закрыл — Codex через RAM dump

Codex отказался от Python-эмулятора как ground truth и взял за источник истины **прямой RAM dump** (F12 в Unreal → `parse_log_dump.py` + чтение `VDC_Slots`, `VDC_Offsets`, `VDC_Shot2`, `VDC_ExplodeFrame`, `VDC_ExplodeMarker` по адресам из `main.lst`).

Из RAM dump'ов 111/222/333/444 Codex увидел: - В 111 — `MATCH3` без предшествующего `SHOT/BBOX/INSERT` (stale Shot2 trigger). - В 222 — settled одноцветные ряды, но Shot2 уже очищен (преждевременная очистка). - В свежем 111 — `BallsSpawned=35`, `Slots=GAP_STOP`, `GaugeScore=110`, `GaugeFull=0` → normal spawn вообще не срабатывает.

29.4 Применённые ASM-фиксы

1. `VDC_SetShot2OnNeighbors` — narrow trigger:
2. Было: Shot2 на K-1 и K если оба non-gap.
3. Стало: Shot2 ставится только если `slot[K-1] == slot[K]` (реальное one-color closure).
4. `VDC_ScanForNewMatch` — pending Shot2:

5. Было: в конце loop без match'a — clear ALL Shot2.
6. Стало: pending Shot2 держится пока offsets около слота не settled. Half-cell insert match получает несколько кадров на decay.
7. `VDC_TrySpawn_NoHsubGate` — убран stop по `BallsSpawned >= TARGET`. `BallsSpawned` теперь только saturating debug counter. Реальный spawn-stop — `VDC_GaugeFull` или Game Over.
8. Убран `HSA == TrackNumSlots` ранний stop — обработка конца track теперь через `VDC_CheckKillzone`.
9. Убран gate `FrameCounter & 63 == 0` в normal phase — он фазово конфликтовал с внутренним `HSub == 0`, spawn никогда не срабатывал после fast phase. `VDC_TrySpawn` сам gate'ится по HSub.

29.5 Что синхронизировано в Python эмуляторе

`vdc_visual_emulator.py` приведён к ASM: - fast/normal update order совпадает с ASM - `try_spawn(no_hsub_gate=True)` для fast phase - saturating `balls_spawned` - narrow Shot2 trigger (same-color closure only) - pending Shot2 до settled offsets

29.6 Урок

Правило: Когда баг phase-sensitive (timing'и `Shot2`, `ExplodeFrame`, half-cell insert), **НЕ доверять** визуальному Python-эмулятору как oracle. Источник истины: 1. F12 dump → `parse_log_dump.py` для ring buffer событий 2. Прямое чтение `VDC_Slots/Offsets/Shot2/ExplodeFrame` по адресам из `main.lst` 3. Сверка с ASM update-order

Python emulator может «показывать что баг исправлен», но физически — нет. Это худший вариант false positive: тесты зелёные, прод сломан.

Аналогия — это как чинить машину по симулятору, у которого двигатель работает не так, как в реальном железе. Можно «исправить» проблему в симуляторе и думать что готово, а реальное авто продолжаетглохнуть на том же месте.

29.7 Память на будущее

- `[[feedback_zuma_vdac2_emulator_oracle_drift]]` — правило не доверять Python ему для phase-sensitive багов.
- `[[feedback_zuma_vdac2_full_z80_emulator_unreliable_for_render]]` — full Z80 emu тоже не годится для render-багов.
- `[[reference_zuma_debug_env_methodology]]` — официальная методичка SOP для VDC регрессий через RAM dump.

Глава 30. FAT32 с SD-карты в TS-Conf: от WC ZiFi к собственному драйверу RawPak (CMD17 + LFN)

30.0 Зачем эта глава

В TS-Conf нет полноценного DOS. SPG-программа запускается из Wild Commander (WC) с минимальным runtime, без привычной файловой системы. Когда мы решили выносить 22 уровня (и будущее GS2MB-аудио) в отдельный файл на SD-карте, выяснилось — это отдельный кусок науки.

Глава прошла **две эпохи**. Сначала грузили через готовый драйвер **WC ZiFi** — и упёрлись в порчу RAM_G (см. §30.1). Затем написали **собственный FAT32-ридер RawPak** поверх прямого **CMD17** к SD через Z-Controller — он и работает на железе (опорная v039+). Текущий раздел ведёт от current-подхода; ZiFi оставлен как разобранный тупик.

30.1 Историческая развилка: почему отказались от WC ZiFi

Что такое ZiFi. Драйвер file-I/O (Koshi, `WC/ZiFi/zifi.asm`) с JP-таблицей API на странице `#0F` от адреса `#2002` (`CORE`): `DEV_INI` / `HDD` / `LOAD512` / `FENTRY` / `LOADNON` / `SETDIR` / `SEEK0` и т.д. Драйвер кладётся в наш SPG как блок на `#0F`, инициализация: `DOS_SWP` **первым** (распаковывает trampoline в `#3C00+`), потом `DEV_INI` → `HDD` → `SETR00T`. Slot-o swap — только через MMIO-макрос `SetPage0_A` (порт `#10AF` write-only, читать нельзя — вернёт `#FF`).

Где сломалось. ZiFi-стриминг фона уровня **клат мусор в RAM_G на реальном железе** (в эмуляторе было чисто — fake-ZiFi путь не воспроизводил баг). Данные, палитра и сам draw были корректны (Python-рендер давал чистую спираль), но именно ZiFi-путь портил выгруженные в FT812 байты. Сопутствующее: `CurrentLevel` в TSLib-region (`#1938`) корраптился; `StreamSection` заикливался; на части уровней `cmd_inflate` вис. Корень — гонка SPI-шины / MMU вокруг `FT.WriteMem` при чередовании с чтением SD внутри ZiFi.

Вывод: чужой драйвер с непрозрачным mount-state и собственным управлением шиной давал баги, которые мы не могли ни увидеть в эмуляторе, ни поправить. Решили написать **свой** минимальный ридер, где мы контролируем каждый SPI-такт.

30.2 RawPak: собственный FAT32-ридер на прямом CMD17

`Source/ASM/ts-dos.asm`, символы `RawPak_*`. Принципы:

- **Прямой CMD17** (single-block read, опкод `%01000000+17`) к SD через Z-Controller (`#57` / `#77`). Никакого mount-state: каждый LBA-сектор RawPak вычисляет сам (`cluster>>7 + FatStart`), поэтому ничему «дрейфовать» нечем.
- **CMD17, не CMD18.** Multi-block `CMD18` на этом железе вёл себя нестабильно; одиночный `CMD17` на хосте Unreal и на реале работает идеально (снимок BPB корректен). Совет подтверждён практикой.
- **Self-contained:** BPB, FAT-цепочки, LFN-поиск — всё своё, минимально достаточное под одну задачу «найти и прочитать `ZUMALVL.PAK`».

Низкоуровневый `CMD17` — в `Source/ASM/sd_zc.asm` (`sd_read_sector` / `sd_cmd17` / `sd_wait_token`). Адресация байтовая (host `sd_blk=0`), `sd_wait` **ограничен таймаутом** (а не вечный спин — иначе виснет шина).

30.3 BPB и открытие тома — RawPak_OpenRoot

Читаем сектор 0 (BPB «superfloppy», без MBR), валидируем и кэшируем геометрию:

```
RawPak_OpenRoot:
; CMD17 sector 0 -> BPB в IX-буфер
; требуем bytes/sector == 512 (иначе ошибка #A2)
; требуем sectors/cluster != 0 (иначе #A3)
; FatStart = (IX+14) reserved sectors (32-бит)
; DataStart = FatStart + NumFATS*FATSz32
; RootClus = BPB+44
; CMD17-ошибка чтения BPB -> #A1
```

Коды ошибок (A при возврате): #A1 BPB CMD17 fail, #A2 bytes/sec#512, #A3 spc==0. Дальше CurClus = RootClus — мы «в корне».

30.4 LFN-сопоставление пути /Games/Zuma Deluxe VDAC2/ZUMALVL.PAK

RawPak_FindInCurrentDir ищет одно имя в текущей директории, сопоставляя по ДЛИННОМУ имени (LFN), а если LFN нет — по 8.3. Это робастно к коротким алиасам инжектора (GAMES~1, ZUMAD~1):

- RawPak_StoreLfn — собирает фрагменты LFN (offsets 1,3,5,7,9, 14,16,18,20,22,24, 28,30) в RawPak_EntName по (seq-1)*13.
- RawPak_Upcase + RawPak_Build83 + RawPak_NameMatch — регистронезависимое сравнение с RawPak_TargetName.

Путь проходим по шагам: найти Games → SETDIR (CurClus = найденный кластер) → найти Zuma Deluxe VDAC2 → найти ZUMALVL.PAK. Проверено на реальном хост-образе (verify_lfn_walk.py).

30.5 FAT-цепочка: FatNext + ловушка AdvanceOne

RawPak_FatNext по номеру кластера читает запись FAT: сектор = cluster>>7 + FatStart, смещение в секторе = (cluster & 127)*4. Использует отдельный буфер RawPak_FatBuf (512 Б, резидент в Core/slot 1), чтобы не конфликтовать с буфером данных.

● **B-clobber (исправлено).** FatNext клобаёт B. RawPak_SkipB держал в B счётчик DJNZ для пропуска секторов → после первого FatNext счётчик ломался, SkipB(1) уезжал по всей цепочке до EOC → нулевой ТОС. Фикс — обёртка RawPak_AdvanceOne = PUSH BC : CALL FatNext : POP BC. (Воспроизведено локально на харнесе — §30.9.)

30.6 Таблица секторов: LBA = PakLba + N (допущение непрерывности)

PAK выложен непрерывно, поэтому вместо пер-секторного FAT-walk кэшируем LBA логического сектора O файла (RawPak_SetPakLba → RawPak_PakLba), а дальше любой логический сектор N читается как LBA = PakLba + N (верно для непрерывных кластеров при любом spc). Убраны seek-by-read и пер-секторный обход FAT — быстро и просто.

⚠ ⚠ **КРИТИЧЕСКОЕ ОГРАНИЧЕНИЕ — фрагментация файла.** $LBA = PakLba + N$ верно ТОЛЬКО для непрерывного (нефрагментированного) PAK. Если файл лежит на карте кусками (FAT-цепочка с разрывами — обычное дело после многократной перезаписи карты), то логический сектор N окажется не там, и на реальной SD-карте игра **загрузит мусор или зависнет** — баг, который не воспроизведётся в эмуляторе с «чистым» образом.

Наш инжектор (inject_zuma_to_wc_img.py) пишет PAK **одним непрерывным куском** и это проверяется check_pak_chain.py. Если выкладываете PAK на карту вручную — кладите на свежееотформатированную/дефрагментированную карту и проверяйте непрерывность. Универсальное решение (без допущения непрерывности) — мультиран-таблица секторов, построенная обходом FAT-цепочки (в бэклоге).

30.7 Двухфазная загрузка: FT812 и SD на одной SPI-шине

FT812 и SD-карта висят на **одной** SPI-шине (#57 / #77). Чередование чтения сектора SD и FT.WriteMem крашило шину. Решение — **двухфазно**:

1. **Фаза 1:** прочитать с SD в RAM ВСЁ нужное (bg → страницы `#07..#0E`, palette → `#03`, track → `#06` (+ `#0F`)).
2. **Фаза 2:** залить ВСЁ в FT812 одним проходом `FT.WriteMem`.

Один переход SD→FT за загрузку вместо тысяч чередований — шина стабильна.

30.8 Трек на 2 страницы (`#06` + `#0F`) — верхние уровни

Самостоятельная подзадача (v040→v041). Загрузчик отвергал трек ≥ 33 секторов (трек не влезал в одну 16-КБ страницу `#06`) → падал на L04/07/16/17/18/20–22 (их трек > 3276 сэмплов). Это был **не** эффект фрагментации ПАК (ПАК непрерывен).

Фикс — трек на **две страницы**:

- Pack (`make_level_pack.pagesplit_track`): chunk A → `#06`, chunk B → `#0F` (page-aligned).
- Loader: лимит `CP 65` (вместо 33), читаем 32 сектора в `#06` + остаток в `#0F`.
- Runtime (`RawPak_ReadSampleAtHL`, вынесен в Core): чтение сэмпла трека **page-aware**. Константы: `TRACK_PAGE2 EQU #0F`, `TRACK_SPLIT_SAMPLE EQU 3276` (= $(16384-2)/5$; сэмпл трека = 5 байт, в `#06` первые 2 байта — заголовок, адрес сэмпла `t = #8000+2+t*5`; для `t ≥ 3276` берём из `#0F` по `t-split`).

Проверено harness'ом (L21 CF=1, round-trip PASS) и на хосте.

30.9 Инструмент: локальный Z80-харнесс FAT (не гонять хост зря)

`Source/OTHER/test_rawpak_z80.py` — гоняет **реальный Z80** `RawPak` в эмуляторе (`zuma_full_z80_emulator`) с хуком `sd_read_sector` на FAT-образ инжектора (`Build/test_wc.img`, собран `inject_zuma_to_wc_img.py --out-img`) — **без** хостовых циклов. Поймал B-clobber (§30.5), проверил двухфазный loader (CF=1, `GpDbgStep=#06`, `FT.WriteMem` ×9).

Границы: эмулятор (cburbridge) поддерживает не все опкоды и медленный на seek-by-read; HW-баги шины не воспроизводит (`FT.WriteMem` захукан). Доп. инструменты: `verify_lfn_walk.py`, `check_pak_chain.py`, `check_host_wc_img.py`, `parse_dump_diag.py`.

30.10 Pack-формат ZUMALVL.PAK

Секции выровнены по 512 Б (`CMD17` читает блоками). Собирает `Source/OTHER/make_level_pack.py`:

```
Sector 0 (512 B): Header
+0 magic "ZLVP", +4 version=1, +5 level_count=22, +6 sector_size=512
Sector 1 (512 B): TOC[22] × 20 байт на уровень
per entry: bg_off/size, pal_off/size, track_off/size, title_off/size,
            preview_off/size (по 2+2 байта; 0xFFFF в *_off = absent)
Sector 2..N: data blob (на уровень: bg → pal → track → title → preview),
            каждая секция с 512-кратного offset
```

Python-верификатор распаковывает ПАК и сверяет каждую секцию байт-в-байт с исходными `.bin`. Запускать после каждой правки pack-builder'a. (Превью уровней вынесены в ПАК ещё в v038 — страницы `#CC..#FA` закомментированы в `spgbld_vdac2.ini`.)

30.11 Миф «размер SPG ломает загрузку» — опровергнут

Долгая сессия v039 шла по ложному следу: казалось, что WC SPG-loader виснет, если SPG «слишком большой», и помогло срезание (страница `#0F` бандл-драйвера + диаг-буферы: 1929216 → 1284096 Б). Но позже **контрпример**: SPG Слободчикова `SFv1.1.spg` = 3172352 Б (2.5× нашего, 1.6× «сломанного») грузится тем же WC-loader'ом. **Потолка по размеру нет**. Реальная причина наших фейлов — флака SD (v037 грузился со 2-й попытки) / битый блок / гонка шины. Тримминг был обходом симптома.

Практический вывод: при зависшем WC-загрузчике или сломанном пейджинге RAM-дамп бесполезен — slot 1 ≠ Core, диагностика не читается. Не гнать размер SPG вверх без нужды, но и не считать его причиной.

30.12 Ловушки (anti-patterns)

Грабли	Симптом	Урок
ZiFi-стриминг фона	Мусор в RAM_G только на реале (эмулятор чист)	Гонка SPI/MMU вокруг FT.WriteMem; ушли на свой CMD17-ридер
<code>CMD18</code> multi-block	Нестабильно на железе	Только <code>CMD17</code> single-block
<code>FatNext</code> клобаёт <code>B</code> под <code>DJNZ</code>	<code>SkipB</code> уезжает до EOC → нулевой TOC	<code>AdvanceOne = PUSH BC : FatNext : POP BC</code>
Чередование SD-read и <code>FT.WriteMem</code>	Краш шины	Двухфазно: сначала всё в RAM, потом всё в FT
Трек > 1 страницы (16 КБ)	Loader отвергал ≥33 сект → fallback (мусор)	Трек на 2 страницы <code>#06</code> + <code>#0F</code> , runtime page-aware
<code>LOADNON B=255</code> для skip 500+ сект	<code>B</code> 8-битный, переполнение	16-битный счётчик через цикл
<code>IN A, (#10AF)</code> для save current page	Возвращает <code>#FF</code> (write-only)	Не читать MMU-порт; restore жёстко на TSLibPage
Поиск по 8.3 (<code>GAMES~1</code>)	Не совпадает с реальным алиасом	Сопоставлять по LFN
<code>samefile()</code> для UNC <code>\\tsclient</code>	Ложный False	Сравнивать <code>os.path.normcase(abspath(...))</code> строк
Срезать SPG «потому что большой»	Лечит симптом, не причину	Потолка размера нет; причина — флака SD/шины

30.13 Build pipeline

`build_wc_img.cmd` (CRLF + UTF-8; кириллица в UNC собирается через PowerShell):

- `sjasmplus Source\ASM\main.asm` → `Build/{Core.bin, main1_play.bin, TSLib.bin, zuma.sym}`
- `spgbld -b spgbld_vdac2.ini Build\zuma_vdac2.spg`
- `python Source\OTHER\make_level_pack.py` → `Build\ZUMALVL.PAK`
- `python Source\OTHER\inject_zuma_to_wc_img.py` — in-place inject в хостовый `wc.img`

Перед запуском **закрыть Unreal.exe на хосте**, иначе `WinError 32` (файл занят).

30.14 Связано

- `Source/ASM/ts-dos.asm` — драйвер `RawPak_*` (BPB, LFN, FAT-цепочка, sector-table).
- `Source/ASM/sd_zc.asm` — низкоуровневый `CMD17` (Z-Controller).
- `Source/OTHER/make_level_pack.py` — pack builder + verifier (+ `pagesplit_track`).
- `Source/OTHER/test_rawpak_z80.py` — локальный Z80-харнесс FAT.
- `WC/ZiFi/zifi.asm` / `_sd/VBI.ASM` (Koshi) — каноничный ZiFi (исторический референс).
- Memory: `reference_zuma_vdac2_fat32_driver_rawpak`, `reference_zuma_vdac2_baseline_2026-05-26_v039`, `reference_zuma_vdac2_upper_levels_track_too_big`, `reference_zuma_vdac2_spg_size_breaks_wc_loader`, `reference_zuma_vdac2_rawpak_z80_harness`, `reference_zuma_vdac2_zifi_streaming_blockers`.

Глава 31. Adventure-режим: уровни из таблицы, перенос счёта, Win/Pause (v035–v041)

После того как заработали загрузка уровней с SD (глава 30) и весь рендер, осталось собрать из этого **игру**: меню → выбор уровня → партия → победа → следующий уровень, с настройками каждого из 22 бордов и четырьмя уровнями сложности. Эта глава — про игровые системы поверх движка.

31.1 Поток adventure

```
Main Menu → Level Select → Game (PLAY) → Win (LEVEL DONE) → AdvanceToNextLevel → Intro → ...
\ Game Over → restart
```

- **Level Select**: превью борда (280×170 ARGB4) + название; кнопка Back заблокирована на L1; PREV/NEXT листают, палитра превью переключается под уровень.
- **4 сложности** — кнопки Rabbit / Eagle / Jaguar / SunGod (`LevelSelect.asm`), пишут `CurrentDifficulty` 0..3.
- Вход в партию — `FadeLevelSelectToGameplay` (fade-out комнаты + загрузка ассетов уровня из ПАК + `VDC_Init`).

31.2 Таблица параметров уровня → геймплей

Источник — `zuma_levels_parameters.xlsx` («Difficulty settings»), генерится в `level_runtime_table.inc` как `LevelSettingsTable` (9 байт/запись):

off	поле	смысл
+0	speed	скорость цепи ×100
+1	start	lead-in: порог fast→normal фазы (стартовое заполнение)
+2	score (word)	gauge target — очки до отсечки хвоста
+4	colors	число цветов шаров (1..6)
+5	repeat	повтор групп
+6	single	одиночные
+7	slowfactor	замедление
+8	partime	par-time бонус

Lookup: `GetCurrentLevelSettingRecord` (board = `CurrentLevel` → TIER по `CurrentDifficulty` → индекс → запись ×9). Геттеры в Core: `GetCurrentSpeed(+0)`, `GetCurrentStart(+1)`, `GetCurrentTargetScore(+2)`, `GetCurrentColors(+4)`, `GetCurrentPartime(+8)`. Загрузка — `VDC_LoadLevelSettings` (зовётся из `VDC_Init`), заполняет runtime-переменные:

- **Цвета:** `VDC_LevelColors` → `VDC_RandomColor` катит `0..N-1` (было захардкожено 6).
- **Скорость цепи:** `VDC_LevelSpeed` (speed×100) + аккумулятор: норм-фаза `accum += speed; при ≥100 → один MoveChain` (= speed/100 продвижений/кадр, значения 0.5–0.9 — аутентичные из `levels.xml` оригинала). Хелпер `VDC_SpeedAdvance` в Core.
- **Lead-in:** `VDC_LevelStart` — порог fast-фазы (был `EQU 35`, теперь per-level 35..60).

31.3 Отсечка хвоста (gauge) — два пойманных бага

`VDC_GaugeScore` копит очки; при достижении per-level target бар полон и спавн хвоста останавливается. Два бага:

1. **HL-clobber:** `CALL GetCurrentTargetScore` глобал `HL` (внутри `GetCurrentLevelSettingRecord`), а следующий `SBC HL, DE` сравнивал мусор (адрес записи!) → `GaugeFull=1` на ПЕРВОМ же match'e. Фикс: переписать `LD HL, (VDC_GaugeScore)` после CALL.
2. **Не сбрасывался:** `VDC_Init` не обнулял `GaugeScore/Shown/Full` → тащились между уровнями. Фикс: добавлены в XOR-блок `VDC_Init`.

HUD-бар нормирован к реальному target (было фикс. 1000): `d = target/63`, `fill = GaugeShown/d`, кламп 63.

31.4 Накопительный счёт adventure

В оригинале Zuma счёт **накопительный** (доп. жизнь за каждые 50k — только если cumulative; подтверждено wiki/GameFAQs). Было: `VDC_Init` обнулял `VDC_PlayerScore` на каждый вход в уровень. Фикс: сброс убран из `VDC_Init` (счёт переносится Win→next); обнуляется только в начале нового прогона (`FadeLevelSelectToGameplay`) и при full-restart (`RestartLevel` при lives=0). Retry (lives>0) и advance — счёт сохраняют.

31.5 Win-flow «LEVEL DONE»

Было: по концу win-анимации сразу `AdvanceToNextLevel`. Стало — диалог как у Game Over:

```

win-аним → DialogState=DLG_WIN_DONE(5)    ; диалог «LEVEL DONE» + статистика + OK
OK       → DialogState=DLG_WIN_FADE(6)    ; пер-кадровый рамп FadeAlpha 0→255
255      → DialogState=0; AdvanceToNextLevel → VDC_Init(state=INTRO)

```

Затемнение — переиспользуем `DrawFadeOverlay` (чёрный RECTS + COLOR_A), вызов добавлен в конец `ZL_DrawFrame` (безвреден при alpha=0).

● **Грабли (исправлено):** `FadeOutRoom` оставляет `FadeAlpha=255`; геймплей его не сбрасывал → `DrawFadeOverlay` чёрнил экран (видны были только часы, т.к. `DrawDebugClock` рисуется поверх overlay). Фикс: `XOR A : LD (FadeAlpha),A` после `FadeOutRoom` в `FadeLevelSelectToGameplay`. Урок: любой вход в геймплей через `FadeOutRoom` обязан сбрасывать `FadeAlpha`.

⚠ **Cross-slot:** ссылки на Core-символы (`DLG_*`, `AdvanceToNextLevel`) из wrapper-кода `main.asm` (`UpdateDialog` / `DrawDialogContent` — не в `MODULE Core`) нужны с префиксом `Core.`.

31.6 Пауза-fade

`VDC_DialogState=4` = пауза-fade (как Win/Game Over/Intro — не-Play: лягушка не стреляет, время не идёт). На «Но» в диалоге паузы → state 4 + `PauseFadeTimers≈74` (~1 с): окно (рамка+заголовок+кнопки) гаснет с убывающей альфой, лягушка рисуется с 1-го кадра fade; по концу → state 0 (PLAY), edge кнопки съеден (нет выстрела). Привязка к кадровому INT (не RTC — он заморожен паузой).

31.7 Интро уровня: динамичный «LEVEL X-X»

Было запечённое «LEVEL 1-1» (атлас). Стало — динамический build «LEVEL N-M» (N= `CurrentLevel` +1, 1..22; M= `CurrentDifficulty` +1, 1..4) шрифтом native через `DrawStr_Scale` + scale-матрица (2× от названия), right-align к x=610, ниже — название уровня. Потребовало добавить `-` в charset `make_font_native.py`.

31.8 Заметки по бюджету страниц

Main1 (`main1_play`, slot 3) почти полон — **16366/16384** байт. Тяжёлую логику (`VDC_LoadLevelSettings`, `VDC_SpeedAdvance`, `VDC_ReadSampleAtHL`) выносили в Core-хелперы, иначе следующая правка в Main1 переполняет страницу → corruption.

31.9 Связано

- `Source/ASM/LevelSelect.asm`, `VDC.asm`, `main.asm`; `level_runtime_table.inc`.
- Memory: `project_zuma_vdac2_level_table_to_gameplay`, `reference_zuma_vdac2_baseline_2026-05-26_v041`, `reference_zuma_vdac2_baseline_2026-05-24_v035`.

Глава 32. Опрос клавиатуры (Mr.Gluk PS/2) и единый глобальный модуль ввода (v044, 2026-05-27)

Задача: дать полноценное управление с PC-клавиатуры (плюс Kempston и мышь) и сделать ввод **настоящему глобальным** — чтобы любая клавиша работала во всех сценах одинаково. Формулировка юзера: «или глобально работает, или глобально глюк». До этого ввод был фрагментирован (лягушка читала матрицу `#FE` + Kempston, диалоги — только `#7FFE`, More Games — ЛКМ+ `#7FFE`), отчего Enter стрелял у лягушки, но не подтверждал в диалоге, а Влево/Вправо в паузе не работали.

32.1 Железо: расширенная PC-клавиатура через Mr.Gluk Z-контроллер

ZX-Evolution/TS-Conf имеют PS/2-порт, обслуживаемый контроллером Mr.Gluk (тот же чип, что и RTC из Главы 28). Доступ — через те же порты:

```
OUT #EFF7, #80      ; enable Mr.Gluk
OUT #DFF7, reg      ; выбор внутреннего регистра
IN/OUT #BFF7        ; данные выбранного регистра
```

Регистр **#F0** — FIFO scancode'ов PS/2 (набор **set-2**). Чтение **#BFF7** достаёт по одному коду; **0** = FIFO пуст. Инициализация (**Input_Init**):

```
LD BC,#EFF7 : LD A,#80 : OUT (C),A      ; enable
LD BC,#DFF7 : LD A,#0C : OUT (C),A
LD BC,#BFF7 : LD A,#01 : OUT (C),A      ; сброс буфера PS/2
LD BC,#DFF7 : LD A,#F0 : OUT (C),A
LD BC,#BFF7 : LD A,#02 : OUT (C),A      ; PS/2 ON
```

Скан-коды set-2 (make-коды), которые отслеживаем: ESC=**#76**, ↑=**#75**, ↓=**#72**, ←=**#6B**, →=**#74**, Enter=**#5A**, Space=**#29**, Q=**#15**, A=**#1C**, O=**#44**, P=**#4D**. Стрелки приходят с префиксом **#E0**, но их коды уникальны → префикс игнорируем. Отпускание клавиши — префикс **#F0**, затем её make-код. Переполнение FIFO — **#FF**.

● **Главная грабля — RTC гасит #EFF7.** Чтение часов (**VDC.ReadRTCSeconds**, Глава 28) в конце своей работы выключает Mr.Gluk (**OUT #EFF7,#00**). Поэтому **Input_Scan** **каждый кадр заново** включает **OUT #EFF7,#80** и заново выбирает регистр **#F0** перед дренажем FIFO. Без этого **IN #BFF7** отдаёт **#FF** (= «пусто» после маски), и клавиатура «немеет» сразу после первого тика часов.

32.2 Архитектура: один резидентный модуль **Input.asm**

Весь ввод вынесен в **Source/ASM/Input.asm** (резидент, **module Core**, slot 1). Раз он в Core — его видят и геймплейный overlay (**#04**), и UI-overlay (**#41**) без переключения страниц. Контракт:

- Input_Init** — раз на старте.
- Input_Scan** — **РОВНО раз за кадр в КАЖДОЙ сцене** (меню, выбор уровня, More Games, геймплей). Обновляет и мышь (**Input.Mouse.UpdateMouseState**), и клавиатуру (дренаж FIFO в флаги). Без него флаги клавиш «застывают».
- Опросы (возвращают **NZ = активно**, объединяя все источники): **Input_Up / Down / Left / Right / Esc / FireKey / Fire**.

Дренаж FIFO — маленький автомат с ограничителем (макс 24 байта за скан, чтобы не зависнуть на мусоре/потоке):

```
.drain: IN A,(C)
        OR A : RET Z          ; 0 = FIFO пуст
        CP #FF : JR Z,.next    ; переполнение -> пропустить
        CP #E0 : JR Z,.next    ; extended-префикс -> игнор
        CP #F0 : JR Z,.set_brk ; break-префикс -> следующий код = отпускание
        CALL Input_SetKey      ; иначе: scancode -> выставить/сбросить флаг
```

Input_SetKey сопоставляет код с адресом байт-флага (**Input_KEsc/KUp/.../KO/KP**) и пишет туда 1 (make) или 0 (break, если перед этим был **#F0**).

32.3 Схема управления и объединение источников

Действие	Клавиатура	Kempston	Мышь
Вверх	↑ Q	Up	—
Вниз	↓ A	Down	—
Влево	← O	Left	—
Вправо	→ P	Right	—
ESC	ESC	— (нет)	—
Огонь	Space Enter	Fire	ЛКМ

О/Р — классическая ZX-раскладка «влево/вправо». Так как **все** сцены читают направление только через `Input_Left` / `Input_Right`, добавление О/Р в эти две функции включило их **сразу везде** (лягушка, диалоги, меню, выбор уровня) — это и есть «глобально».

Огонь намеренно разделён на две функции:

- `Input_FireKey` = Space | Enter | Kempston-Fire (БЕЗ ЛКМ). Для сцен, где ЛКМ обрабатывается отдельно (лягушка-aim, hit-test диалогов/кнопок) — иначе ЛКМ дала бы двойной огонь.
- `Input_Fire` = `Input_FireKey` + ЛКМ. Для сцен без своего hit-test'a (выход из More Games — любой ввод = «дальше»).

32.4 Лягушка: убрали хрупкий хак с FM_EN

Раньше `ZL_AimUpdate` читал матрицу `#FE` (клавиши О/Р/Space) напрямую, для чего **временно гасил** `FM_EN` в регистре `FMADDR` — а `FM_EN` нужен включённым для пейджинга. Это был источник риска (см. историю brick'ов пейджинга). Теперь вращение = `Input_Left` / `Input_Right`, огонь = `Input_FireKey`; матрица `#FE` больше не читается, переключение `FM_EN` удалено (`ZL_FmEnRestore` сведён к `RET`).

32.5 Навигация по меню: детектор фронта `Input_EdgeZ`

Для «одно нажатие = одно действие» (без автоповтора при удержании) добавлен резидентный хелпер. Хитрость: состояние клавиши передаётся **через флаг Z**, а `LD HL, addr` между опросом и вызовом флага не трогает — поэтому Z доходит целым:

```
CALL Input_Up      ; NZ = вверх нажато (флаг Z = состояние)
LD  HL, MenuKbdUpPrev ; LD флаги HE меняет -> Z сохраняется
CALL Input_EdgeZ   ; NZ = ФРОНТ (0->1); обновляет (HL)
JR  NZ, .do_up
```

```
Input_EdgeZ: JR Z, .released
              LD A, (HL) : LD (HL), 1 : XOR 1 : RET ; было 0 -> NZ(фронт); было 1 -> Z
.released:   LD (HL), 0 : XOR A : RET ; отпущено -> Z, сброс
```

● **Грабля — перенос нажатия между сценами.** Если войти в сцену, удерживая Fire (которым выбрали пункт), её `*FirePrev` = 0 → первый же кадр даст ложный фронт → мгновенный «выбор». Фикс: на входе в сцену все `*Prev`-флаги ставим в 1 («как будто нажато») — тогда фронт требует сначала отпустить клавишу. Не-нажатая клавиша сама сбросит флаг в 0 на первом кадре, так что отклика это не задерживает.

32.6 Раскладка по сценам

- **Главное меню** (`MenuKeyboardNav`): Вверх/Вниз ходят по активным кнопкам Adventure→More→Quit (Gauntlet/Options без действия — пропущены), по умолчанию Adventure. По просьбе юзера в этом меню **Вверх=Вправо, Вниз=Влево** (кнопки по диагонали). Выбранная кнопка подсвечивается (hover), Огонь = её нажатие (ставит тот же `*Click` -флаг, что и мышь).
- **Выбор уровня** (`LevelSelectKeyboard`): Вверх/Вниз = сложность (`CurrentDifficulty` 0..3, Вверх легче / Вниз сложнее), Влево/Вправо = уровень (ставит `Back/Next` -Click — дальше штатный `LevelSelectApplyLevelClick` с guard на L1 и заворотом), Огонь = Play, ESC = выход в меню. Вызывается в `LevelSelectUpdateControls` после hit-test'ов мыши, но до `Apply*` -рутин.
- **Диалоги (пауза/win)**: Влево/Вправо — активная кнопка, `Input_FireKey` — подтверждение, ESC в геймплее = нажать кнопку «Меню».

32.7 Связано

- `Source/ASM/Input.asm` (весь модуль), `MainLoop.asm` (`ZL_AimUpdate`), `MenuMain.asm` (`MenuKeyboardNav`), `LevelSelect.asm` (`LevelSelectKeyboard`), `main.asm` (диалоги `.upd_fire` / `.udlg_fire`), `MoreGamesSlot0.asm`.
- Memory: `reference_zuma_vdac2_zxevo_ps2_keyboard` , `reference_zuma_vdac2_mrsluk_rtc` , baseline v044.

33. General Sound на TS-Config: музыка MOD и SFX из PAK

Этот раздел фиксирует рабочую схему, проверенную в Zuma VDAC2 после ошибки `BASS_MusicLoad() error 0x0014` при возврате из игры в главное меню. Ключевой вывод: меню-музыку нельзя без необходимости заново загружать в GS/BASS при каждом возврате. Надёжнее один раз загрузить MOD в GS, сохранить handle, а дальше делать только `STOP_MODULE` перед gameplay и `PLAY_MODULE` при возврате.

33.1. Порты и базовый handshake

В используемой схеме General Sound доступен через два порта:

```
GS_PORT_DATA      EQU #00B3
GS_PORT_CMD       EQU #00BB
GS_CMD_PLAY_MODULE EQU #31
GS_CMD_STOP_MODULE EQU #32
GS_CMD_PLAY_FX    EQU #98
GS_WAIT_TIMEOUT   EQU #FFFF
```

Команда отправляется в `GS_PORT_CMD` , данные - в `GS_PORT_DATA` . После каждого `OUT` обязательно ждать готовности GS по статусным битам command port. Нельзя писать поток байт вслепую: при переполнении/неверном состоянии эмулятор GS/BASS легко уходит в ошибку загрузки или даёт мусор на конце sample.

Минимальные резидентные переменные:

```
GS_Present:      DEFB 0
GS_MenuMusicLoaded: DEFB 0
GS_MenuMusicHandle: DEFB 0
GS_SfxLoaded:    DEFB 0
GS_RamPages:     DEFB 0
```

`GS_MenuMusicLoaded` и `GS_MenuMusicHandle` должны жить в резидентной памяти, потому что overlay загрузчика и gameplay overlay меняются, а состояние GS должно переживать переходы menu -> game -> menu.

33.2. Загрузка MOD музыки

Рабочий порядок для потоковой загрузки menu MOD из `ZUMAAUD.PAK`:

```
CALL GS_Detect
CALL GS_QueryRamPages
CALL GS_InitFxMixer
CALL ZiFi_Init
CALL ZiFi_AudioPakOpen

LD  A, #30      ; Load Module
CALL GS_SendCommandOverlay

LD  A, #D1      ; Open Stream
CALL GS_SendCommandOverlay

CALL GS_StreamAudioPakToDevice

LD  A, #D2      ; Close Stream
CALL GS_SendCommandOverlay

CALL GS_ReadHandleMaybe
LD  A, 1
LD  (GS_MenuMusicLoaded), A
JP  GS_PlayMenuMusic
```

Важная деталь: не читать data port между `Load Module` и `Open Stream`, если протокол не гарантирует готовый handle именно в этот момент. В текущей рабочей версии handle читается только после `Close Stream`. Лишнее чтение перед stream может сдвинуть внутреннее состояние GS/BASS и привести к некорректной повторной загрузке.

33.3. Возврат в меню без повторного BASS_MusicLoad

Ошибка `BASS_MusicLoad() error 0x0014` проявлялась при возврате из игры, когда код снова заходил в ветку загрузки MOD с диска. Исправленная схема:

```

GS_PlayMenuMusic:
    LD    A, (GS_Present)
    OR    A
    RET    Z
    LD    A, (GS_MenuMusicLoaded)
    OR    A
    RET    Z
    LD    A, (GS_MenuMusicHandle)
    OR    A
    RET    Z
    CALL  GS_SendDataResident
    RET    NC
    LD    A, GS_CMD_PLAY_MODULE
    JP    GS_SendCommandResident

GS_StopMenuMusic:
    LD    A, (GS_Present)
    OR    A
    RET    Z
    LD    A, (GS_MenuMusicLoaded)
    OR    A
    RET    Z
    LD    A, GS_CMD_STOP_MODULE
    CALL  GS_SendCommandResident
    RET    NC
    SCF
    RET

```

`GS_StopMenuMusic` не должен очищать `GS_MenuMusicLoaded` и `GS_MenuMusicHandle`. Это не выгрузка модуля, а только остановка проигрывания. При возврате в главное меню код видит, что MOD уже загружен, и вызывает `PLAY_MODULE` по старому handle. Повторный `BASS_MusicLoad()` не выполняется.

33.4. SFX pack и почему нельзя сбрасывать GS перед gameplay

Gameplay SFX грузятся отдельно из `ZUMASND.PAK`, но загрузчик SFX не должен делать `#F3 reset`, если нужно сохранить меню MOD в памяти:

```

OVL_GS_LoadGameplaySoundsMaybe:
    LD    A, (GS_Present)
    OR    A
    RET    Z
    LD    A, (GS_SfxLoaded)
    OR    A
    RET    NZ
    JP    GS_LoadSfxPackNoReset

```

Ранний вариант делал `#F3 reset` для GS с небольшим объёмом RAM и очищал `GS_MenuMusicLoaded` / `GS_MenuMusicHandle`. На возврате это принудительно загоняло меню в повторную загрузку MOD, после чего BASS выдавал `0x0014`. Для текущей цели reset-путь удалён: SFX догружаются без сброса menu MOD.

33.5. Частоты sample и скорость проигрывания

В Zuma VDAC2 большая часть gameplay sample упакована как unsigned PCM с частотой 22050 Hz. Для `chant1` опытным тестом рабочая скорость получилась при payload 8000 Hz. Важно фиксировать частоту на этапе упаковки PAK, а не пытаться компенсировать её случайными параметрами GS:

```
RATE_OVERRIDES = {
    "SND_CHANT1": 8000,
}
```

Если sample играет в два раза медленнее или быстрее, сначала проверить фактический payload в `ZUMASND.PAK` и таблицу размеров/секторов, затем только менять rate override. Нельзя делать вывод по имени исходного WAV: в игре играет не WAV, а уже сконвертированный payload из PAK.

33.6. Устранение щелчков/хвоста sample

Для коротких SFX с шумным хвостом рабочая практика:

- обрезать хвост silence/noise при упаковке;
- добавить короткий fade-out на последние сотни/тысячу sample;
- для проблемного игрового события ставить отложенный `SND_SILENCE`, если GS продолжает держать хвост после завершения эффекта;
- при открытии in-game Menu/ESC глушить текущий SFX через `SND_SILENCE`, чтобы хвост не доигрывал поверх меню.

Пример runtime-глушения:

```
LD A, SND_SILENCE
CALL GS_PlaySfx
```

Для match/destroy эффекта в Zuma VDAC2 дополнительно используется маленький таймер, который через несколько кадров отправляет `SND_SILENCE`. Это прагматичный guard против хвоста sample на конкретном GS/BASS playback path.

33.7. Практические правила

- Загружать MOD один раз, хранить handle в резидентной памяти.
- `STOP_MODULE` - это пауза/остановка playback, не повод очищать loaded/handle.
- Не делать `#F3 reset` перед gameplay SFX, если нужно вернуть menu MOD без повторной загрузки.
- Поток в GS открывать через `#D1`, закрывать через `#D2`, каждый байт отправлять только после handshake.
- Для release хранить отдельно `ZUMAAUD.PAK` (music) и `ZUMASND.PAK` (SFX), чтобы тест на host мог проверить не только SPG, но и звуковые ресурсы.
- При ошибке `BASS_MusicLoad() 0x0014` первым делом проверить, почему код снова вошёл в ветку Load Module, и не был ли сброшен `GS_MenuMusicLoaded`.